

Siemens
Industry
Online
Support

APPLICATION EXAMPLE

Libraries for Communication for SIMATIC Controllers

SIMATIC Controllers, TIA Portal

SIEMENS

Legal information

Use of application examples

Application examples illustrate the solution of automation tasks through an interaction of several components in the form of text, graphics and/or software modules. The application examples are a free service by Siemens AG and/or a subsidiary of Siemens AG ("Siemens"). They are non-binding and make no claim to completeness or functionality regarding configuration and equipment. The application examples merely offer help with typical tasks; they do not constitute customer-specific solutions. **The application examples are not subject to standard tests and quality inspections of a chargeable product and may contain functional and performance defects or other faults and security vulnerabilities. You are responsible for the proper and safe operation of the products in accordance with all applicable regulations, including checking and customizing the application example for your system, and ensuring that only trained personnel use it in a way that prevents property damage or injury to persons. You are solely responsible for any productive use.**

Siemens grants you the non-exclusive, non-sublicensable and non-transferable right to have the application examples used by technically trained personnel. Any change to the application examples is your responsibility. Sharing the application examples with third parties or copying the application examples or excerpts thereof is permitted only in combination with your own products. Any further use of the application examples is explicitly not permitted and further rights are not granted. You are not allowed to use application examples in any other way, including, without limitation, for any direct or indirect training or enhancements of AI models.

Disclaimer of liability

Siemens shall not assume any liability, for any legal reason whatsoever, including, without limitation, liability for the usability, availability, completeness and freedom from defects of the application examples as well as for related information, configuration and performance data and any damage caused thereby. This shall not apply in cases of mandatory liability, for example under the German Product Liability Act, or in cases of intent, gross negligence, or culpable loss of life, bodily injury or damage to health, non-compliance with a guarantee, fraudulent non-disclosure of a defect, or culpable breach of material contractual obligations. Claims for damages arising from a breach of material contractual obligations shall however be limited to the foreseeable damage typical of the type of agreement, unless liability arises from intent or gross negligence or is based on loss of life, bodily injury or damage to health. The foregoing provisions do not imply any change in the burden of proof to your detriment. You shall indemnify Siemens against existing or future claims of third parties in this connection except where Siemens is mandatorily liable.

By using the application examples you acknowledge that Siemens cannot be held liable for any damage beyond the liability provisions described.

Other information

Siemens reserves the right to make changes to the application examples at any time without notice and to terminate your use of the application examples at any time. In case of discrepancies between the suggestions in the application examples and other Siemens publications such as catalogs, the content of the other documentation shall have precedence.

The Siemens terms of use (https://www.siemens.com/global/en/general/terms-of-use.html) shall also apply.

Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit www.siemens.com/cybersecurity-industry.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under https://www.siemens.com/cert.

Table of Contents

1. Introduction 8

2. LCom 10

2.1. Overview 10
2.1.1. Range of Functions 10
2.1.2. Components of the Library 13
2.1.3. Validity 13
2.2. LCom_Communication 14
2.3. PLC Data Types 16
2.4. Integration into the User Project 21
2.5. Error Handling 21
2.5.1. Status Outputs 21
2.5.2. Status Messages 23

3. LFTP 34

3.1. Overview 34
3.1.1. Range of Functions 34
3.1.2. Components of the Library 35
3.1.3. Validity 35
3.2. LFTP_Client 36
3.2.1. Interface Description 36
3.2.2. Error Handling 37
3.3. PLC Data Types 39
3.4. Integration into the user project 39

4. LHTTP 40

4.1. Overview 40
4.1.1. Range of Functions 40
4.1.2. Components of the Library 41
4.1.3. Validity 41
4.2. LHTTP_Get 42
4.2.1. Interface Description 42
4.2.2. Operation 43
4.3. LHTTP_PostPut 45
4.3.1. Interface Description 45
4.3.2. Operation 46
4.4. LHTTP_FindStringInArray 49
4.5. LHTTP_ExtractStringFromArray 49
4.6. LHTTP_ExtractStringFromArrayExt 50

© Siemens 2026

4.7. PLC Data Types 51
4.8. Error Handling 51

5. LMQTT 55

5.1. Overview 55
5.1.1. Range of Functions 55
5.1.2. Components of the Library 55
5.1.3. Validity 55
5.2. LMQTT_Client 56
5.2.1. Interface Description 56
5.2.2. LMQTT_Client_MultipleMsg 59
5.2.3. Error Handling 60
5.3. LMQTT_ConvertToUtf8 63
5.4. PLC Data Types 63
5.5. Integration into the User Project 64

6. LOpcUa 65

6.1. Overview 65
6.1.1. Range of Functions 65
6.1.2. Components of the Library 66
6.1.3. Validity 67
6.2. PubSub via UDP 68
6.2.1. Principle of Operation 68
6.2.2. LOpcUa_PubUdp 70
6.2.3. LOpcUa_SubUdp 71
6.2.4. Error Handling 72
6.2.5. Integration into the User Project 72
6.3. PubSub via MQTT 73
6.3.1. Principle of Operation 73
6.3.2. LOpcUa_PubMqtt 75
6.3.3. LOpcUa_SubMqtt 76
6.3.4. LOpcUa_PubMqttJson 77
6.3.5. LOpcUa_SubMqttJson 79
6.3.6. Error Handling 80
6.3.7. Integration into the User Project 80
6.4. PLC Data Types 80

7. LSNMP 86

7.1. Overview 86
7.1.1. Range of functions 86
7.1.2. Function blocks 87

7.1.3. Validity ... 87
7.2. LSNMP_Get ... 88
7.3. LSNMP_GetBulk ... 89
7.4. LSNMP_Set ... 91
7.5. LSNMP_GetOrSet ... 92
7.6. LSNMP_ReceiveTrap ... 96
7.7. LSNMP_SendTrap ... 97
7.8. PLC Data Types ... 98
7.9. Integration into the user program ... 99
7.10. Error Handling ... 100

8. LSNTP ... 102
8.1. Overview ... 102
8.1.1. Range of Functions ... 102
8.1.2. Components of the Library ... 103
8.1.3. Validity ... 103
8.2. LSNTP_Server ... 104
8.2.1. Interface Description ... 104
8.2.2. Error Handling ... 105

9. LSQL ... 106
9.1. Overview ... 106
9.1.1. Range of functions ... 106
9.1.2. Components of the library ... 106
9.1.3. Validity ... 106
9.2. LSQL_Microsoft ... 107
9.2.1. Interface description ... 107
9.2.2. Error handling ... 108
9.3. LSQL_AnalyzeTokens ... 110
9.3.1. Interface description ... 110
9.3.2. Error handling ... 111
9.4. PLC Data Types ... 112
9.5. Integration into the User Project ... 114

10. LSyslog ... 115
10.1. Overview ... 115
10.1.1. Range of Functions ... 115
10.1.2. Components of the Library ... 116
10.1.3. Validity ... 116
10.2. LSyslog_Send ... 117
10.2.1. Interface Description ... 117

10.2.2. Error Handling ... 118
10.3. PLC Data Types ... 119
10.4. Integration into the User Project ... 119

11. LCoAP ... 120

11.1. Overview ... 120
11.1.1. Range of functions ... 120
11.1.2. CoAP Protocol Overview ... 120
11.1.3. Components of the library ... 122
11.1.4. Validity ... 123
11.2. LCoAP_ClientServer ... 124
11.2.1. Interface description ... 124
11.2.2. Operation ... 125
11.2.3. Error handling ... 129
11.3. LCoAP_EncodeHeader ... 130
11.3.1. Interface description ... 130
11.3.2. Operation ... 131
11.3.3. Auxiliar functions ... 132
11.3.3.1. LCoAP_EncodeOptionLengthDelta ... 132
11.3.3.2. LCoAP_EncodeOptionString ... 133
11.3.3.3. LCoAP_EncodeOptionUInt ... 134
11.3.3.4. LCoAP_EncodePayloadMarker ... 135
11.3.4. Error handling ... 135
11.4. LCoAP_DecodeHeader ... 136
11.4.1. Interface description ... 136
11.4.2. Operation ... 137
11.4.3. Auxiliar functions ... 138
11.4.3.1. LCoAP_DecodeType ... 138
11.4.3.2. LCoAP_DecodeCode ... 138
11.4.3.3. LCoAP_DecodeToken ... 139
11.4.3.4. LCoAP_DecodeOptionLengthDelta ... 139
11.4.3.5. LCoAP_DecodeOptionString ... 140
11.4.3.6. LCoAP_DecodeOptionUInt ... 141
11.4.3.7. LCoAP_DecodePayloadMarker ... 142
11.4.4. Error handling ... 143
11.5. Utilities ... 144
11.5.1. LCoAP_GenerateMessageID ... 144
11.5.2. LCoAP_GenerateToken ... 144
11.5.3. LCoAP_TokenCompare ... 144
11.6. PLC Data types ... 146

12. LDNS 150
12.1. Overview 150
12.1.1. Range of functions 150
12.1.2. Components of the library 151
12.1.3. Validity 151
12.2. LDNS_ResolveFqdn 152
12.2.1. Interface Description 152
12.2.2. Error handling 153
12.3. PLC Data Types 154
12.4. Integration into the User Project 154

13. Master Copies 155
13.1. OUC Master Copies 155
13.1.1. Overview 155
13.1.1.1. Range of Functions 155
13.1.1.2. Components 155
13.1.1.3. Validity 156
13.1.2. IsoOnTcpTemplate 157
13.1.3. TcpTemplate 159
13.1.4. TcpSecTemplate 161
13.1.5. UdpTemplate 163
13.1.6. Error Handling 165
13.2. OPC UA structured data types 165
13.3. OPC UA methods-templates 166
13.4. LSQL_Examples 166
13.5. LDNS Example 166

14. Useful Information 167
14.1. Libraries in TIA Portal 167
14.2. Diagnostics 168

15. Appendix 170
15.1. Service and Support 170
15.2. Links and Literature 171
15.3. Change Documentation 172

1. Introduction

Overview

The Libraries for Communication are a collection of blocks for various communication tasks, functions, and protocols for SIMATIC Controllers.

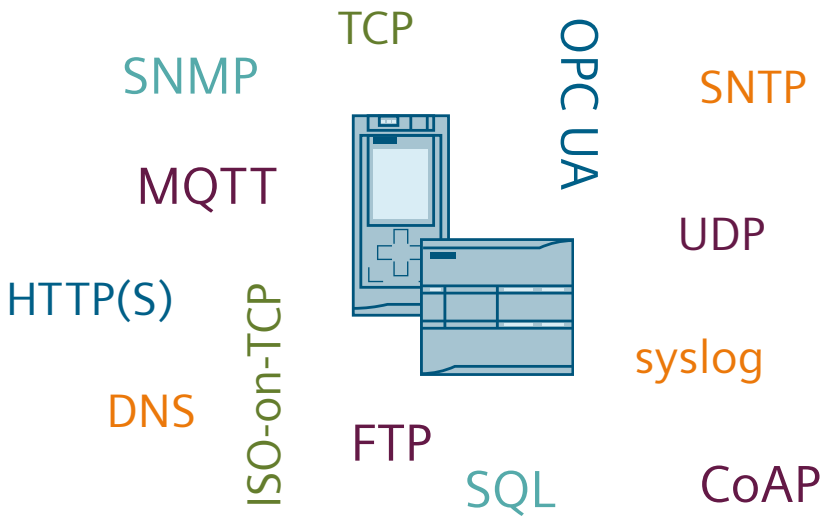


Figure 1-1: Libraries for Communication for SIMATIC Controllers

Range of Functions

The following libraries are included:

- LCom:
  This library enables communication based on TCP and provides additional communication functionalities using its own protocol (see chapter 2).
- LFTP:
  With this library, a controller can act as an FTP client (see chapter 3).
- LHTTP:
  This library enables data exchange with a web server in the local network or on the internet via HTTP or HTTPS (see chapter 4).
- LMQTT:
  This library enables the communication of a controller as MQTT Client (see chapter 5).
- LOpcUa:
  This library provides function blocks for OPC UA PubSub communication (see chapter 6).
- LSNMP:
  This library can be used to monitor and control SNMP-enabled network components from a controller or to send messages to a network management system (see chapter 7).
- LSNTP:
  With this library, a controller can act as an SNTP server to synchronize the time throughout different areas of the system (see chapter 8).
- LSQL:
  This library enables interacting with an SQL database directly from a user program of a controller (see chapter 9).
- LSyslog:
  This library allows sending syslog messages to a syslog server over UDP or TCP with optional TLS encryption (see chapter 9.1).

© Siemens 2026

- LCoAP:
  This library provides a set of function blocks for implementing the CoAP protocol in the controller (see chapter 11).
- LDNS:
  This library extends the built-in DNS resolution of the Ethernet interfaces with a DNS resolver (see chapter 12).

In addition, the library provides master copies that you can use to easily implement your own communication functions:

- Master copies for communication via TCP, UDP, and ISO-on-TCP (see chapter 13.1)
- Master copies for OPC UA methods
- Master copies for common OPC UA structured data types
- Constants for OPC UA status codes

Application

These libraries are available for TIA Portal V18 and higher.

All libraries included are valid for SIMATIC S7-1500 and S7-1200 controllers. If the respective library is also valid for other controllers, this is described in the section for the corresponding library.

© Siemens 2026

2. LCom

2.1. Overview

2.1.1. Range of Functions

Possible applications for the use of the LCom library

The "LCom_Communication" function block of the LCom library is used to establish a point-to-point full duplex connection via Industrial Ethernet, based on the TCP standard.

The function block can be used for standard TCP communication to other devices (e.g. camera, controller).

Since the range of functions of the TCP transport protocol is not sufficient for many applications in the automation sector, a separate protocol (LCom protocol) has been defined and implemented in the LCom block library. The LCom protocol enables additional communication functionalities.

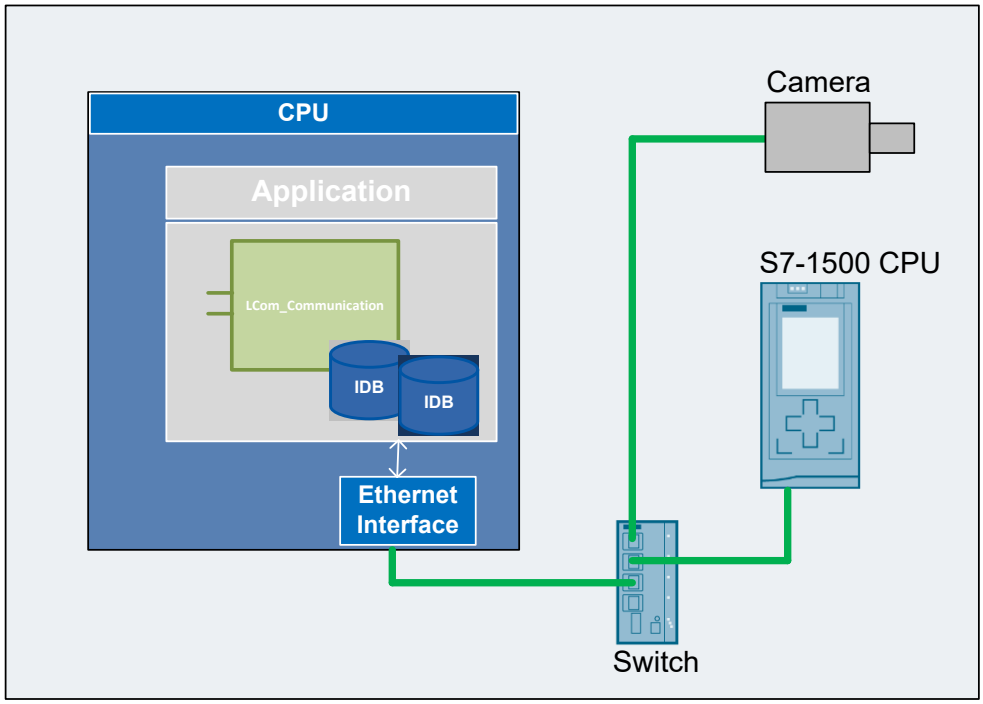


Figure 2-1: Application scenarios

© Siemens 2026 |

Some scenarios for a possible use of the LCom library are shown below:

Scenario 1

The "LCom_Communication" function block is used for standard TCP communication. The TCP transport protocol ensures that a continuous flow of data is transferred. TCP is not packet-oriented and, therefore, does not permit the transmission of data records with a defined overall length.

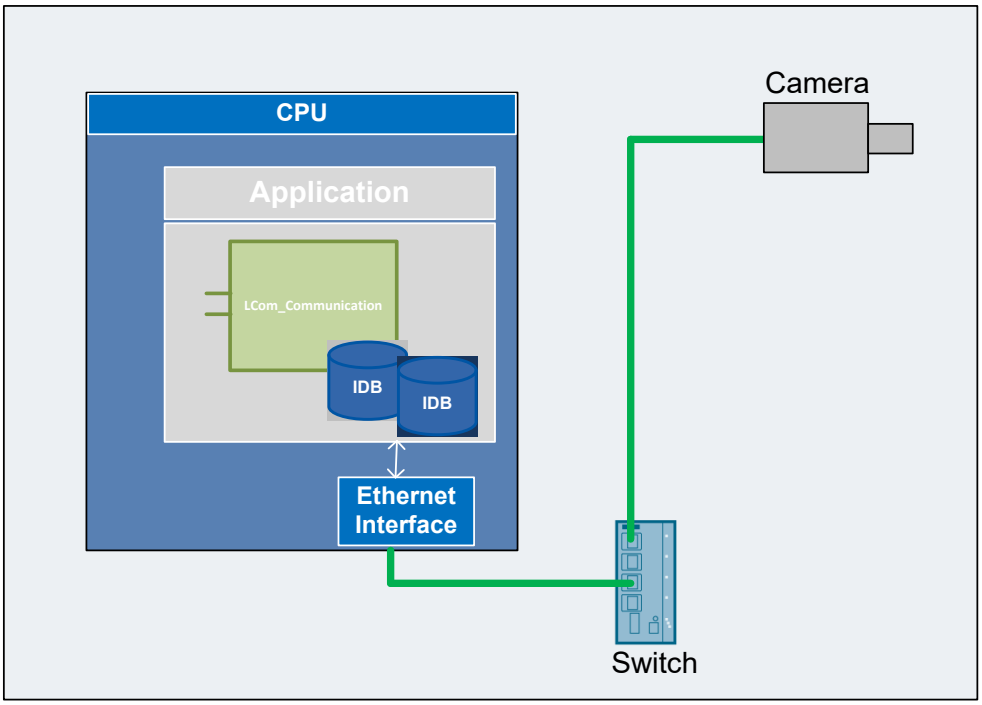


Figure 2-2: LCom_Communication – Standard TCP communication

The LCom library offers the following advantages:

- The user does not need to know and program the OUC system function to establish, send/receive, and terminate a connection
- In the event of errors/faults, the function block automatically closes the connection and attempts to reestablish it
- Cyclic data transmission, the user defines the cycle time
- One time data transmission
- Response from sender/receiver about successfully transmitted data at application level
- The function block contains a data structure for diagnostics

© Siemens 2026

Scenario 2

When using the LCom protocol, data records of a defined total length can be sent and received consistently by the partner. In order to quickly detect a connection failure, cyclic vital signs are sent. The cycle time of the periodic vital signs is specified by the user.

To synchronize the time of two controllers, the current time of one controller can be sent to the partner and transferred there as the system time.

To use the additional communication functionalities, the communication partner (e.g. S7-1500 CPU) must support the LCom protocol.

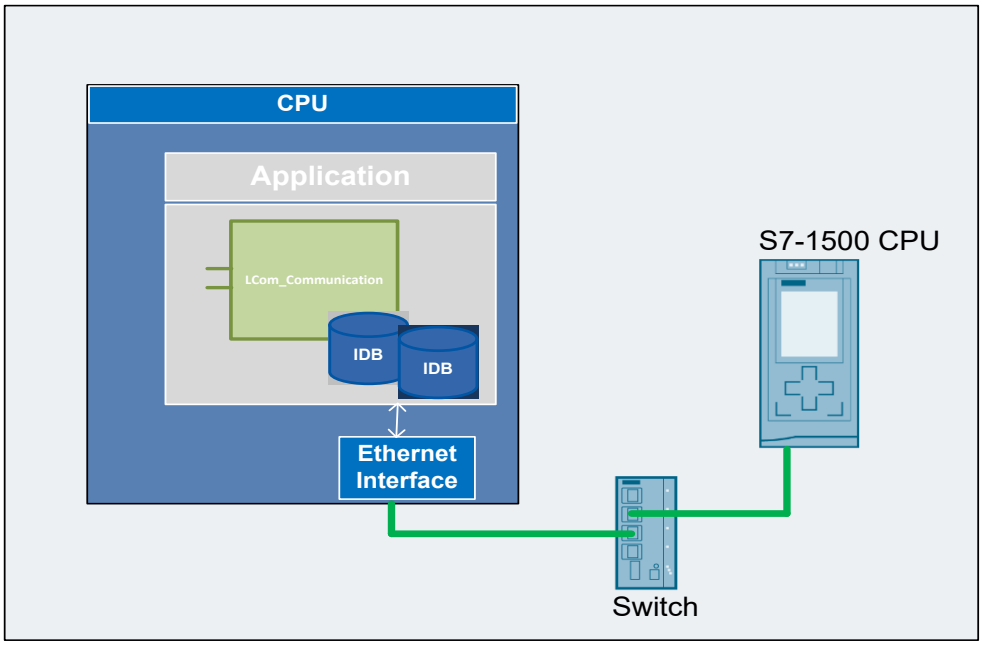


Figure 2-3: LCom_Communication – LCom protocol

The LCom library offers the following advantages:

- The user does not need to know and program the OUC system function to establish, send/receive, and terminate a connection
- In the event of errors/faults, the function block automatically closes the connection and attempts to reestablish it
- Cyclic data transmission, the user defines the cycle time
- One time data transmission
- Consistent data transmission
- Synchronizing the communication parameters
- Monitoring of the communication link by cyclically sending vital signs (fast response times in case of link failure). For pure TCP communication, this is typically in the range of seconds.
- Data records with defined length up to 64 kB with LCom protocol V1
- Data records with defined length up to 16 MB with LCom protocol V2
- Response from sender/receiver about successfully transmitted data at application level
- Simple time synchronization
- The function block contains a data structure for diagnostics

© Siemens 2026

2.1.2. Components of the Library

The "LCom" library contains the following objects.

Function blocks

Table 2-1: Function blocks of the library

Name	Version	Description
LCom_Communication	V2.1.0	Controls the communication between the controller and the partner.

PLC data types

Table 2-2: PLC library data types

Name	Version	Description
LCom_typeConfig	V2.0.1	Contains the necessary communication parameters.
LCom_typeDiagnostics	V2.0.1	Provides a detailed diagnostic structure.

Master copies

An example how a configuration and execution of a communication with user defined data in connection with the "LCom_Communication" FB looks like is provided in the master copies templates.

In addition, constants are provided for global usage. With help of that for example, the status and error codes of "LCom_Communication" FB can be processed more readable in user application program.

Table 2-3 Master copies templates

Name	Type	Description
LCom_Example	OB	Implementation of an example communication with LCom
LCom_ExampleComData	DB	Data of example communication
LCom_ExampleComBuffers	DB	Send- and receive buffer
LCom_typeComUserData	Data type	User defined data type of example communication
LCom_Constants	PLC tags	LCom constants (i.e. status and error codes) for global usage in user application program

2.1.3. Validity

This library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.4
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- CP 1543-1, CP 1542SP-1, CP 1542SP-1 IRC, CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V2.0

© Siemens 2026

2.2. LCom_Communication

Parameters

Figure 2-4: LCom_Communication

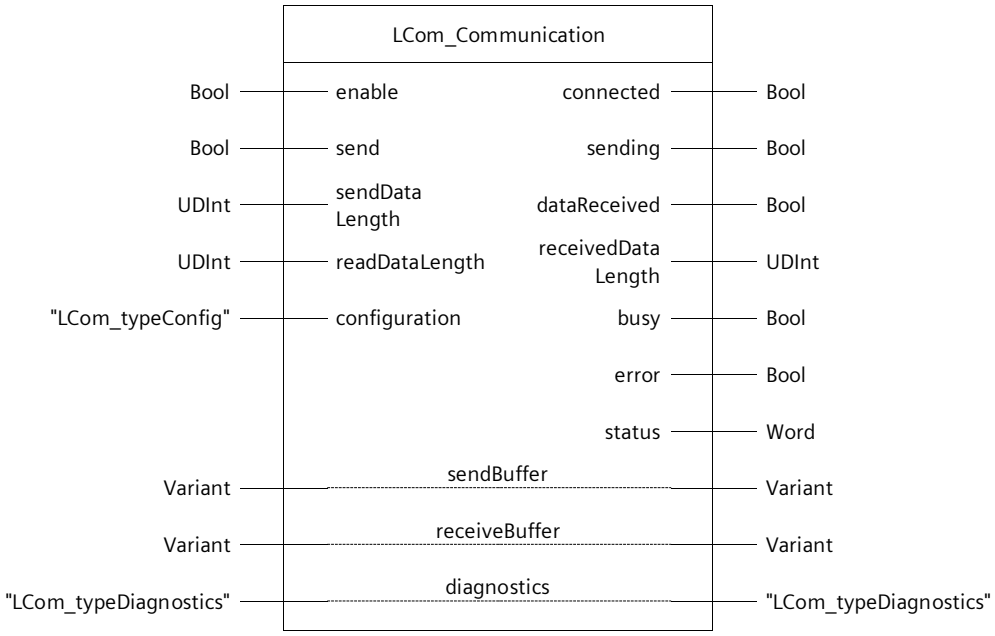


Table 2-4: Parameters of LCom_Communication

Name	Declaration	Data type	Comment
enable	Input	Bool	• TRUE: Starts the FB with a rising edge. The connection to the partner is established automatically. • FALSE (default): With a falling edge, the connection is broken.
send	Input	Bool	• TRUE: Sends the data connected to the "sendBuffer" input parameter. • FALSE (default): Data that is connected at the "sendBuffer" input parameter is not sent.
sendDataLength	Input	UDInt	Data length to be sent in bytes. (default: 4294967295)
readDataLength	Input	UDInt	Without LCom protocol, ("configuration.connection.comService" = "LCOM_TCP_CONNECTION"): • 0: Data from the TCP buffer is not read. FB, therefore, does not receive any data from the partner. • 1..4294967294: Number of bytes that must be received by the interface before the output tag "dataReceived" = TRUE. • 4294967295, 16#FFFFFF (default): All data available at the interface is read (ad hoc mode). With LCom protocol, ("configuration.connection.comService" = "LCOM_LCOM_CONNECTION"): • 0: Data from the TCP buffer is not read. The FB therefore does not receive any data from the partner. • 1..4294967295: Not relevant for the receiving behaviour.
configuration	Input	"LCom_typeConfig"	FB configuration

© Siemens 2026

Name	Declaration	Data type	Comment
connected	Output	Bool	Without LCom protocol, ("configuration.connection.comService" = "LCOM_TCP_CONNECTION"): • TRUE: The TCP connection to the partner is established. • FALSE: The TCP connection to the partner is not established. With LCom protocol, ("configuration.connection.comService" = "LCOM_LCOM_CONNECTION"): • TRUE: TCP connection to partner is established and configuration data is negotiated successfully • FALSE: TCP connection to partner is not established or waiting for negotiation of configuration data.
sending	Output	Bool	• TRUE: Sends the data connected to the "sendBuffer" input parameter. Do not change send data. • FALSE: Data that is connected at the "sendBuffer" input parameter is not sent.
dataReceived	Output	Bool	• TRUE: New data was received. The value is present for one cycle. • FALSE: No data is available for the user.
receivedDataLength	Output	UDInt	• Returns the received data length in bytes.
busy	Output	Bool	• TRUE: The block processing is done automatically. No intervention is required from the user. • FALSE: There is no block processing. Intervention is required from the user. Passing parameters may be incorrect. More detailed information is provided by the output parameter "status" or the diagnostic buffer.
error	Output	Bool	• TRUE: An error has occurred. More detailed information is provided by the output parameter "status" or the diagnostic buffer. • FALSE: No error has occurred.
status	Output	Word	Status of the FB, see chapter 2.5.2
sendBuffer	InOut	Variant	Send data (array of bytes)
receiveBuffer	InOut	Variant	Receive data (array of bytes)
diagnostics	InOut	"LCom_typeDiagnostics"	Diagnostics information

© Siemens 2026

2.3. PLC Data Types

LCom_typeConfig

The following table shows the structure of the PLC data type "LCom_typeConfig".

Table 2-5: PLC data type LCom_typeConfig

Name	Type	Start value	Comment
connection	Struct		
interfaceID	HW_ANY	64	Hardware identifier of the local interface • 0: if connectType = 10 (QDN)
connectionID	CONN_ANY	16#0FFF	Reference to the connection 16#0001..16#0FFF
comService	USInt	2	Configuration of the communication protocol: • 1: The standard TCP protocol is used. • 2 (default): The LCom protocol is used. With the LCom protocol, additional communication functionalities are enabled
isClient	Bool	FALSE	• TRUE: Active connection establishment as TCP client. • FALSE: Passive connection establishment as TCP server.
connectType	USInt	1	Connection type: • 1 (default) = IPv4 • 10 = QDN (Qualified Domain Name)
localPort	UInt	3456	Local port, see system function TCON.
partnerPort	UInt	3456	Only when the connection is active ("configuration.connection.isClient" = TRUE): Port of the partner-side, see system function TCON.
partnerIP	IP_V4	-	IP address of the partner
partnerQDN	String [254]	''	Fully qualified domain name Must finish with "."
acceptUnknownPartner	Bool	TRUE	Only for passive connection establishment ("configuration.connection.isClient" = FALSE): • TRUE: Previously not configured connection partner is accepted • FALSE: Only the configured connection partner is accepted
lifeSignCycleTime	Time	T#1s	Vital signs cycle T#1ms..T#24d20h30m20s630ms
sender	Struct		
cycleTime	Time	T#1s	Send cycle T#0ms..T#24d20h30m20s630ms

© Siemens 2026

Name	Type	Start value	Comment
ackTimeout	Time	T#1s	Only relevant when using the LCom protocol ("configuration.connection.comService" = 2): The tag "ackTimeout" determines the monitoring time after which an acknowledgement of the sent packet is expected at the latest. If this period is exceeded, the connection is closed and re-established. T#1ms..T#24d20h30m20s630ms
timeSync	Struct		Only relevant when using the LCom protocol ("configuration.connection.comService" = 2)
usePartnerTimestamps	Bool	FALSE	• TRUE: The received time is taken over as system time • FALSE: Received time telegrams are ignored
sendMode	USInt	0	Send mode time synchronization: • 0: Inactive • 1: Cyclic time synchronization • 2: Time when the time synchronization should take place
cycleTime	Time	T#1h	Only relevant if "sendMode" = 1; Send cycle time of the timestamps T#1ms..T#24d20h30m20s630ms
sendAtTimeOfDay	Time_Of_Day	TOD#05:00:0.000	Only relevant with "sendMode" = 2; Time at which a time synchronization telegram is sent.

LCom_typeDiagnostics

The following table shows the structure of the PLC data type "LCom_typeDiagnostics".

Table 2-6: PLC data type LCom_typeDiagnostics

Name	Type	Comment
localConfig	Struct	
connection	Struct	
interfaceID	HW_ANY	Hardware identifier of the local interface
connectionID	CONN_ANY	Reference to the local connection
comService	USInt	• 1: The standard TCP protocol is used • 2: The LCom protocol is used
isClient	Bool	• TRUE: Active connection establishment as TCP client • FALSE: Passive connection establishment as TCP server
connectType	USInt	• 1 = IPv4 • 10 = QDN (Qualified Domain Name)
localPort	UInt	Local port
localIP	IP_V4	Local IP address
partnerPort	UInt	Partner-side port
partnerIP	IP_V4	IP address of the partner

© Siemens 2026

Name	Type	Comment
partnerQDN	String [254]	Fully qualified domain name
acceptUnknownPartner	Bool	• TRUE: Previously not configured connection partner is accepted • FALSE: Only the configured connection partner is accepted
useLComProtocol	Bool	• TRUE: The LCom protocol is used • FALSE: The LCom protocol is not used
lifeSignCycleTime	Time	Local vital signs cycle
sender	Struct	
cycleTime	Time	Local send cycle
ackTimeout	Time	The tag *ackTimeout* determines the local monitoring time after which an acknowledgement of the sent packet is expected at the latest. If this time is exceeded, the connection is closed and re-established.
timeSync	Struct	
usePartnerTimestamps	Bool	• TRUE: The received time is adopted as the system time. • FALSE: Received time telegrams are ignored.
sendMode	USInt	Send mode time synchronization: • 0: Inactive • 1: Cyclic time synchronization • 2: Time when the time synchronization should take place
cycleTime	Time	Send cycle time of the timestamps
sendAtTimeOfDay	Time_Of_Day	Time at which a synchronization telegram is sent
sizeOfSendBuffer	UDInt	Size of the local send buffer in bytes
sizeOfReceiveBuffer	UDInt	Size of the local receive buffer in bytes
partnerConfig	Struct	
connnection	Struct	
lifeSignCycleTime	Time	Partner vital signs cycle
sender	Struct	
cycleTime	Time	Partner send cycle
ackTimeout	Time	The tag *ackTimeout* determines the partner monitoring time after which an acknowledgement of the sent packet is expected at the latest. If this time is exceeded, the connection is closed by the partner and re-established.
timeSync	Struct	
usePartnerTimestamps	Bool	• TRUE: The received time is taken over as system time • FALSE: Received time telegrams are ignored

© Siemens 2026 |

Name	Type	Comment
sendMode	USInt	Send mode time synchronization: • 0: Inactive • 1: cyclic time synchronization. • 2: Time when the time synchronization should take place
sizeOfSendBuffer	UDInt	Size of the send buffer in bytes of the partner
sizeOfReceiveBuffer	UDInt	Size of the receive buffer in bytes of the partner
statistics	Struct	Statistics
avgCallCycle	Real	Mean value between two FB calls [ms]
avgReceiveMsgCycle	Real	Mean value in which cycle data was received [ms]
maxReceiveMsgCycle	Time	Maximum occurred receive cycles
LComProtocolUsed	Bool	• TRUE: The LCom protocol is used. With the LCom protocol additional communication functionalities are enabled • FALSE: The TCP transport protocol is used
activeLComVersion	USInt	Active LCom protocol version
avgMsgSendingTime	Real	Average value of how long a sending process (*sending* = TRUE) takes [ms]
maxMsgSendingTime	Time	Maximum value of a sending process (*sending* = TRUE) [ms]
avgMsgReceivingTime	Real	Mean value of how long a receiving process takes [ms]
maxMsgReceivingTime	Time	Maximum value of a receiving process [ms]
numberOfSentMessages	UDInt	Number of messages sent, since the L/H edge at the *enable* input
numberOfReceivedMessages	UDInt	Number of received messages, since the L/H edge at the *enable* input
totalAckTimeouts	UInt	Number of times the monitoring time was exceeded (diagnostics.localConfig.sender.ackTimeout)
totalSendCycleViolations	UInt	Number of times the send cycle is exceeded (diagnostics.localConfig.sender.cycleTime)
totalReceiveCycleViolations	UInt	Number of times the receive cycle is exceeded (*diagnostics.partnerConfig.sender.cycleTime*)
totalReconnects	UInt	Number of times the FB has reestablished the TCP connection
lastConnect	DTL	Time when the connection was successfully established to the partner
lastTimeSync	DTL	Time of the last time synchronization
bufferIndex	USInt	Index to the last diagnostic entry
buffer	Array[0..63] of Struct	Diagnostic buffer
status	Word	Status message from FB

© Siemens 2026

Name	Type	Comment
timestamp	DTL	Timestamp at appearance of the message
isActive	Bool	• TRUE: State is still available • FALSE: State is no longer available
subFunctionErrorID	Word	If necessary, Return value of a system function
additionalValue1	Real	Additional value 1
additionalValue2	Real	Additional value 2
additionalValue3	Real	Additional value 3
additionalValue4	Time	Additional value 4

© Siemens 2026 |

2.4. Integration into the User Project

You will find an application example for integrating the LCom blocks into your user application in the master copies of this library.

Additionally, you will find a detailed application example for the integration of the library into your user project in the Siemens Industry Online Support:

https://support.industry.siemens.com/cs/ww/en/view/48955385

2.5. Error Handling

2.5.1. Status Outputs

An error is indicated by setting the boolean tag "error". A distinction must be made between whether the cause of the error can be remedied by the user or by the block. If it is an error that can be corrected by the block, "busy" remains set. If an error occurs that must be corrected by the user, "busy" is reset (block processing deactivated).

The following figure shows the two scenarios as a signal flow diagram.

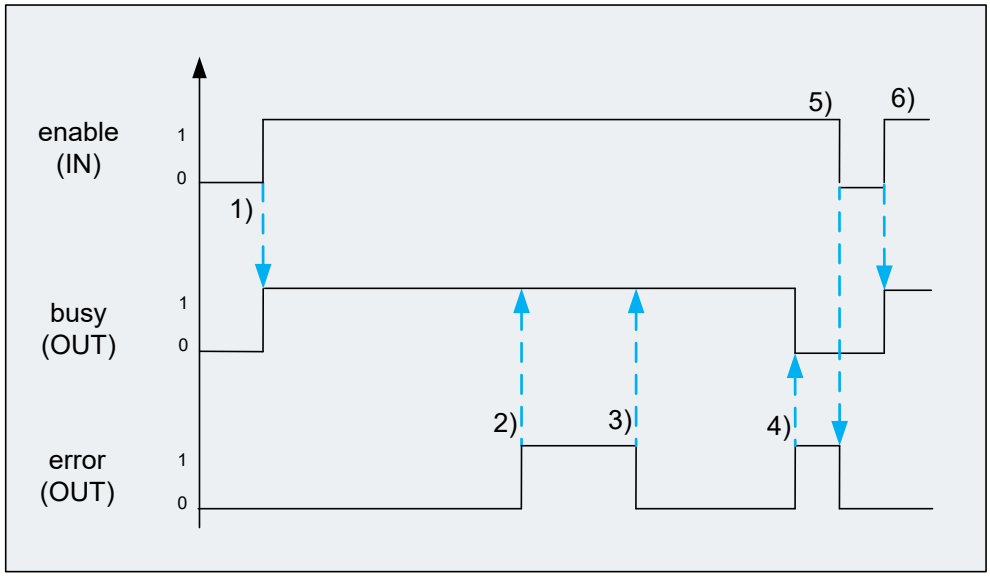


Figure 2-5 Signal flow diagram error/busy

1. "busy" is set with a rising edge of "enable".
2. An error occurs and "error" is set. Since this is an error that can be corrected by the block itself, "busy" remains set.
3. After correcting the cause of the error (e.g. re-establishing the connection), "error" is reset.
4. An error that can only be corrected by the user occurs. Here "error" is set and "busy" is reset.
5. The pending error, which must be remedied by the user, can only be cleared with a falling edge at "enable".
6. The block is started again with a rising edge of "enable".

© Siemens 2026 |

Output parameters diagnostics

In the output parameter "diagnostics", there are different substructures that are defined by the PLC data type "LCom_typeDiagnostics". The diagnostic buffer is discussed below. The complete description of the PLC data type "LCom_typeDiagnostics" can be found in chapter 2.3.

Various status and error messages are entered in the diagnostic buffer. The array is set to 64 entries. This array operates as a ring buffer. The tag "bufferIndex" points to the last (current) entry.

Each entry consists of the following elements:

- Status or error number
- Date and time of occurrence
- Current state of the error

The state when entering the buffer is always active. When resetting the error, the status in the buffer is set to inactive. The structure contains the return value of the system function as well as four additional values that can contain detailed information depending on the error that occurred.

Table 2-7 Structure of the diagnostic buffer

Name	Data type	Description
bufferIndex	USInt	Index to last entry
buffer	Array [0..63] of Struct	Diagnostic buffer
status	Word	Status from FB
timestamp	DTL	Timestamp at appearance of the message
isActive	Bool	• TRUE: State is still present. • FALSE: State is no longer present.
subFunctionErrorID	Word	If necessary, Return value of a system function
additionalValue1	Real	Additional value 1
additionalValue2	Real	Additional value 2
additionalValue3	Real	Additional value 3
additionalValue4	Time	Additional value 4

© Siemens 2026

2.5.2. Status Messages

Table 2-8: Status messages

Code	Meaning
16#7000	No job in process. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4* T#0ms
16#7001	First call after receipt of a new job (rising edge at *enable*). Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4* T#0ms
16#7002	Follow-up call during active FB processing. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4* T#0ms
16#7003	FB tries to connect as TCP client. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4* T#0ms
16#7004	FB is configured as TCP server and waits for a TCP client request. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1...3:* 0.0 *additionalValue4* T#0ms
16#7005	The TCP connection to the partner is established. Diagnostic entry:

© Siemens 2026 |

Code	Meaning	
	timestamp:	Time of the event
	isActive:	State
	subFunctionErrorID:	16#0
	additionalValue1..3:	0.0
	additionalValue4	T#0ms

© Siemens 2026 |

Code	Meaning
16#7006	The FB has successfully exchanged configuration data with the partner via LCom protocol V1. The maximum possible data length is 64 KByte. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4:* T#0ms
16#7007	The FB has successfully established the connection and exchanged the configuration data (LCom protocol V2) with the partner. The maximum possible data length is 16 MByte. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4:* T#0ms
16#7008	The FB terminates the connection. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4* T#0ms
16#7008	The FB terminates the connection. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3:* 0.0 *additionalValue4* T#0ms
16#7600	The specified send length (*sendDataLength*) is too large and is automatically limited by the FB. The maximum possible send length is determined by the size of the own send buffer (*sendBuffer*). Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* sendDataLength *additionalValue2:* sendBuffer *additionalValue3:* 0.0 *additionalValue4:* T#0ms
16#7610	The specified time (*configuration.connection.lifeSignCycleTime*) for connection monitoring is too large and is automatically limited to 65535 ms.

© Siemens 2026 |

Code	Meaning
	Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* Limitation to 65535 *additionalValue2..3:* 0.0 *additionalValue4:* Value of the user
16#7611	The specified time for the send cycle (*configuration.sender.cycleTime*) is too large and is automatically limited to 65535 ms. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* Limitation to 65535 *additionalValue2..3:* 0.0 *additionalValue4:* Value of the user
16#7612	The configured monitoring time (*configuration.sender.ackTimeout*) is too large and is automatically limited to 65535 ms Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* Limitation to 65535 *additionalValue2..3:* 0.0 *additionalValue4:* Value of the user
16#7613	The configured send cycle (*configuration.sender.CycleTime*) cannot be kept because the previous sending process has not yet been completed. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* Value of the user *additionalValue2..3:* 0.0 *additionalValue4:* Previous time of the previous send job
16#7614	When using the LCom protocol (*configuration.connection.comService = 2*): The specified send length (*sendDataLength*) is larger than the receive buffer of the partner. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* sendDataLength *additionalValue2:* Receive buffer of the partner-side *additionalValue3:* 0.0 *additionalValue4:* T#0ms

© Siemens 2026 |

Code	Meaning
	Without using the LCom protocol (*configuration.connection.comService = 1*): The specified *readDataLength* at the FB is larger than the specified receive buffer (receiveBuffer). The *readDataLength* is limited to the size of the specified receive buffer. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* User-defined receive buffer *additionalValue2:* readDataLength *additionalValue3:* 0.0 *additionalValue4:* T#0ms

© Siemens 2026 |

Code	Meaning
16#7615	No time synchronization is performed. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..4:* 0.0
16#8200	The specified communication service *(configuration.connection.comService)* is invalid → see system function TCON. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1* Value of the user *additionalValue2...3:* 0.0 *additionalValue4:* T#0ms
16#8201	The specified connection reference (*configuration.connection.connectionID*) is invalid → see system function TCON. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1* Value of the user *additionalValue2..3:* 0.0 *additionalValue4:* T#0ms
16#8202	The specified hardware identifier for the local interface (*configuration.connection.interfaceID*) is invalid → see system function TCON. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1* Value of the user *additionalValue2..3:* 0.0 *additionalValue4:* T#0ms
16#8203	The specified local port (*configuration.connection.localPort*) is not allowed → see system function TCON. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* TCON status *additionalValue1* Value of the user *additionalValue2..3:* 0.0 *additionalValue4:* T#0ms
16#8204	The specified partner IP address (*configuration.connection.partnerIP*) is invalid → see system function TCON.

© Siemens 2026 |

Code	Meaning
	Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* TCON status *additionalValue1* Value of the user *additionalValue2..3:* 0.0 *additionalValue4:* T#0ms
16#8205	The specified mode for time synchronization (*configuration.timeSync.sendMode*) is invalid. Diagnostics entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1* Value of the user *additionalValue2..3:* 0.0 *additionalValue4:* T#0ms
16#8206	The specified cycle time for time synchronization (*configuration. timeSync.cycleTime*) is invalid. Diagnostics entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3* 0.0 *additionalValue4:* Value of the user
16#8207	The specified time for the time synchronization(*configuration.timeSync.sendAtTimeOfDay*) is invalid. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3* 0.0 *additionalValue4:* T#0ms
16#8208	The value passed to the input tag *sendBuffer* is not of type ARRAY of BYTE and is invalid. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3* 0.0 *additionalValue4:* T#0ms
16#8209	The value passed to the output tag *receiveBuffer* is not of type ARRAY of BYTE and is invalid. Diagnostic entry:

© Siemens 2026 |

Code	Meaning	
	timestamp:	Time of the event
	isActive:	State
	subFunctionErrorID:	16#0
	additionalValue1..3	0.0
	additionalValue4:	T#0ms
16#8210	The specified time (*configuration.connection.lifeSignCycleTime*) for connection monitoring is invalid: Diagnostic entry:	
	timestamp:	Time of the event
	isActive:	State
	subFunctionErrorID:	16#0
	additionalValue1..3	0.0
	additionalValue4:	Value of the user

© Siemens 2026 |

Code	Meaning
16#8211	The specified send cycle (*configuration.sender.cycleTime*) is invalid. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3* 0.0 *additionalValue4:* Value of the user
16#8212	The configured monitoring time (*configuration.sender.ackTimeout*) is invalid. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1..3* 0.0 *additionalValue4:* Value of the user
16#8213	The configured connection type (*configuration.connection.connectType*) is invalid Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1* Value of the user *additionalValue2..3* 0.0 *additionalValue4:* T#0ms
16#8600	An error has occurred when using the TCON system function, see *subFunctionErrorID* in the diagnostic buffer and system documentation. The connection is reestablished. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* TCON status *additionalValue1:* 0.0 *additionalValue2:* 0.0 *additionalValue3:* 0.0 *additionalValue4:* T#0ms
16#8601	An error has occurred when using the TCON system function, see *subFunctionErrorID* in the diagnostic buffer and system documentation. The connection is reestablished. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* TCON status *additionalValue1:* Local port *additionalValue2:* 0.0 *additionalValue3:* 0.0 *additionalValue4:* T#0ms

© Siemens 2026 |

Code	Meaning
16#8602	An error occurred while using the system function TSEND, see *subFunctionErrorID* in the diagnostic buffer and system documentation. The connection is reestablished. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* TSEND status *additionalValue1:* 0.0 *additionalValue2:* Length of send data *additionalValue3:* 0.0 *additionalValue4:* T#0ms
16#8603	An error has occurred when using the TRCV system function, see *subFunctionErrorID* in the diagnostic buffer and system documentation. The connection is reestablished. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* Status of TRCV *additionalValue1..3:* 0.0 *additionalValue4:* T#0ms
16#8604	An error has occurred when using the TDISCON system function, see *subFunctionErrorID* in the diagnostic buffer and system documentation. The connection is reestablished. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* Status of TDISCON *additionalValue1..3:* 0.0 *additionalValue4:* T#0ms
16#8610	The vital signs of the partner were not received in time. The connection is reestablished. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* Vital signs cycle *additionalValue2..3:* 0.0 *additionalValue4:* Time in which the FB has not received any data from the partner.
16#8611	The monitoring time of a sent data packet has expired without an acknowledgement being received from the partner. The connection is reestablished. Diagnostic entry: *timestamp:* Time of the event *isActive:* State *subFunctionErrorID:* 16#0 *additionalValue1:* Monitoring time *additionalValue2..3:* 0.0 *additionalValue4:* Time in which the FB has not received any data from the partner.

© Siemens 2026

Code	Meaning	
16#8612	An invalid acknowledge number was received from the partner, the connection will be re-established.	
	Diagnostic entry:	
	timestamp:	Time of the event
	isActive:	State
	subFunctionErrorID:	16#0
	additionalValue1:	Received acknowledge message number
	additionalValue2:	Send acknowledge message number
	additionalValue3:	0.0
	additionalValue4:	T#0ms
16#8613	The LCom (protocol) versions of the library LCom do not match the partner or an invalid (protocol) version was received. The connection is reestablished.	
	Diagnostic entry:	
	timestamp:	Time of the event
	isActive:	State
	subFunctionErrorID:	16#0
	additionalValue1:	received version
	additionalValue2:	local, active version
	additionalValue3:	0.0
	additionalValue4:	T#0ms
16#8614	A telegram was received which has an ID that was not expected. The connection is reestablished.	
	Diagnostic entry:	
	timestamp:	Time of the event
	isActive:	State
	subFunctionErrorID:	16#0
	additionalValue1:	received ID
	additionalValue2..3:	0.0
	additionalValue4:	T#0ms

If errors occur during parameterization (16#8200 to 16#83FF), intervention by the user is required. The user must replace the faulty value with a permissible value and restart the function block with a rising edge at the "enable" input.

In case of an internal error (16#8600 to 16#87FF) the function block will automatically close the connection and try to re-establish it. A new rising edge at the "enable" input is not necessary.

© Siemens 2026

3. LFTP

3.1. Overview

3.1.1. Range of Functions

A simple protocol that works according to the client/server principle and which meets the demands of this task is the **F**ile **T**ransfer **P**rotocol (FTP).

FTP allows you to store data on server systems. FTP supports almost all server systems and operating systems.

The controllers from the SIMATIC S7-300, S7-400 and S7-1500 product families support FTP communication with the help of specialized communications processors (CPs).

With this library you can implement FTP communication with an S7-1200, S7-1500 or ET200SP CPU based on Open User Communication **without a special CP**.

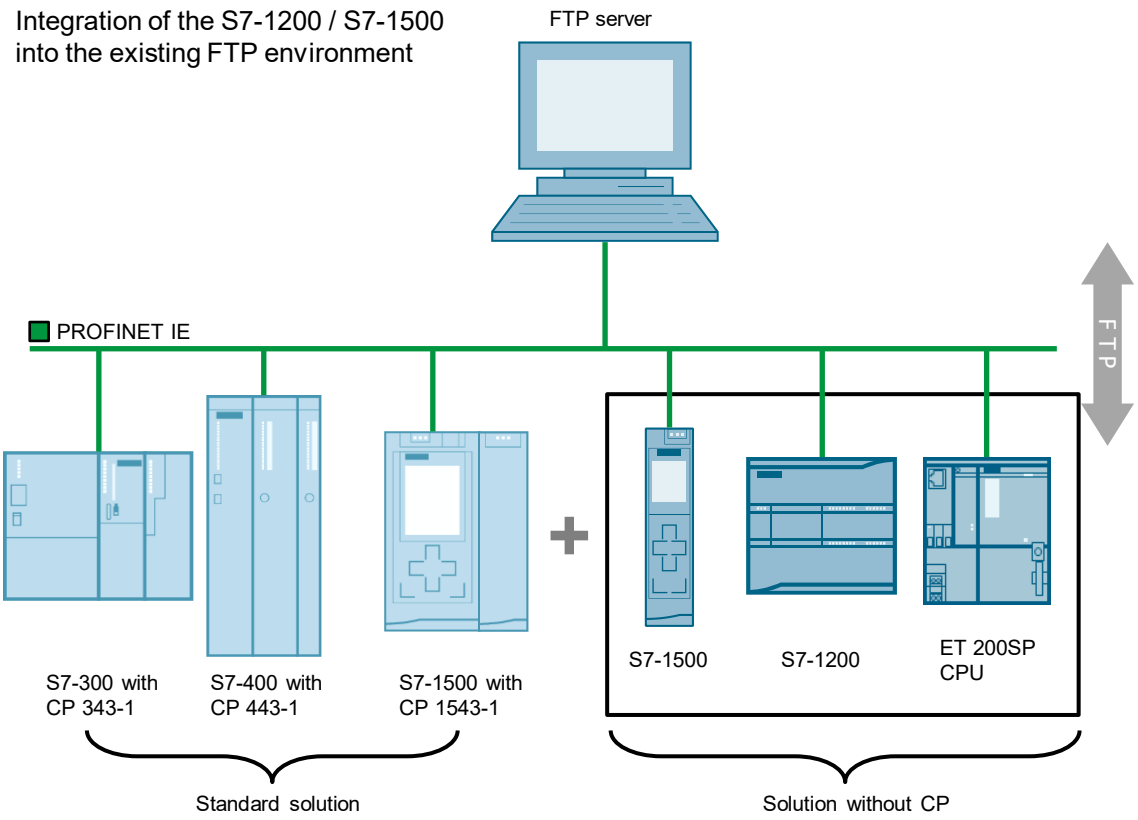


Figure 3-1: Application scenarios

© Siemens 2026

3.1.2. Components of the Library

Function blocks

Table 3-1: Function blocks

Name	Version	Description
LFTP_Client	V6.2.1	Organizes connection setup of the FTP control connection and data connection as well as sending and receiving of files.

PLC data types

Table 3-2: PLC data types

Name	Version	Description
LFTP_typeConnParam	V1.0.0	This data type describes all parameters of the connection to the FTP server.
LFTP_typeFtpParam	V1.0.1	This data type describes all parameters of the FTP commands.

3.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 controllers with firmware V2.9 or higher
- SIMATIC S7-1200 controllers with firmware V4.5 or higher
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers with firmware V2.9 or higher
- CM 1542-1

© Siemens 2026 |

3.2. LFTP_Client

3.2.1. Interface Description

Description

The FTP function block "LFTP_Client" implements FTP based on Open User Communication. It can execute the following FTP commands:

- CONNECT (connect and log on)
- DISCONNECT (disconnect and log off)
- NOOP (no operation)
- STORE (save data)
- APPEND (append data)
- RETRIEVE (retrieve data)
- DELETE (delete a file)

The FB supports both modes Active FTP and Passive FTP.

Parameters

Figure 3-2: LFTP_Client

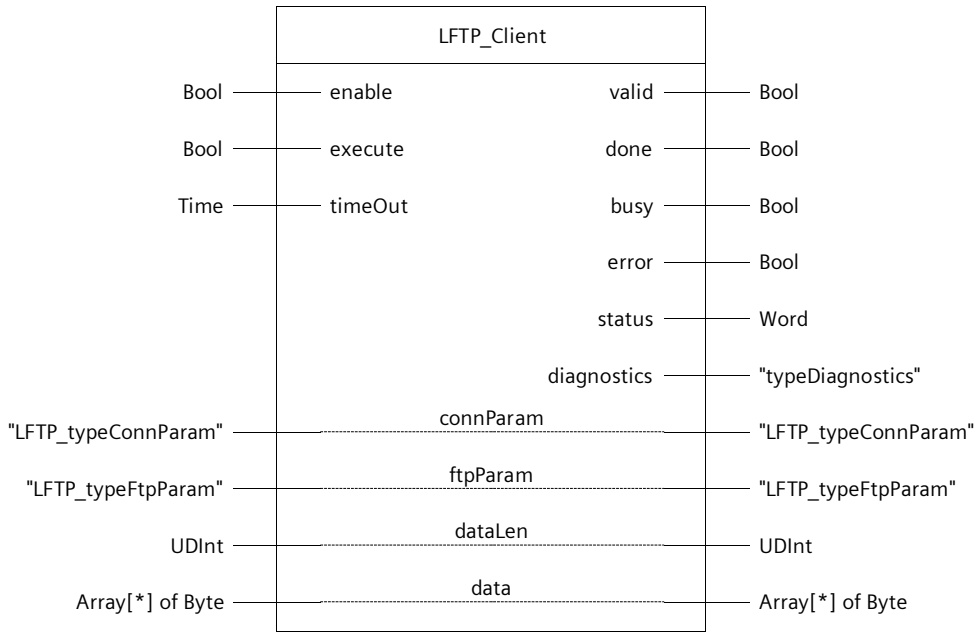


Table 3-3: Parameters of LFTP_Client

Name	Declaration	Data type	Comment
enable	IN	Bool	TRUE = control connection to the FTP server is connected FALSE = control connection is terminated
execute	IN	Bool	FTP job trigger on rising edge
timeOut	IN	Time	Time after which a job should be automatically cancelled

© Siemens 2026

Name	Declaration	Data type	Comment
valid	OUT	Bool	TRUE: The FB executes its function without error.
done	OUT	Bool	TRUE: The current job (send message or activate shell) was completed successfully.
busy	OUT	Bool	TRUE: The FB is busy.
error	OUT	Bool	TRUE: An error has occurred. If "busy" is TRUE at the same time, the block attempts to correct the error itself.
status	OUT	Word	Status and error codes (see chapter 3.2.2)
diagnostics	OUT	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
connParam	IN_OUT	"LFTP_typeConnParam"	Sets connection parameters. The hardware ID, URL of the FTP server, username and password are defined here.
ftpParam	IN_OUT	"LFTP_typeFtpParam"	FTP parameters
dataLen	IN_OUT	UDInt	Length of data to be sent or received
data	IN_OUT	Array[*] of Byte	Data to be sent or received

3.2.2. Error Handling

Table 3-4: Status and error codes

Status	Meaning
16#0000	FTP command successfully executed
16#7000	Idling, no active connections
16#7001	First call of the block
16#7002	Control connection is being established
16#7003	Control connection is being terminated
16#7004	Control connection is established, no job active
16#7005	FTP command is being executed
16#8202	Invalid URL
16#8400	Error received from FTP server. Some possible causes could be that the login credentials are wrong or that the file name does not exist. The FTP-specific error code is output at "diagnostics.subfunctionStatus". The meaning of the FTP-specific error code can be found in the FTP specification.
16#8600	The FB is in an invalid state.
16#8601	Error in subordinate instruction "TCON" The error code of the instruction is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

© Siemens 2026

Status	Meaning
16#8602	Error in subordinate instruction "TSEND" The error code of the instruction is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8603	Error in subordinate instruction "TRCV" The error code of the instruction is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8604	Error in subordinate instruction "TCONSettings" The error code of the instruction is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8605	Error in subordinate instruction "MOVE_BLK_VARIANT" The error code of the instruction is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8086	Error in subordinate instruction "RDREC" The error code of the instruction is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8D11	Control connection not connected; data connection cannot be opened
16#8D26	Data connection cannot be opened
16#8D28	FTP server timeout
16#8DF1	Request timeout
16#8DF2	Unknown reply code
16#8DF3	Unknown command
16#8DF4	Port number below 2000
16#8F62	Action aborted; data connection will be terminated
16#8F69	Connection attempt on existing connection

NOTE If the FB outputs "done" but no data after a "RETRIEVE" command, this may be caused by the communication load being set too low. Increase the communication load in the device configuration.

NOTE Some FTP servers (such as IIS Server) may have a “TIME_WAIT” state, which can delay the reuse of the same data port in active mode. In this case you might need to wait some time before you can use that port again.

Additionally, these servers might also have a “Control Channel Timeout”, after which the server will send a reset if no commands are sent for some time. Sending NOOP commands can prevent this timeout.

© Siemens 2026

3.3. PLC Data Types

LFTP_typeConnParam

This data type describes all the parameters required for the connection to the FTP server.

Table 3-5: Parameters of LFTP_typeConnParam

Name	Type	Comment
hwID	HW_ANY	Hardware ID of the IE interface module
serverAddress	String	IP address or hostname of the FTP server
username	String	FTP username
password	String	FTP user password

LFTP_typeFtpParam

This data type describes all FTP parameters necessary for the data connection.

Table 3-6: Parameters of LFTP_typeFtpParam

Name	Type	Comment
ftpActiveMode	BOOL	FTP mode; TRUE: active, FALSE: passive
portActiveMode	UInt	Local port for the FTP active mode, must be set to a value higher than 2000.
ftpCmd	Int	FTP command • 0: NOOP • 2: STORE • 3: RETRIEVE • 4: DELETE • 6: APPEND
filename	String	File name

3.4. Integration into the user project

You will find a detailed application example for the integration of the library into your user project in Siemens Industry Online Support:

https://support.industry.siemens.com/cs/ww/en/view/81367009

© Siemens 2026

4. LHTTP

4.1. Overview

4.1.1. Range of Functions

The Hypertext Transfer Protocol (HTTP) is a data transfer protocol used primarily to load Web pages from the World Wide Web.

Due to the increasing networking of plants and the advancement of the Internet of Things (IoT), HTTP and HTTPS also play an increasingly important role in automation technology.

The library for HTTP communication (LHTTP) enables the data exchange of a SIMATIC S7-1200/1500 CPU via the integrated Ethernet interface with another device in the local network or a web server on the internet via HTTP respectively HTTPS.

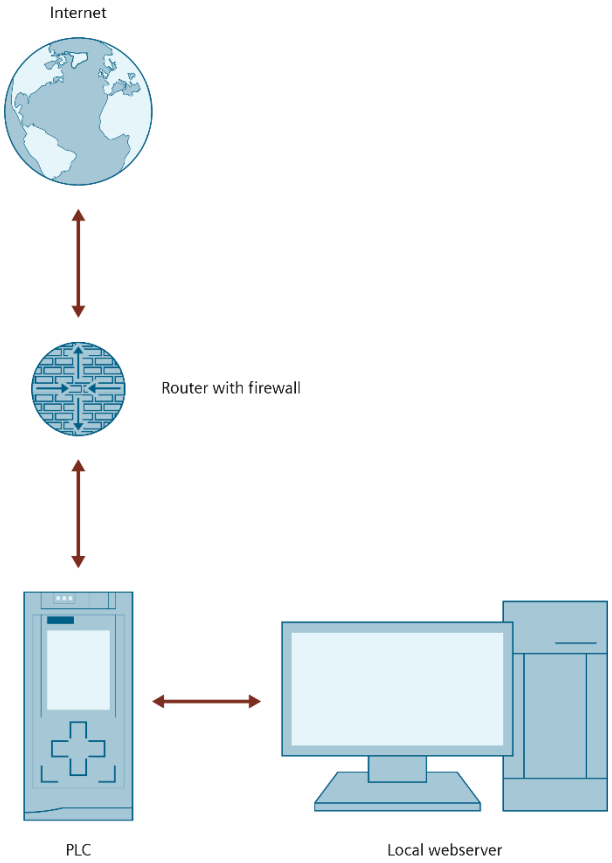


Figure 4-1: Overview

The LHTTP library provides function blocks with which the most conventional HTTP request methods can be implemented in the user program:

- GET
- POST
- PUT

Due to the integrated certificate management in the TIA Portal, it is also possible to transfer data securely with HTTPS using the same blocks.

4.1.2. Components of the Library

Blocks

Table 4-1: Blocks of the library

Name	Type	Version	Description
LHTTP_Get	FB	V2.3.0	Realizes the HTTP method GET
LHTTP_PostPut	FB	V2.3.0	Realizes the HTTP methods POST and PUT
LHTTP_FindStringInArray	FC	V1.0.0	Searches an array of chars for a given string
LHTTP_ExtractString FromArray	FC	V1.0.0	Extracts a string between two specified text parts from an array of chars.
LHTTP_ExtractString FromArrayExt	FC	V1.0.0	Same function as "LHTTP_ExtractStringFromArray" with advanced options

PLC Data Types

Table 4-2: PLC library data types

Name	Version	Description
LHTTP_typeTLS	V1.0.0	Data type for transferring certificates for secure communication via HTTPS
LHTTP_typeContentFormat	V1.0.0	Data type for setting both the content formats supported by the PLC when making GET requests, and the format of the content being sent with POST and PUT request

4.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.4
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- CP 1543-1 as of firmware V2.0
- CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V3.2

© Siemens 2026

4.2. LHTTP_Get

4.2.1. Interface Description

Description

The block implements the HTTP method GET to fetch data from a web server.

Parameter

Figure 4-2: LHTTP_Get

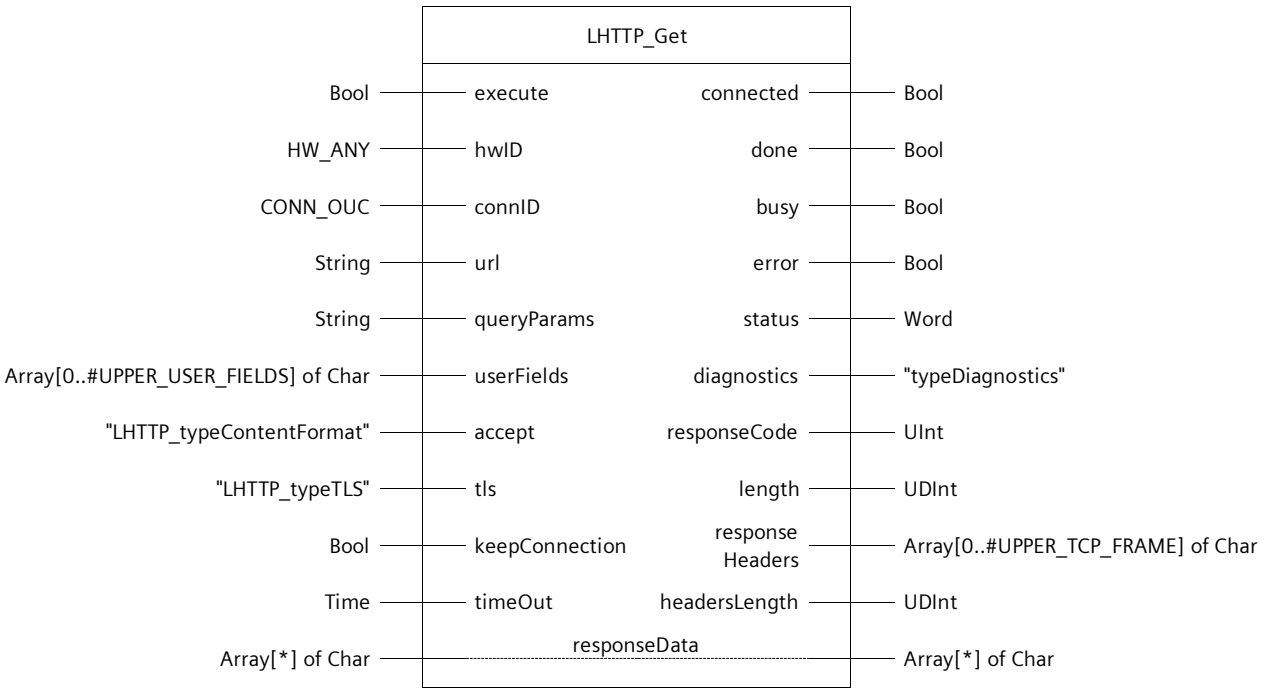


Table 4-3: Parameters of LHTTP_Get

Name	Declaration	Data type	Description
execute	Input	Bool	With a positive edge, a connection to the web server is established and the HTTP request is sent.
hwID	Input	HW_ANY	Hardware identifier of the PN/IE interface for establishing the connection If "0" is selected, a suitable one is selected automatically.
connID	Input	CONN_OUC	Unique connection ID
url	Input	String	URL, e.g. "https://httpbin.org/get" HTTPS is used per default. To use HTTP "http://" must be explicitly specified in the URL.
queryParams	Input	String	Additional query parameters if parameter "url" is not long enough, i.e. "lang=en&q=simatic".
userFields	Input	Array[0..#UPPER_USER_FIELDS] of Char	Additional user-defined header fields The header fields must be formatted according to HTTP and terminated with the "$00" character.
accept	Input	"LHTTP_typeContentFormat"	Sets the Accept header field, used to notify the HTTP server of the content formats supported by the PLC ("text/html" by default). Multiple can be selected.

© Siemens 2026

Name	Declaration	Data type	Description
tls	Input	"LHTTP_typeTLS"	TLS certificates for secure data transmission (HTTPS) For unsafe data transmission (HTTP) leave unconnected.
keepConnection	Input	Bool	TRUE: Keeps the connection established after the job has been executed
timeOut	Input	Time	Time after which a job should be automatically cancelled
connected	Output	Bool	TRUE: Connection to the web server is established
done	Output	Bool	TRUE: Job finished
busy	Output	Bool	TRUE: Job is being executed
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 4.8)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
responseCode	Output	UInt	Received HTTP status code (see Table 4-11)
length	Output	UDInt	Length of the received user data
responseHeaders	Output	Array[0..#UPPER_TCP_FRAME] of Char	Headers contained in the response received from the GET request
headersLength	Output	UDInt	Length of the response headers
responseData	InOut	Array[*] of Char	Received user data. The array must start at "0".

4.2.2. Operation

The user specifies the requested resource in the form of a URL, e.g. "https://httpbin.org/get" or "http://192.168.0.1:80/index.html", at the "url" parameter. HTTPS is used per default. To use HTTP "http://" must be explicitly specified in the URL.

Optional parameters can be passed with two variants:

- Appended to the URL at the parameter "url" with a "?" in front, e.g. "http://httpbin.org/get?lang=en&q=simatic"
- At the separate parameter "queryParams", e.g. "lang=en&q=simatic"; "?" or "&" is inserted automatically in between "url" and "queryParameters".

With a rising edge at the "execute" parameter, the block requests the IP address belonging to the host at the configured DNS Server if necessary and establishes a connection to the web server.

If HTTPS is used, the certificate of the requested web server must be referenced at the "tls" parameter (see chapter 4.7). If the web server requires client authentication, the PLC's certificate must also be referenced.

The block creates the HTTP request from the specified URL and sends it to the web server.

The user data of the web server's response is output at the "responseData" parameter. The received HTTP status code is output at the "responseCode" parameter. And the HTTP headers are output at the “responseHeaders” parameters, with their length at the “headersLength” parameter.

If the web server responds with an error (HTTP status code 4xx), no data is output.

© Siemens 2026

NOTE The FB does not clear the receive area at "responseData" between two requests. The receive area might contain old data. Use the length of the received user data of the current request at the output "length" to only evaluate this data.

Depending on the parameter "keepConnection", the block terminates the connection to the web server after all telegrams have been successfully received or keeps it established for further requests.

NOTE The block outputs the HTTP status code received from the server with the first received telegram and writes the user data of each received telegram directly into the memory area connected to the parameter "responseData". Evaluate the user data only after the output "done" has been set.

NOTE The block does not support redirections. If the server responds with a redirection (HTTP status code 3xx), the block issues an error (see chapter 4.8).

Check the URL and correct it if necessary.

User-defined header fields

The "userFields" input allows the user to insert additional header fields into the request. The user has 256 characters available in the form of an Array of Char.

The data must be formatted according to HTTP: Two header fields are separated by an end-of-line ("RL"). The end of the user-defined header fields is signalled to the FB with the character "$00". A closing line end is not necessary.

For example, the "Accept Language" and "Authorization" header fields can be inserted as follows:

```
Accept-Language: en$R$LAuthorization: Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ==$00
```

If no user-defined header fields are required, the input is not connected.

NOTE If 256 characters are not enough, the array can be increased by adjusting the local constant "UPPER_USER_FIELDS".

Keep connection established

If the parameter "keepConnection" is set, the connection is not broken after the completion of a job and can be used for further requests. Especially if there are many requests to the same web server over HTTPS, this shortens the required duration of subsequent requests.

The connection is then broken by resetting "keepConnection".

© Siemens 2026

4.3. LHTTP_PostPut

4.3.1. Interface Description

Description

The block implements the HTTP methods POST and PUT to transfer data to a web server.

Parameter

Figure 4-3: LHTTP_PostPut

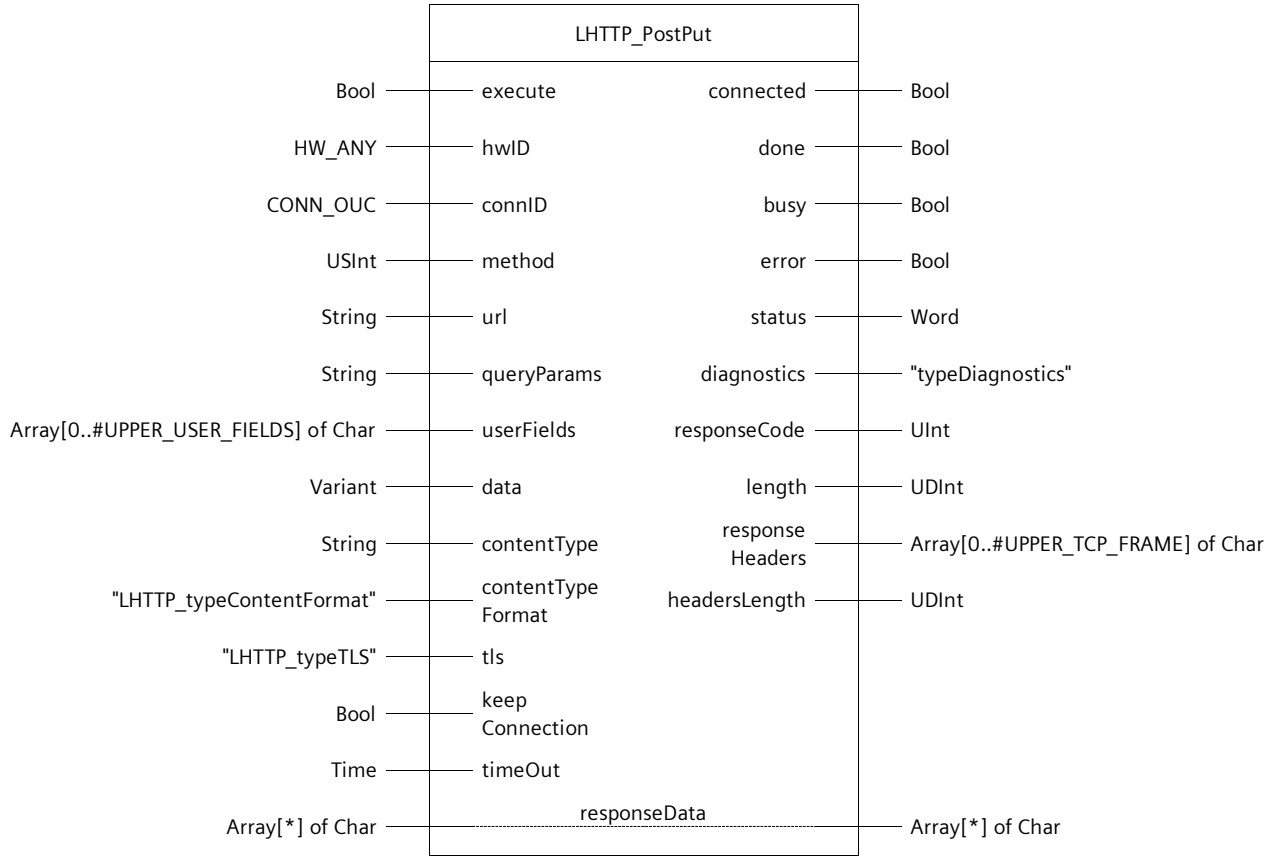


Table 4-4: Parameters of LHTTP_PostPut

Name	Declaration	Data type	Description
execute	Input	Bool	With a positive edge, a connection to the web server is established and the HTTP request is sent.
hwID	Input	HW_ANY	Hardware identifier of the PN/IE interface for establishing the connection If "0" is selected, a suitable one is selected automatically.
connID	Input	CONN_OUC	Unique connection ID
method	Input	USInt	HTTP method 0: POST 1: PUT
url	Input	String	URL, e.g. "https://httpbin.org/post" HTTPS is used per default. To use HTTP "http://" must be explicitly specified in the URL.

© Siemens 2026

Name	Declaration	Data type	Description
queryParams	Input	String	Additional query parameters if parameter "url" is not long enough, i.e. "lang=en&q=simatic".
contentType	Input	String	Content type of the data being sent. (Deprecated, only kept for library consistency. Has been replaced by the "contentTypeFormat" input, but can still be used if the data format is different from all the options offered by the "LHTTP_typeContentFormat" UDT)
contentTypeFormat	Input	"LHTTP_typeContentFormat"	Sets the Content-Type header field, used to notify the HTTP server of the content format of the data being sent ("text/plain by default"). If multiple content formats are selected, only the topmost selected one will be used
data	Input	Variant	Message body Valid data types: String and Array of Char In the case of Array of Char, the data must be terminated with the character "$00".
userFields	Input	Array[0..#UPPER_USERFIELDS] of Char	Additional user-defined header fields The header fields must be formatted according to HTTP and terminated with the "$00" character.
tls	Input	"LHTTP_typeTLS"	TLS certificates for secure data transmission (HTTPS) For unsafe data transmission (HTTP) leave unconnected.
keepConnection	Input	Bool	TRUE: Keeps the connection established after the job has been executed
timeOut	Input	Time	Time after which a job should be automatically cancelled
connected	Output	Bool	TRUE: Connection to the web server is established
done	Output	Bool	TRUE: Job finished
busy	Output	Bool	TRUE: Job is being executed
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 4.8)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
responseCode	Output	UInt	Received HTTP status code (see chapter 4.8)
length	Output	UDInt	Length of the received user data
responseHeaders	Output	Array[0..#UPPER_TCP_FRAME] of Char	Headers contained in the response received from the POST or PUT request
headersLength	Output	UDInt	Length of the response headers
responseData	InOut	Array[*] of Char	Received user data. The array must start at "0".

4.3.2. Operation

The user specifies the requested resource in the form of a URL, e.g. "https://httpbin.org/post" or "http://192.168.0.1:80/index.html", at the "url" parameter. HTTPS is used per default. To use HTTP "http://" must be explicitly specified in the URL.

© Siemens 2026

Optional parameters can be passed with two variants:

- Appended to the URL at the parameter "url" with a "?" in front, e.g. "http://httpbin.org/post?lang=en&q=simatic"
- At the separate parameter "queryParams", e.g. "lang=en&q=simatic"; "?" or "&" is inserted automatically in between "url" and "queryParameters".

The user data can be passed as a string or, if 254 characters are not sufficient, as an array of chars at the "data" parameter. If Array of Char is used, the user data must be terminated with the character "$00". The format of the data can be set with the parameter "contentTypeFormat".

Via the "method" parameter the user chooses whether the HTTP method POST or PUT should be used.

With a rising edge at the "execute" parameter, the block requests the IP address belonging to the host at the configured DNS Server if necessary and establishes a connection to the web server.

If HTTPS is used, the certificate of the requested web server must be referenced at the "tls" parameter (see chapter 4.7). If the web server requires client authentication, the PLC's certificate must also be referenced.

The block creates the HTTP request from the specified URL and sends it to the server.

The user data of the web server's response is output at the "responseData" parameter. The received HTTP status code is output at the "responseCode" parameter. And the HTTP headers are output at the “responseHeaders” parameters, with their length at the “headersLength” parameter.

If the web server responds with an error (HTTP status code 4xx), no data is output.

NOTE The FB does not clear the receive area at "responseData" between two requests. The receive area might contain old data. Use the length of the received user data of the current request at the output "length" to only evaluate this data.

Depending on the parameter "keepConnection", the block terminates the connection to the server after all telegrams have been successfully received or keeps it established for further requests.

NOTE The block outputs the HTTP status code received from the server with the first received telegram and writes the user data of each received telegram directly into the memory area connected to the parameter "responseData". Evaluate the user data only after the output "done" has been set.

NOTE The block does not support redirections. If the server responds with a redirection (HTTP status code 3xx), the block issues an error (see chapter 4.8).

Check the URL and correct it if necessary.

User-defined header fields

The "userFields" input allows the user to insert additional header fields into the request. The user has 256 characters available in the form of an Array of Char.

The data must be formatted according to HTTP: Two header fields are separated by an end-of-line ("RL"). The end of the user-defined header fields is signalled to the FB with the character "$00". A closing line end is not necessary.

For example, the "Accept Language" and "Authorization" header fields can be inserted as follows:

```
Accept-Language: en$R$LAuthorization: Basic QWxhZGRpbjpvcGVuIHNlc2FtZQ==$00
```

If no user-defined header fields are required, the input is not connected.

© Siemens 2026

NOTE If 256 characters are not enough, the array can be increased by adjusting the local constant "UPPER_USER_FIELDS".

Keep connection established

If the parameter "keepConnection" is set, the connection is not broken after the completion of a job and can be used for further requests. Especially if there are many requests to the same web server over HTTPS, this shortens the required duration of subsequent requests.

The connection is then broken by resetting "keepConnection".

© Siemens 2026 |

4.4. LHTTP_FindStringInArray

Description

The function "LHTTP_FindStringInArray" searches an Array of Char for a string and returns its position as return value.

Parameter

Figure 4-4: LHTTP_FindStringInArray

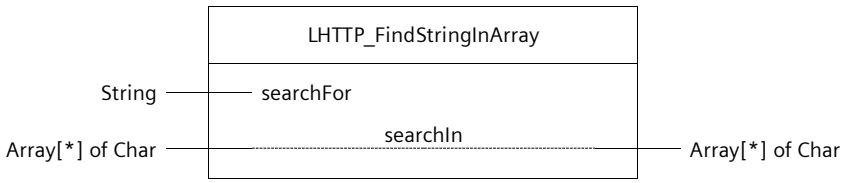


Table 4-5: Parameters of LHTTP_FindStringInArray

Name	Declaration	Data type	Description
searchFor	Input	String	Text to search for.
searchIn	InOut	Array[*] of Char	Array of Char to be searched.
Ret_Val	Return	DInt	Position (index) of the searched string in the array. -1: String was not found.

4.5. LHTTP_ExtractStringFromArray

Description

The LHTTP_ExtractStringFromArray function extracts a string between two specified text parts from an array of chars.

Parameter

Figure 4-5: LHTTP_ExtractStringFromArray

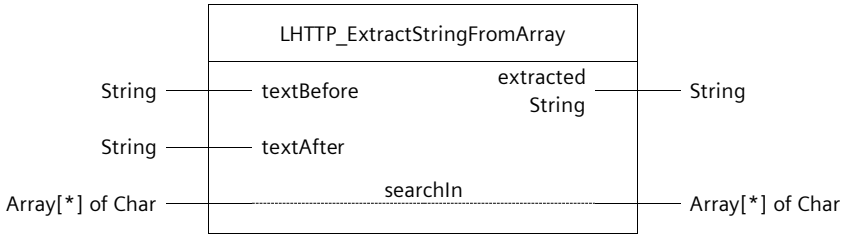


Table 4-6: Parameters of LHTTP_ExtractStringFromArray

Name	Declaration	Data type	Description
textBefore	Input	String	Text part before the text that is to be extracted
textAfter	Input	String	Text part after the text that is to be extracted
extractedString	Output	String	Extracted Text
searchIn	InOut	Array[*] of Char	Array of Char to be searched

© Siemens 2026

Name	Declaration	Data type	Description
Ret_Val	Return	Word	Return value: 16#0000: Successful, Text was found 16#9001: Only text before was found. String is output with max. length 16#9002: Neither text before nor after was found

4.6. LHTTP_ExtractStringFromArrayExt

Description

The function "LHTTP_ExtractStringFromArrayExt" extracts a string between two specified text parts from an array of char with extended options.

NOTE The function is part of the library to allow easy parsing of the received data, but is not used by the library blocks themselves.

Parameter

Figure 4-6: LHTTP_ExtractStringFromArrayExt

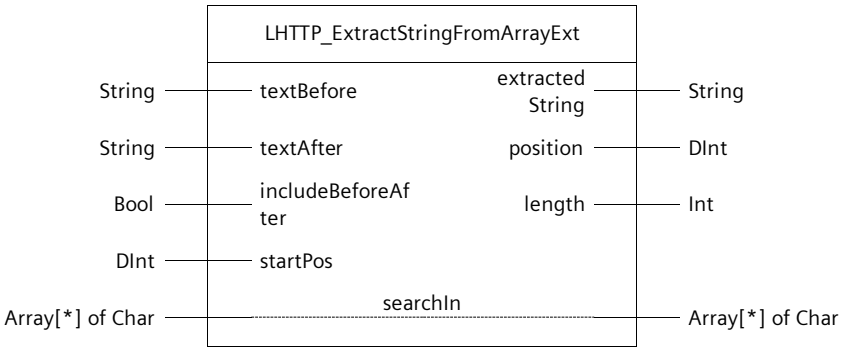


Table 4-7: Parameters of LHTTP_ExtractStringFromArrayExt

Name	Declaration	Data type	Description
textBefore	Input	String	Text part before the text that is to be extracted
textAfter	Input	String	Text part after the text that is to be extracted
includeBeforeAfter	Input	Bool	If TRUE, the text parts before and after are extracted as well
startPos	Input	DInt	Position in the array from which the search is to be started
extractedString	Output	String	Extracted Text
position	Output	DInt	position (index) of the extracted text within the array
length	Output	Int	Length of the extracted text
searchIn	InOut	Array[*] of Char	Array of Char to be searched
Ret_Val	Return	Word	Return value: 16#0000: Successful, Text was found 16#9001: Only text before was found. String is output with max. length 16#9002: Neither text before nor after was found

© Siemens 2026

4.7. PLC Data Types

LHTTP_typeTLS

The PLC data type "LHTTP_typeTLS" references the certificates required for secure connections via HTTPS.

Table 4-8: Parameters of LHTTP_typeTLS

Name	Data type	Description
validateServerIdentity	Bool	A set bit means that the client validates the subjectAlternateName in the server's X.509 V3 certificate to verify the server's identity. The certificates are checked when the connection is established.
serverCert	UDInt	ID of the X.509 V3 certificate (usually a CA certificate) used by the TLS client to validate the TLS server authentication. If this parameter is "0", the TLS client uses all (CA) certificates currently loaded in the client's Certificate Store to validate the server authentication.
clientCert	UDInt	ID of your own X.509 V3 certificate. This is only relevant if the TLS server requires client authentication and can otherwise remain set to "0".

NOTE You will find an application example for the usage of certificates in TIA Portal in the Siemens Industry Online Support:
https://support.industry.siemens.com/cs/ww/en/view/109769068

LHTP_typeContentFormat

The PLC data type "LHTTP_typeContentFormat" references the different content formats supported by LHTTP_Get and LHTTP_PostPut.

Table 4-9: Parameters of LHTTP_typeContentFormat

Name	Data type	Description
plainText	Bool	Content type "text/plain"
html	Bool	Content type "text/html"
json	Bool	Content type "application/json"
xml	Bool	Content type "application/xml"

4.8. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The following are the status and error codes specific to the "LHTTP" library.

The function blocks "LHTTP_Get" and "LHTTP_PostPut" output internal status and error codes describing the state of the function block as well as the HTTP status code received from the partner.

Internal status codes

Table 4-10: Internal status codes

Code	Description
16#0000	Job completed without errors

© Siemens 2026

Code	Description
16#0003	Connection broken
16#7000	No job active
16#7001	First call of the command
16#7002	Follow-up call of the command
16#7003	Connection is broken
16#7004	Connection is established and monitored. FB is waiting for job.
16#7005	Request has been sent, response pending
16#8201	The array at the parameter "responseData" does not begin with "0".
16#8202	Invalid URL. Check the spelling.
16#8203	The host name in the URL is different from the host to which the current connection exists. Reset "keepConnection" and run the job again.
16#8204	Data type at parameter "data" is neither String nor Array of Char
16#8205	End of data could not be detected. The data must be terminated with the "$00" character.
16#8206	The end of the user-defined header fields at parameter "userFields" could not be detected. Terminate the header fields with the "$00" character.
16#8210	The internal send buffer is too small for the request to be transmitted. Adjust the size of the send buffer at the local constant "UPPER_REQUEST_ARRAY".
16#8220	The size of the array at the parameter "responseData" is not sufficient to store the received user data. The array was completely filled, and the length of the received user data is output at the parameter "length".
16#8403	The web server responded with a redirection. Redirections are not supported by the LHTTP library.
16#8404	The web server responded with an error message. The received HTTP status code is output at the "responseCode" output.
16#8405	The header of the response telegram is invalid.
16#8408	Request timeout. Check the Ethernet connection of the PLC and the URL.
16#84C4	The connection to the web server was interrupted. Check the network connection.
16#84C5	Connection to the web server has broken.
16#8600	The FB is in an invalid state.
16#8601	Error in subordinate command "TSEND_C" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8603	Error of subordinate command "TRCV" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8612	Error in subordinate command "MOVE_BLK_VARIANT" when inserting user-defined header fields. The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

© Siemens 2026

Code	Description
16#8620	Encoding of the received message could not be detected or is not supported.
16#8630	Error of subordinate command "MOV_BLK_VARIANT" in region "TE_IDENTITY". The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8640	Chunk-coded message could not be decoded.
16#8641	Error of subordinate command MOV_BLK_VARIANT in region "TE_CHUNKED". The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

© Siemens 2026 |

HTTP status codes

The following table lists common HTTP status codes:

Table 4-11: Common HTTP status codes

Status code	Message	Meaning
2xx - Successful operation		
200	OK	The request was processed successfully.
3xx - Redirection		
301	Moved Permanently	The requested resource is now available at the address specified in the "Location" header field. The old address is no longer valid.
304	Not Modified	The content of the requested resource has not changed since the last request from the client and is therefore not transferred.
4xx – Client error		
400	Bad Request	The request message was structured incorrectly.
401	Unauthorized	The request cannot be performed without valid authentication.
403	Forbidden	The request was not executed due to lack of client authorization.
404	Not Found	The requested resource is not available in the required form.
5xx – Server error		
500	Internal Server Error	Unexpected service error.
502	Bad Gateway	The server could not fulfil its functionality as a gateway or proxy because it received an invalid response.
503	Service Unavailable	The server is temporarily unavailable.
504	Gateway Timeout	The server could not perform its function as a gateway or proxy because it did not receive a response from the servers or services it was using within a specified period of time.

Further HTTP status codes can be found on Wikipedia, for example:

https://de.wikipedia.org/wiki/HTTP-Statuscode

© Siemens 2026 |

5. LMQTT

5.1. Overview

5.1.1. Range of Functions

The library "LMQTT" provides you with a function block to implement the MQTT protocol in SIMATIC S7-1200/1500 controllers. The FB allows you to submit MQTT messages to a broker (publisher role) and to create subscriptions (subscriber role). The communication can be secured via a TLS connection.

5.1.2. Components of the Library

The "LMQTT" library contains the following objects.

Blocks

Table 5-1: Blocks of the library

Name	Type	Version	Description
LMQTT_Client	FB	V4.3.0	Implements the MQTT Client function and allows to submit MQTT messages to a broker (Publisher role) and create subscriptions (Subscriber role).
LMQTT_Client_MultipleMsg	FB	V4.4.0	Implements the same functions as the LMQTT_Client block. Additionally, can process multiple messages received during the same PLC cycle.
LMQTT_ConvertToUtf8	FC	V3.0.0	Implements a function to convert a WCHAR to UTF-8 compliant formatting.

PLC Data Types

Table 5-2: PLC library data types

Name	Version	Description
LMQTT_typeConnParam	V4.0.0	Contains all information to establish a connection with the MQTT Broker.

5.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.4
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- CP 1543-1 as of firmware V2.0
- CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V3.2

© Siemens 2026

5.2. LMQTT_Client

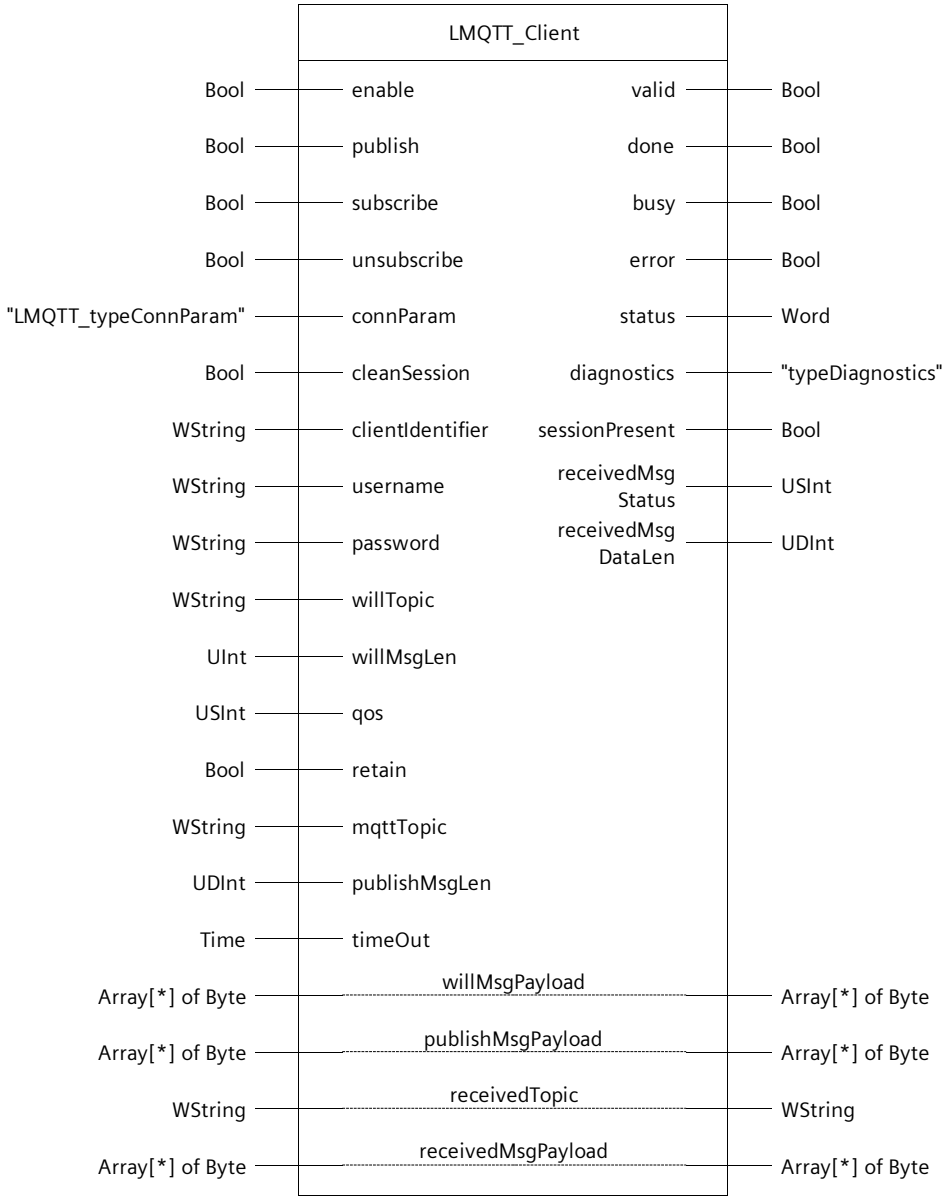
5.2.1. Interface Description

Description

The function block "LMQTT_Client" integrates the MQTT Client function and allows you to submit MQTT messages to a broker (Publisher role) and to create subscriptions (Subscriber role). The communication can be secured via a TLS connection.

Parameter

Figure 5-1: LMQTT_Client



© Siemens 2026 |

Name	Declaration	Data type	Comment
enable	Input	Bool	• TRUE: The connection to the MQTT Broker is established and maintained • FALSE: The connection is broken
publish	Input	Bool	Publish "publishMsgPayload" to "mqttTopic" with "retain" and "qos"
subscribe	Input	Bool	Subscribe to "mqttTopic" with "qos"
unsubscribe	Input	Bool	Unsubscribe from "mqttTopic"
connParam	Input	"LMQTT_typeConnParam"	Parameters to establish a connection to MQTT broker
cleanSession	Input	Bool	• TRUE: Establish a new clean session with the broker • FALSE: The broker must maintain the session state after disconnection. See note below for further information.
clientIdentifier	Input	WString	Client identifier used when establishing a connection
username	Input	WString	Optional: Username for connection establishment
password	Input	WString	Optional: Password for connection establishment
willtopic	Input	WString	Optional: Topic to which the "Last Will" message is sent
willMsgLen	Input	UInt	Current length of valid data in the array parameter "willMsgPayload"
qos	Input	USInt	Quality of Service • 0: Messages are sent or topics are subscribed to with QoS 0 • 1: Messages are sent or topics are subscribed to with QoS 1 2: Messages are sent or topics are subscribed to with QoS 2
retain	Input	Bool	• TRUE: The data is sent with the "retain" flag. • FALSE: The data is sent without the "retain" flag.
mqttTopic	Input	WString	• Topic used for publish / subscribe / unsubscribe
publishMsgLen	Input	UDInt	Current length of valid data in the "publishMsgPayload" array parameter
timeOut	Input	Time	Optional parameter to configure time monitoring. After the time has expired, the current job will be considered as failed.
willMsgPayload	InOut	Array [*] of bytes	Optional: Message sent as "Last Will". It must start at index 0
publishMsgPayload	InOut	Array [*] of bytes	Message that is transmitted as user data during publishing. It must start at index 0.
receivedTopic	InOut	WString	Subscribed topic on which a message was received.

© Siemens 2026

Name	Declaration	Data type	Comment
receivedMsgPayload	InOut	Array [*] of bytes	User data received in the message of the subscribed topic. It must start at index 0. See note below for further information.
valid	Output	Bool	TRUE: The FB executes its function without error
done	Output	Bool	TRUE: The current job (publish/subscribe/unsubscribe) was executed successfully
busy	Output	Bool	TRUE: The FB is busy
error	Output	Bool	TRUE: An error has occurred If "busy" is TRUE at the same time, the block attempts to correct the error itself.
status	Output	Word	Status and error codes (see chapter 5.2.3)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
sessionPresent	Output	Bool	• TRUE: Indicates that the broker has a previously stored session state • FALSE: Indicates that there is no session state stored in the broker. The stablished session is clean See note below for further information.
receivedMsgStatus	Output	USInt	Indicates for one cycle at a time when a new message has been received (subscription): • 0: No new message received • 1: New valid message received. Data outputs can be evaluated • 2: New message received, but the message is invalid or received data is larger than the memory area of the receive topic or the receive message. Data outputs can be evaluated
receivedMsgDataLen	Output	UDInt	Number of valid data at the "receivedMsgPayload" array parameter in bytes

Table 5-3: Parameters of LMQTT_Client

NOTE Regardless of the size of the “receivedMsgPayload” array, messages of topics which have been subscribed to with QoS 2 will only be as big as the block’s internal buffer. This buffer stores QoS 2 messages until the corresponding PUBREL has been received, and its maximum length is 1459 bytes, defined by the constant MAX_BUFFER_LEN_RCV.

© Siemens 2026 |

NOTE The "cleanSession" parameter must be configured before enabling the block, as it is relevant when establishing the connection to the broker. The "sessionPresent" parameter will be updated after the connection has been correctly established.

Both parameters must be considered together.

If the broker accepts a connection with "cleanSession" set to TRUE, the broker must set "sessionPresent" to FALSE, indicating that the session is clean.

If the broker accepts a connection with "cleanSession" set to FALSE, the value set in "sessionPresent" depends on whether the broker already has a stored session state for the supplied "clientIdentifier". If the broker has a stored session state, it must set "sessionPresent" to TRUE. If the broker does not have a stored session state, it must set "sessionPresent" to FALSE.

In practice, if the "sessionPresent" flag is set to TRUE after the connection is established with "cleanSession" set to FALSE, the block does not need to resubscribe to any topics that had been previously subscribed to. The broker will automatically start sending any relevant messages to the PLC.

NOTE While a job is being processed, the block does not accept any other jobs – this includes the periodic keep alive function. Wait until the existing job is completed by evaluating the status 16#7004 and then generate an edge at the respective input.

NOTE If multiple MQTT publish messages are received within a single call cycle of the block, only the last one will be available at the outputs.

5.2.2. LMQTT_Client_MultipleMsg

When multiple messages are expected to be received within a single call cycle of the block, the "LMQTT_Client_MultipleMsg" block can be used. While its functionality as a publisher remains consistent with the standard version, its primary advantage lies in its ability to process multiple messages originating from various topics and featuring different Quality of Service (QoS)

The standard output variables "receivedTopic", "receivedMsgDataLen" and "receivedMsgPayload" from the "LMQTT_Client" block are now implemented as arrays. "receivedTopics" and "receivedMsgDataLens**"** are one-dimensional arrays, whereas "receivedMsgPayload**"** is a two-dimensional array. The first dimension of the arrays will define the maximum number of messages that can be received during the same cycle.

For instance, if you expect to receive a maximum of 5 messages of 1000 bytes during the same PLC cycle, you will have to define the variables as follows: receivedTopics[0..4], receivedMsgDataLens[0..4], receivedMsgPayloads[0..4,0..999]. The data for a message is split across these three variables, but always at the same index.

The "receivedMsgStatus" variable will now contain the number of messages received on the cycle so that the output arrays can be evaluated correctly.

The table below contains the parameters from the interface that have been modified from the original "LMQTT_Client" block. The rest of the parameters stay the same.

Table 5-4: Parameters of LMQTT_Client_MultipleMsg

Name	Declaration	Data type	Comment
receivedMsgStatus	Output	USInt	Indicates for one cycle at a time when new messages have been received (subscription): • 0: No new message received • 1-255: Number of valid messages that have been received. Data outputs can be evaluated
receivedMsgDataLens	InOut	Array [*] of UDInt	Amount of valid data at the "receivedMsgPayloads" array parameter in bytes. Must start at index 0.

Name	Declaration	Data type	Comment
receivedTopics	InOut	Array [*] of WString	Subscribed topics on which the messages were received. Must start at index 0.
receivedMsgPayloads	InOut	Array [*,*] of Byte	User data received in the message of the subscribed topic. Both dimensions must start at index 0. See note below for further information.

NOTE See notes of “LMQTT_Client” block, as the same restrictions still apply for this block. The only exception is the last note, which is precisely what this block is addressing.

5.2.3. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

Interaction of status outputs

The following table describes the interaction of the status outputs for the "LMQTT_Client" FB:

Table 5-5: Status outputs for "LMQTT_Client"

valid	done	busy	error	Description
FALSE	FALSE	FALSE	FALSE	FB is inactive
TRUE	FALSE	TRUE	FALSE	FB performs its function without error
TRUE	TRUE	TRUE	FALSE	FB executes its function without error and current job (publish/subscribe/unsubscribe) has been completed successfully
FALSE	FALSE	TRUE	TRUE	An error has occurred that the FB is trying to fix automatically (for example, when the connection is lost) or the current job has failed, but FB can continue to function.
FALSE	FALSE	FALSE	TRUE	An error occurred that the FB is unable to fix automatically, and the FB has aborted the execution of its function (for example, in the case of invalid connection parameters). After you have fixed the error, a new edge at the "enable" input is necessary.

© Siemens 2026 |

Status- and error codes

The following are the status and error codes specific to the "LMQTT_Client" and "LMQTT_Client_MultipleMsg" blocks, which are issued at the "status" output:

Table 5-6: Status codes and error codes for "LMQTT_Client"

Code	Description
16#0000	Last job (publish/subscribe/unsubscribe) successfully executed
16#7000	Block is idle/client not connected
16#7001	First call after "enable" = TRUE
16#7002	Block processes job
16#7003	Establishing connection to MQTT broker
16#7004	MQTT client connected and ready for job
16#7005	Ping is sent to the broker
16#7006	Publish job is executed
16#7008	Subscribe job is executed
16#7010	Unsubscribe job is executed
16#8001	Simultaneous input (rising edge) at publish/subscribe/unsubscribe inputs
16#8200	Client identifier is empty – client identifier must be at least 1 character long
16#8210	Invalid array size: • During connection establishment: invalid array size at the "willMsgLen" parameter • During publish: invalid array size at the "publishMsgLen" parameter
16#8220	Invalid value at the "keepAlive" parameter
16#8230	Invalid value at parameter "qos"
16#8240	Only for "LMQTT_Client_MultipleMsg" Different size of output arrays (receivedTopics, receivedMsgDataLens and receivedMsgPayloads). All must have the same first dimension.
16#8250	Only for "LMQTT_Client_MultipleMsg" Incorrect start index at the output arrays (receivedTopics, receivedMsgDataLens or receivedMsgPayloads). All must start at index 0.
16#8300	Topic empty or invalid (publish)
16#8310	Topic empty or invalid (subscribe)
16#8320	Topic empty or invalid (unsubscribe)
16#8600	Undefined state in the state machine
16#8601	Error of subordinate instruction "TCON" The error code of the instruction is output at "diagnostics.subfunctionStatus". The meaning of the respective error code can be found in the TIA Portal Information System.
16#8620	Connection establishment timeout. If necessary, increase the "timeOut" value.

© Siemens 2026

Code	Description
16#8650	Timeout during the processing of the job
16#8670	Timeout during MQTT connection setup with the MQTT broker
16#8700	Invalid length in received MQTT packet
16#8710	Invalid QoS in received MQTT packet
16#8720	Existing session at the MQTT broker; new connection required
16#8730	Connection rejected by MQTT broker; new connection required. The error is specified at "diagnostics.subfunctionStatus": • 1: The MQTT Broker does not support the MQTT protocol 3.1.1 • 2: The MQTT broker has rejected the client identifier. • 3: The MQTT service at the MQTT broker is not reachable. • 4: The data in the Username or Password field is corrupted. • 5: The client is not authorized to connect (username or password incorrect) • 6-255: unknown error.
16#8740	MQTT packet was acknowledged by the broker with invalid packet ID
16#8750	MQTT subscription rejected by the broker
16#8760	The buffer for QoS 2 messages is full
16#8770	Received message has been skipped for not being fully inside the reception buffer
16#9000	Connection to the MQTT broker lost; automatic reconnect in progress

© Siemens 2026 |

5.3. LMQTT_ConvertToUtf8

This FC converts a WChar into a UTF-8 compliant format, which is necessary for correct transmission in MQTT Topics.

Figure 5-2: LMQTT_ConvertToUtf8

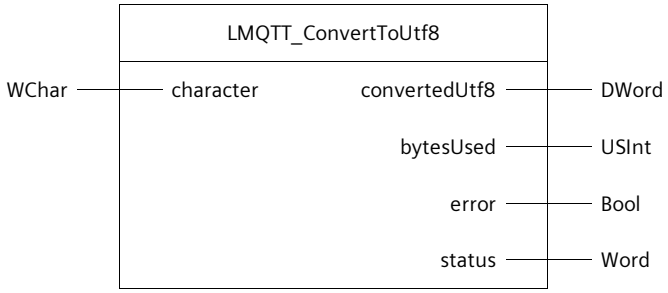


Table 5-7: Parameters of LMQTT_ConvertToUtf8

Name	Declaration	Data type	Comment
character	Input	WChar	Character to be transformed.
convertedUtf8	Output	DWord	Single UTF-8 compliant bytes. Bytes 0-2 are used. Byte 3 is not used.
bytesUsed	Output	USInt	Number of bytes occupied by the conversion.
error	Output	Bool	Error in execution.
status	Output	Word	Status code for editing.

5.4. PLC Data Types

LMQTT_typeConnParam

This data type stores all information required to establish the MQTT connection. You can set these parameters according to your specifications.

Table 5-8: PLC data type LMQTT_typeConnParam

Name	Data type	Description
hwId	HW_ANY	Hardware identifier of the PN/IE interface for establishing the connection 0: A suitable one is selected automatically.
connId	CONN_OUC	Connection ID for establishing the connection (1..4095)
broker	String	Hostname or IP address of the brokers.
port	UInt	MQTT-Port on the broker
tls	Struct	Parameters for TLS encryption
enableTls	Bool	• TRUE: Connection is secured with TLS • FALSE: Unsecured connection

© Siemens 2026

Name	Data type	Description
validateServerIdentity	Bool	• TRUE: The client validates the subject alternate name (SAN) in the certificate of the MQTT broker when the connection is established. • FALSE: No validation of the SAN
brokerCert	UDInt	ID of the MQTT broker certificate 0: The client uses all CA certificates in the certificate store of the PLC to authenticate the MQTT broker.
clientCert	UDInt	ID of own certificate 0: No certificate is used for the authentication of the client
keepAlive	UInt	Keep-alive mechanism of MQTT in seconds 0: No keep-alive

NOTE You will find an application example for the usage of certificates in TIA Portal in the Siemens Industry Online Support:

https://support.industry.siemens.com/cs/ww/en/view/109769068

5.5. Integration into the User Project

You will find a detailed application example for the integration of the library into your user project in the Siemens Industry Online Support:

https://support.industry.siemens.com/cs/ww/en/view/109748872

© Siemens 2026 |

6. LOpcUa

6.1. Overview

6.1.1. Range of Functions

Part 14 of the OPC UA specification introduced the "PubSub" communication mechanism. This mechanism is not based on the Client–Server principle as in conventional OPC UA communication (OPC 10000-4), but uses a "publish/subscribe" principle.

PubSub OPC UA applications, with their roles as "Publisher" (sender) and/or "Subscriber" (receiver), are decoupled from each other, which deviates from the Client–Server principle. The number of Subscribers is not important for the Publisher. Publishers and Subscribers also do not need to connect directly to each other, allowing not only uni- but also multi- or broadcasts. Additionally, the communication is separated from the data storage.

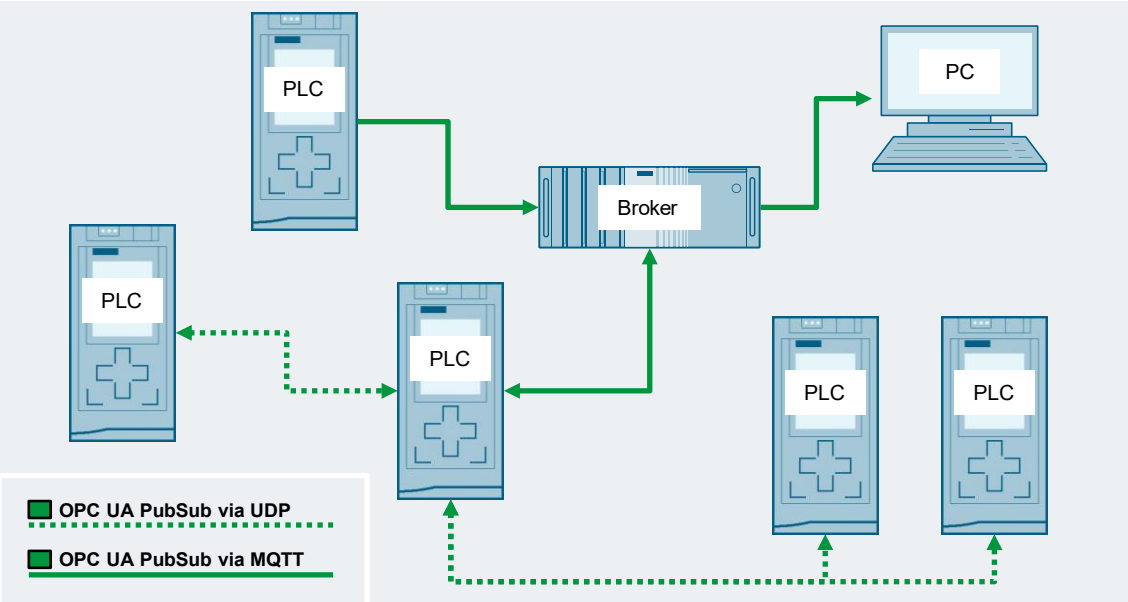


Figure 6-1: Overview

The "LOpcUa" library provides function blocks that can be used to implement OPC UA PubSub on SIMATIC controllers:

- LOpcUa_PubUdp: Publisher via UDP (UADP encoding)
- LOpcUa_SubUdp: Subscriber via UDP (UADP decoding)
- LOpcUa_PubMqtt: Publisher via MQTT (UADP encoding
- LOpcUa_SubMqtt: Subscriber via MQTT (UADP decoding)
- LOpcUa_PubMqttJson: Publisher via MQTT (JSON-Encoding)
- LOpcUa_SubMqttJson: Subscriber via MQTT (JSON-Decoding)

NOTE The blocks "LOpcUa_PubUdp" and "LOpcUa_SubUdp" do not support security.

© Siemens 2026

6.1.2. Components of the Library

The "LOpcUa" library contains the following objects.

Blocks

Table 6-1: Blocks of the library

Name	Type	Version	Description
LOpcUa_PubUdp	FB	V1.3.2	Implements the OPC UA PubSub Publisher via UDP and enables process data to be transmitted to a Subscriber.
LOpcUa_SubUdp	FB	V1.3.2	Implements the OPC UA PubSub Subscriber via UDP and enables process data to be received from a Publisher.
LOpcUa_PubMqttUadp	FB	V2.3.0	Implements the OPC UA PubSub Publisher via MQTT and enables process data to be transmitted to a Subscriber. Uses "UADP" for encoding.
LOpcUa_SubMqttUadp	FB	V2.3.0	Implements the OPC UA PubSub Subscriber via MQTT and enables process data to be received from a Publisher. Uses "UADP" for decoding.
LOpcUa_PubMqttJson	FB	V2.4.0	Implements the OPC UA PubSub Publisher via MQTT and enables process data to be transmitted to a Subscriber. Uses "JSON" for encoding.
LOpcUa_SubMqttJson	FB	V2.3.0	Implements the OPC UA PubSub Subscriber via MQTT and enables process data to be received from a Publisher. Uses "JSON" for decoding.
LOpcUa_Pub_Boolean	FC	V1.0.1	Encoding for "LOpcUa_PubX": Boolean to send buffer (byte array).
LOpcUa_Pub_Byte	FC	V1.0.1	Encoding for "LOpcUa_PubX": Byte to send buffer (byte array).
LOpcUa_Pub_Double	FC	V1.0.1	Encoding for "LOpcUa_PubX": Double to send buffer (byte array).
LOpcUa_Pub_Float	FC	V1.0.1	Encoding for "LOpcUa_PubX": Float to send buffer (byte array).
LOpcUa_Pub_Guid	FC	V1.0.1	Encoding for "LOpcUa_PubX": GUID to send buffer (byte array).
LOpcUa_Pub_Int16	FC	V1.0.1	Encoding for "LOpcUa_PubX": Int16 to send buffer (byte array).
LOpcUa_Pub_Int32	FC	V1.0.1	Encoding for "LOpcUa_PubX": Int32 to send buffer (byte array).
LOpcUa_Pub_Int64	FC	V1.0.1	Encoding for "LOpcUa_PubX": Int64 to send buffer (byte array).
LOpcUa_Pub_SByte	FC	V1.0.1	Encoding for "LOpcUa_PubX": SByte to send buffer (byte array).
LOpcUa_Pub_String	FC	V1.1.1	Encoding for "LOpcUa_PubX": String to send buffer (byte array).
LOpcUa_Pub_UInt16	FC	V1.0.1	Encoding for "LOpcUa_PubX": UInt16 to send buffer (byte array).
LOpcUa_Pub_UInt32	FC	V1.0.1	Encoding for "LOpcUa_PubX": UInt32 to send buffer (byte array).
LOpcUa_Pub_UInt64	FC	V1.0.1	Encoding for "LOpcUa_PubX": UInt64 to send buffer (byte array).

© Siemens 2026

PLC Data Types

Table 6-2: PLC library data types

Name	Version	Description
LOpcUa_typeDataSetWriter	V1.0.0	Maps an OPC UA "DataSetWriter".
LOpcUa_typeDataSetReader	V1.0.3	Maps an OPC UA "DataSetReader".
LOpcUa_typeGuid	V1.0.0	Implements the "GUID" data type.
LOpcUa_typePublishedData	V1.0.0	Implements the send buffer of the Publisher.
LOpcUa_typePublishedDataSet	V1.0.1	Maps an OPC UA "PublishedDataSet". Contains the Publisher process values to be sent.
LOpcUa_typePublisherID	V1.0.4	Maps an OPC UA "PublisherID". Serves to identify the Publisher.
LOpcUa_typeVariant	V1.0.1	Implements the "Variant" data type and maps a "DataSetField".
LOpcUa_typeWriterGroup	V1.0.0	Maps an OPC UA "WriteGroup".
LOpcUa_typeReaderGroup	V1.0.3	Maps an OPC UA "ReaderGroup".
LOpcUa_typeSubscribedDataSet	V1.0.1	Maps an OPC UA "SubscribedDataSet". Contains the received process values of the Publisher.
LOpcUa_typeConnParamMqtt	V2.0.0	Connection parameters for "LOpcUa_PubMqtt" and "LOpcUa_SubMqtt".
LOpcUa_typeDataSetWriterJson	V1.0.0	Maps an OPC UA "DataSetWriter" for JSON encoding.
LOpcUa_typeDataSetReaderJson	V1.0.0	Maps an OPC UA "DataSetReader" for JSON decoding.
LOpcUa_typePublishedDataSet Json	V1.0.0	Maps an OPC UA "PublishedDataSet" for JSON encoding. Contains the Publisher process values to be sent.
LOpcUa_typeSubscribedDataSet Json	V1.0.0	Maps an OPC UA "SubscribedDataSet" for JSON decoding. Contains the received process values of the Publisher.
LOpcUa_typeWriterGroupJson	V1.0.0	Maps an OPC UA "WriteGroup" for JSON encoding.
LOpcUa_typeReaderGroupJson	V1.0.0	Maps an OPC UA "ReaderGroup" for JSON decoding.

6.1.3. Validity

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- CP 1543-1, CP 1542SP-1, CP 1542SP-1 IRC, CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V2.0

© Siemens 2026 |

6.2. PubSub via UDP

6.2.1. Principle of Operation

General information

The Publisher is implemented by the function block "LOpcUa_PubUdp" and the Subscriber by the block "LOpcUa_SubUdp", which are provided by the library "LOpcUa":

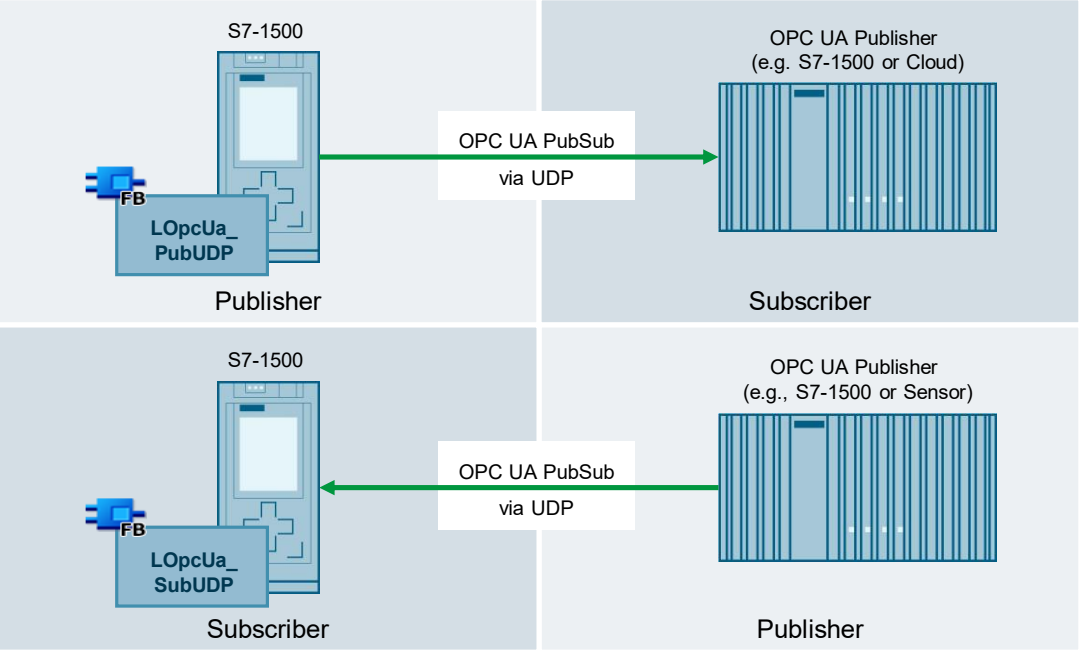


Figure 6-2: Principle of operation

To generate the data storage in a data block, for the process data to be sent or received, the library provides prepared user-defined data types ("UDT"). The Publisher and Subscriber blocks access this data to implement the respective function.

© Siemens 2026 |

Schematic representation

The following figure shows the most important relationships between the components involved in sending data from a Publisher to a Subscriber:

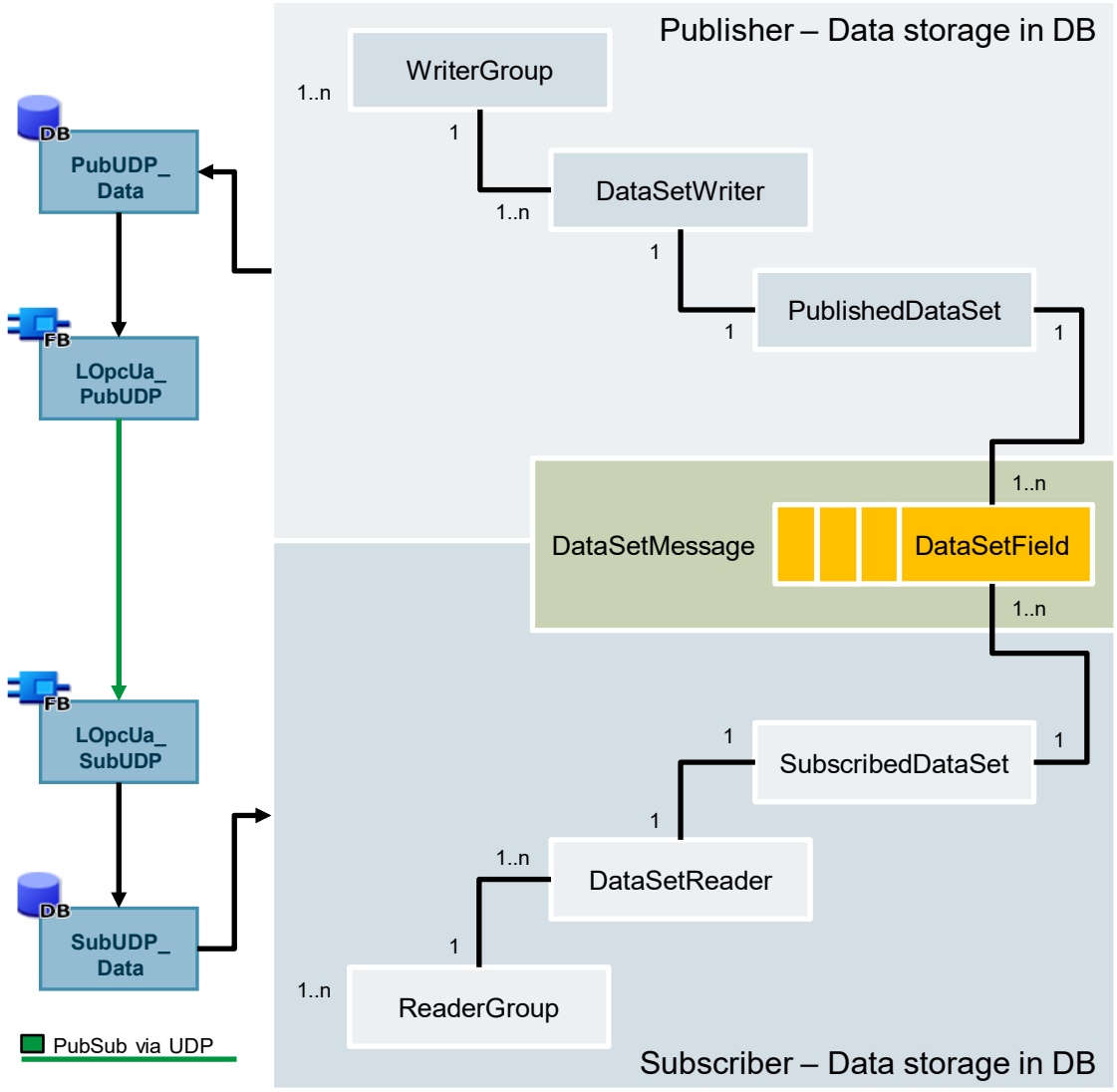


Figure 6-3

To map one or more "WriterGroups" or "ReaderGroups", both the Publisher and the Subscriber function blocks require a data block in which the PubSub data (metadata and process data) are provided according to the schema shown above.

To implement the individual components in data blocks, the library provides you with suitable PLC data types that map the required structures. If a component is required more than once ("1..n"), it must be created as an array in the data block.

The block "LOpcUa_PubUdp" accesses the "WriteGroups" and serializes the "DataSetMessage" from the data contained therein and sends ("Publish") this with the help of the system function block "TUSEND" to the Subscribers.

The block "LOpcUa_SubUdp" receives the "DataSetMessage" via the system function block "TURCV" and de-serializes the data to the data structure of the created "ReaderGroup".

© Siemens 2026

6.2.2. LOpcUa_PubUdp

Description

The block "LOpcUa_PubUdp" sends ("Publish") the PubSub data via UDP to the Subscribers. The block requires a data block in which the components "WriterGroup", "DataSetWriter", "PublishedDataSet", and "PublishedWriterGroup" are predefined.

In an internal sequencer of the block, the data packets are coded and sent via UDP with the help of the system function blocks "TCON", "TUSEND", and "TDISCON".

NOTE To ensure a consistent send clock, we recommend calling the block in a cyclic interrupt OB. The "PublishingInterval" is defined in the "WriterGroups".

Parameter

Figure 6-4: LOpcUa_PubUdp

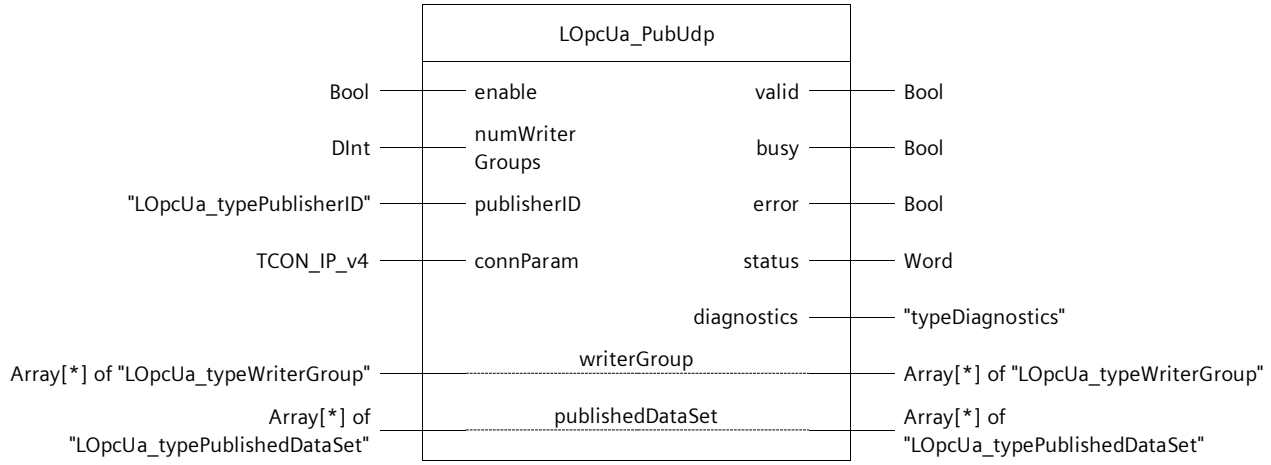


Table 6-3: Parameters of LOpcUa_PubUdp

Name	Declaration	Data type	Comment
enable	Input	Bool	Control parameters TRUE: Cyclic publishing active. FALSE: Publishing disabled.
numWriterGroups	Input	DInt	Number of "WriteGroups" to be published.
publisherID	Input	WString	"PublisherID" to uniquely identify the publisher.
connParam	Input	TCON_IP_v4	UDP connection parameters.
valid	Output	Bool	TRUE, if publishing is active at the block and no error occurs.
busy	Output	Bool	TRUE, if publishing is active at the block.
error	Output	Bool	TRUE: An error has occurred. More detailed information is provided by the output parameter "status" or the diagnostic buffer. FALSE: No error occurred.
status	Output	Word	Status display Information about the error that occurred in FB "LOpcUa_PubUdp" (see chapter 6.2.4).

© Siemens 2026

Name	Declaration	Data type	Comment
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2).
writerGroup	InOut	Array[*] of "LOpcUa_typeWriterGroup"	Data structure for mapping one or more "WriterGroups". Contains metadata for PubSub communication.
publishedDataSet	InOut	Array[*] of "LOpcUa_typePublishedDataSet"	Data structure for mapping one or more "PublishedDataSets". This structure contains one process tag each, which is published.

6.2.3. LOpcUa_SubUdp

Description

The block "LOpcUa_SubUdp" receives ("Subscribe") the PubSub data via UDP from a Publisher. The block requires a data block in which the "ReaderGroup" component is predefined.

In an internal sequencer of the block, the data packets are received and decoded via the system function blocks "TCON", "TURCV", and "TDISCON" using UDP.

To receive the data with minimum delay we recommend calling the block in a cyclic OB.

Parameter

Figure 6-5: LOpcUa_SubUdp

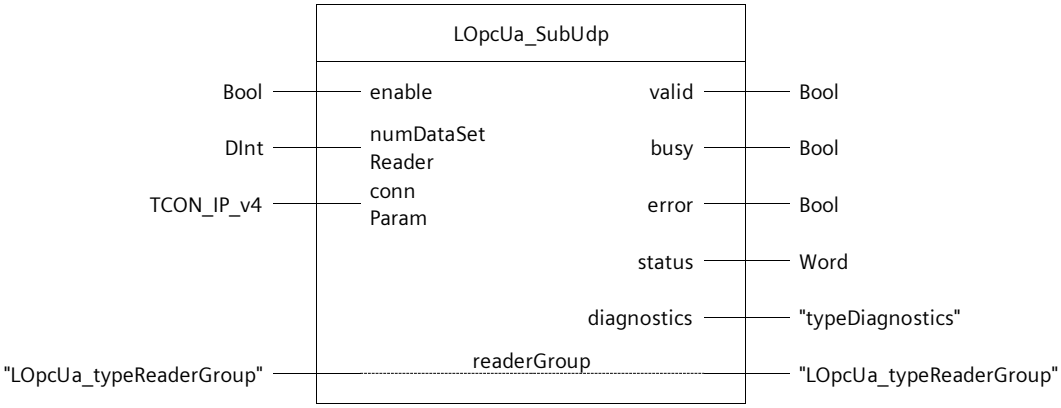


Table 6-4: Parameters of LOpcUa_SubUdp

Name	Declaration	Data type	Comment
enable	Input	Bool	Control parameters TRUE: Cyclic receiving active. FALSE: Receive disabled.
numDataSetReader	Input	DInt	Number of "DataSetReaders" which should receive the data of the "DataSetWriters" of a Publisher.
connParam	Input	TCON_IP_v4	UDP connection parameters.
valid	Output	Bool	TRUE if subscribing is active on the block and no error occurs.
busy	Output	Bool	TRUE if subscribing is active on the block.

© Siemens 2026 |

Name	Declaration	Data type	Comment
error	Output	Bool	TRUE: An error has occurred. More detailed information is provided by the output parameter "status" or the diagnostic buffer. FALSE: No error occurred.
status	Output	Word	Status display Information about the error that occurred in FB "LOpcUa_SubUdp" (see chapter 6.2.4).
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2).
readerGroup	InOut	"LOpcUa_type ReaderGroup"	Data structure for mapping a "ReaderGroup". Contains the structures of the "DataSetReader".

NOTE The block "LOpcUa_SubUdp" supports only one "ReaderGroup".

6.2.4. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The status and error codes specific to the blocks "LOpcUa_PubUdp" and "LOpcUa_SubUdp" are listed below.

Table 6-5: Block output "status"

Code	Description
16#7000	No error; block ready for execution.
16#7002	No error; publishing/subscribing via UDP active.
16#8600	Configuration error: Check whether enough "WriterGroups", related to the block input "numWriterGroups", have been created. In addition, check whether the fields of the structures are assigned correctly.
16#8601	Connection error for TCON: For more information, refer to the "diagnostics" block output.
16#8602	Receive error for TURCV: For more information, refer to the "diagnostics" block output.
16#8603	Send error for TUSEND: For more information, refer to the "diagnostics" block output.
16#8604	Error while encoding the data: Check the metadata on the "writerGroup" parameter.
16#8605	Error decoding the data: Received data type is not supported. Extend the function block or use a different data type with the Publisher.

6.2.5. Integration into the User Project

You will find a detailed application example for the integration of the library into your user project and further information about OPC UA PubSub in the Siemens Industry Online Support:

https://support.industry.siemens.com/cs/ww/en/view/109782455

© Siemens 2026 |

6.3. PubSub via MQTT

6.3.1. Principle of Operation

General information

The Publisher is implemented by the function block "LOpcUa_PubMqtt", and the Subscriber is implemented by the function block "LOpcUa_SubMqtt", which is provided by the library "LOpcUa":

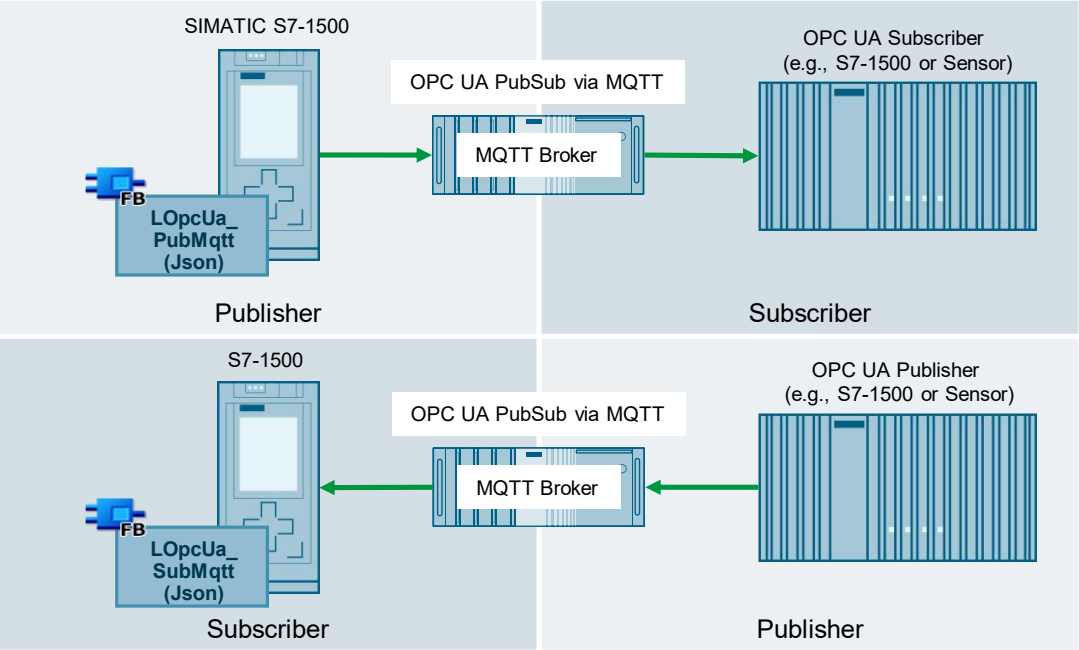


Figure 6-6: Principle of operation

To generate the data storage in a data block, for the process data to be sent or received, the library provides prepared user-defined data types ("UDT"). The Publisher and Subscriber blocks access this data store to implement the respective function. Depending on the chose block the encoding is done via UADP or JSON.

© Siemens 2026 |

Schematic representation

The following figure shows the most important relationships between the components involved in sending data from a Publisher to a Subscriber:

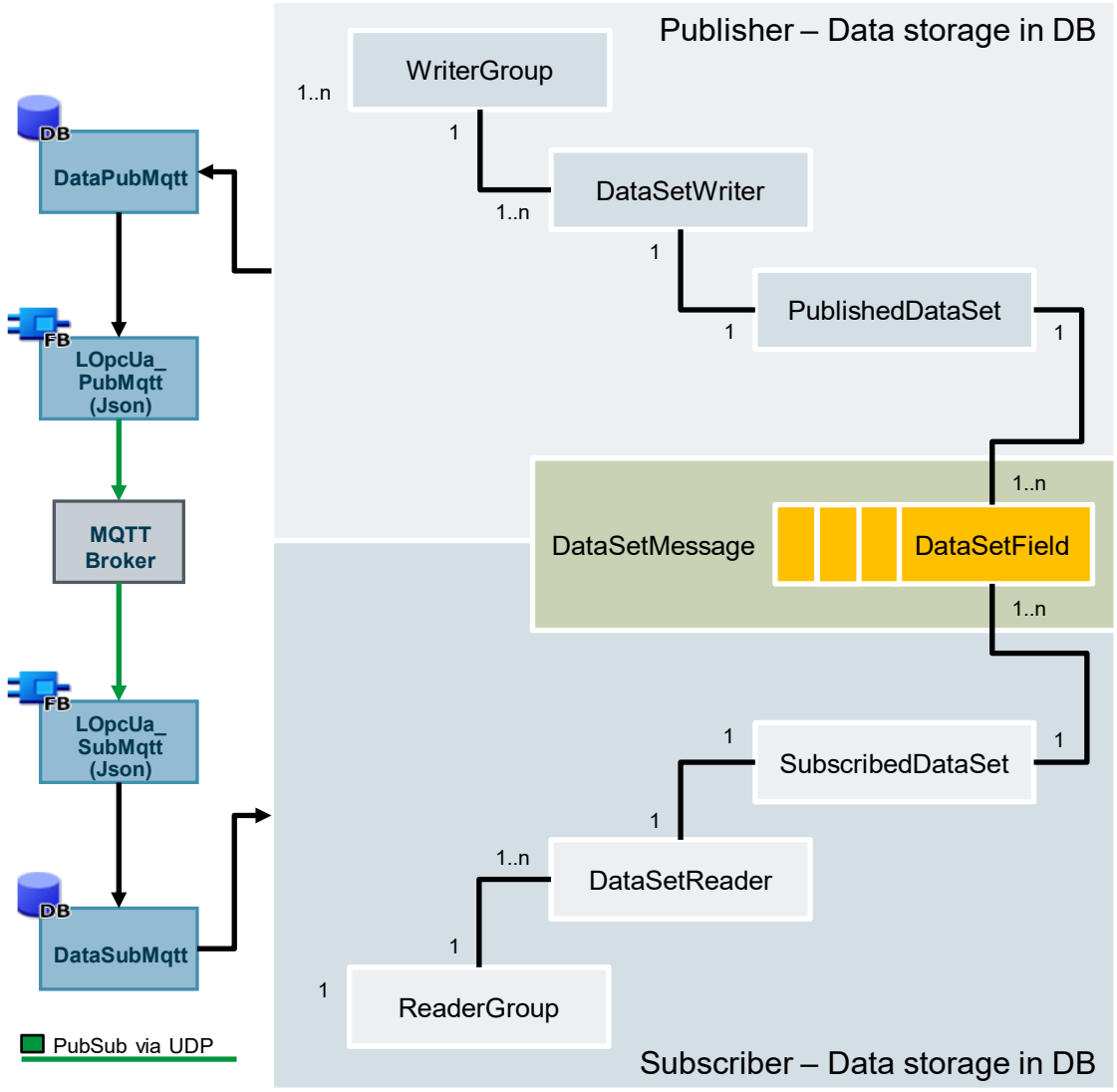


Figure 6-7

To map one or more "WriterGroups" or "ReaderGroups", both the Publisher and the Subscriber function blocks require a data block in which the PubSub data (metadata and process data) are provided according to the schema shown above.

To implement the individual components in data blocks, the library provides you with suitable PLC data types that map the required structures. If a component is required more than once ("1..n"), it must be created as an array in the data block.

The block "LOpcUa_PubUdp" accesses the "WriteGroups" and serializes the "DataSetMessage" from the data contained therein and sends ("Publish") this with the help of the system function block "TUSEND" to the Subscribers.

The block "LOpcUa_SubUdp" receives the "DataSetMessage" via the system function block "TURCV" and de-serializes the data to the data structure of the created "ReaderGroup".

© Siemens 2026

6.3.2. LOpcUa_PubMqtt

Description

The block "LOpcUa_PubMqtt" sends ("Publish") the PubSub data via MQTT to an MQTT Broker. The block requires a data block in which the components "WriterGroup", "DataSetWriter", "PublishedDataSet", and "PublishedWriterGroup" are predefined.

In an internal step chain of the block, the data packets are coded and sent via MQTT using the library block "LMQTT_Client" is used to send them via MQTT.

NOTE To ensure a consistent send clock, we recommend to call the block in a cyclic interrupt OB. The "PublishingInterval" is defined in the "WriterGroups".

Parameter

Figure 6-8: LOpcUa_PubMqtt

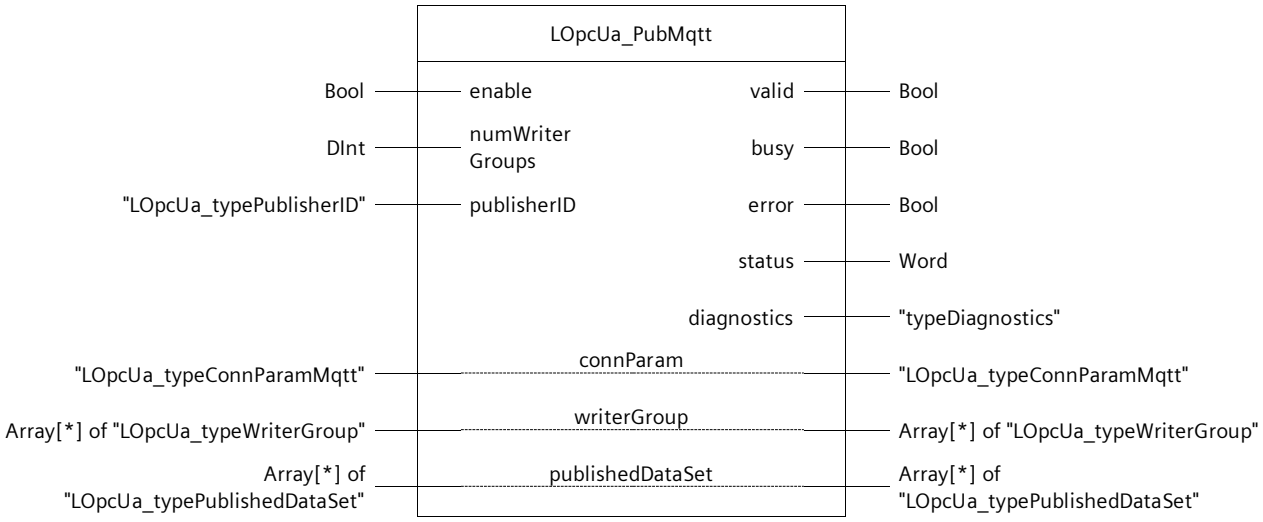


Table 6-6: Parameter of LOpcUa_PubMqtt

Name	Declaration	Data type	Comment
enable	Input	Bool	Control parameters TRUE: Cyclic publishing active. FALSE: Publishing disabled.
numWriterGroups	Input	DInt	Number of "WriteGroups" to be published.
publisherID	Input	"LOpcUa_type PublisherID"	"PublisherID" to uniquely identify the publisher.
valid	Output	Bool	TRUE, if publishing is active at the block and no error occurs.
busy	Output	Bool	TRUE, if publishing is active at the block.
error	Output	Bool	TRUE: An error has occurred. More detailed information is provided by the output parameter "status" or the diagnostic buffer. FALSE: No error occurred.

© Siemens 2026

Name	Declaration	Data type	Comment
status	Output	Word	Status display Information about the error that occurred in FB "LOpcUa_PubUdp" (see chapter 6.3.6).
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2).
connParam	InOut	"LOpcUa_type ConnParamMqtt"	MQTT connection parameters.
writerGroup	InOut	Array[*] of "LOpcUa_type WriterGroup"	Data structure for mapping one or more "WriterGroups". Contains metadata for PubSub communication.
publishedDataSet	InOut	Array[*] of "LOpcUa_type PublishedDataSet"	Data structure for mapping one or more "PublishedDataSets". This structure contains one process tag each, which is published.

6.3.3. LOpcUa_SubMqtt

Description

The block "LOpcUa_SubMqtt" receives ("Subscribe") the PubSub data via MQTT from an MQTT Broker. The block requires a data block in which the "ReaderGroup" component is predefined.

In an internal step chain of the block, the data packets are received and decoded via MQTT using the library block "LMQTT_Client" library module to receive and decode the data packets via MQTT.

To receive the data with minimum delay we recommend to call the block in a cyclic OB.

Parameter

Figure 6-9: LOpcUa_SubMqtt

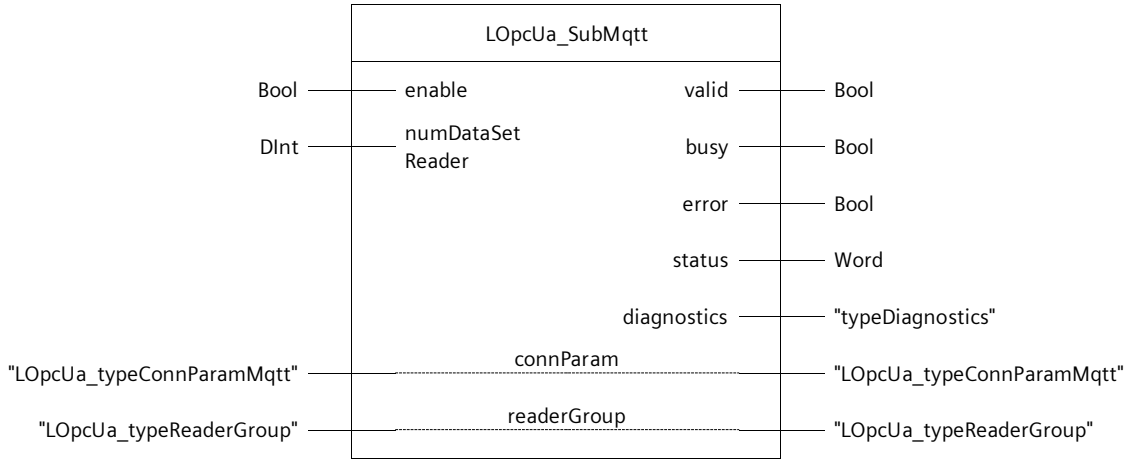


Table 6-7: Parameter of LOpcUa_SubMqtt

Name	Declaration	Data type	Comment
enable	Input	Bool	Control parameters TRUE: Cyclic receiving active. FALSE: Receive disabled.
numDataSetReader	Input	DInt	Number of "DataSetReaders" which should receive the data of the "DataSetWriters" of a Publisher.

© Siemens 2026 |

Name	Declaration	Data type	Comment
valid	Output	Bool	TRUE if subscribing is active on the block and no error occurs.
busy	Output	Bool	TRUE if subscribing is active on the block.
error	Output	Bool	TRUE: An error has occurred. More detailed information is provided by the output parameter "status" or the diagnostic buffer. FALSE: No error occurred.
status	Output	Word	Status display Information about the error that occurred in FB "LOpcUa_SubUdp" (see chapter 6.3.6).
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2).
connParam	InOut	"LOpcUa_type ConnParamMqtt"	MQTT connection parameters.
readerGroup	InOut	"LOpcUa_type ReaderGroup"	Data structure for mapping a "ReaderGroup". Contains the structures of the "DataSetReader".

NOTE The block "LOpcUa_SubMqtt" supports only one "ReaderGroup".

6.3.4. LOpcUa_PubMqttJson

Description

The block "LOpcUa_PubMqttJson" sends ("Publish") the PubSub data via MQTT to an MQTT Broker. The block requires a data block in which the components "WriterGroup", "DataSetWriter", "PublishedDataSet", and "PublishedWriterGroup" are predefined.

In an internal step chain of the block, the data packets are coded and sent via MQTT using the library block "LMQTT_Client" is used to send them via MQTT.

NOTE To ensure a consistent send clock, we recommend to call the block in a cyclic interrupt OB. The "PublishingInterval" is defined in the "WriterGroups".

© Siemens 2026

Parameter

Figure 6-10: LOpcUa_PubMqttJson

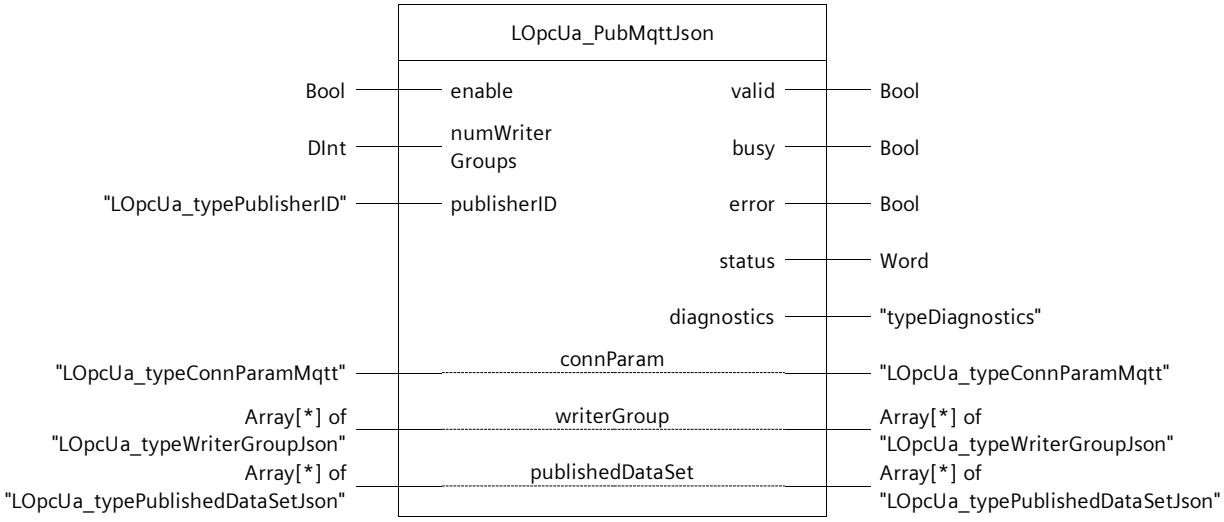


Table 6-8: Parameter of LOpcUa_PubMqtt

Name	Declaration	Data type	Comment
enable	Input	Bool	Control parameters TRUE: Cyclic publishing active. FALSE: Publishing disabled.
numWriterGroups	Input	DInt	Number of "WriteGroups" to be published.
publisherID	Input	WString	"PublisherID" to uniquely identify the publisher.
valid	Output	Bool	TRUE, if publishing is active at the block and no error occurs.
busy	Output	Bool	TRUE, if publishing is active at the block.
error	Output	Bool	TRUE: An error has occurred. More detailed information is provided by the output parameter "status" or the diagnostic buffer. FALSE: No error occurred.
status	Output	Word	Status display Information about the error that occurred in FB "LOpcUa_PubUdp" (see chapter 6.3.6).
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2).
connParam	InOut	"LOpcUa_typeConnParamMqtt"	MQTT connection parameters.
writerGroup	InOut	Array[*] of "LOpcUa_typeWriterGroupJson"	Data structure for mapping one or more "WriterGroups". Contains metadata for PubSub communication.
publishedDataSet	InOut	Array[*] of "LOpcUa_typePublishedDataSetJson"	Data structure for mapping one or more "PublishedDataSets". This structure contains one process tag each, which is published.

© Siemens 2026

6.3.5. LOpcUa_SubMqttJson

Description

The block "LOpcUa_SubMqttJson" receives ("Subscribe") the PubSub data via MQTT from an MQTT Broker. The block requires a data block in which the "ReaderGroup" component is predefined.

In an internal step chain of the block, the data packets are received and decoded via MQTT using the library block "LMQTT_Client" library module to receive and decode the data packets via MQTT.

To receive the data with minimum delay we recommend calling the block in a cyclic OB.

Parameter

Figure 6-11: LOpcUa_SubMqttJson

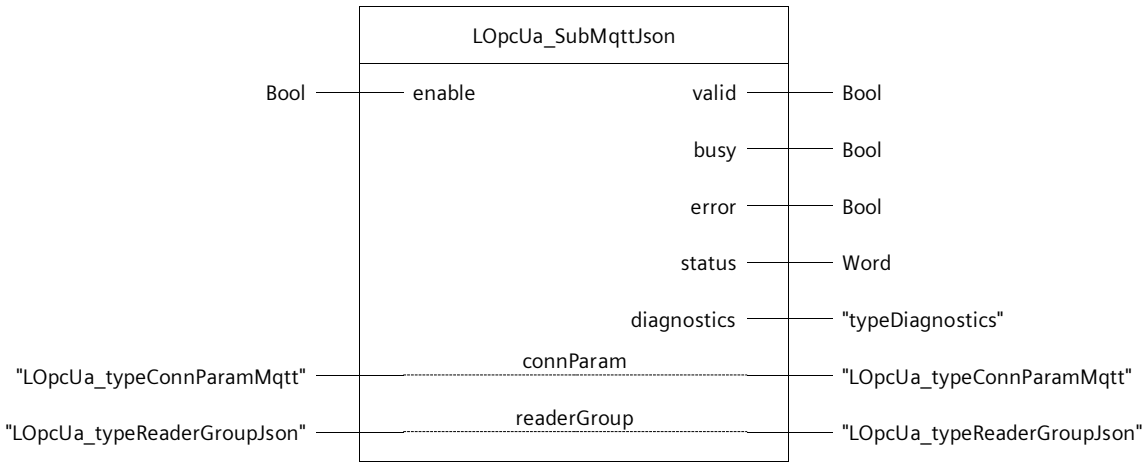


Table 6-9: Parameter of LOpcUa_SubMqtt

Name	Declaration	Data type	Comment
enable	Input	Bool	Control parameters TRUE: Cyclic receiving active. FALSE: Receive disabled.
valid	Output	Bool	TRUE if subscribing is active on the block and no error occurs.
busy	Output	Bool	TRUE if subscribing is active on the block.
error	Output	Bool	TRUE: An error has occurred. More detailed information is provided by the output parameter "status" or the diagnostic buffer. FALSE: No error occurred.
status	Output	Word	Status display Information about the error that occurred in FB "LOpcUa_SubUdp" (see chapter 6.3.6).
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2).
connParam	InOut	"LOpcUa_typeConnParamMqtt"	MQTT connection parameters.
readerGroup	InOut	"LOpcUa_typeReaderGroupJson"	Data structure for mapping a "ReaderGroup". Contains the structures of the "DataSetReader".

© Siemens 2026

NOTE The block "LOpcUa_SubMqttJson" supports only one "ReaderGroup".

6.3.6. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The status and error codes specific to the blocks "LOpcUa_PubMqtt" and "LOpcUa_SubMqtt" are listed below.

Table 6-10: Block output "status"

Code	Description
16#7000	No error; block ready for execution.
16#7002	No error; publishing/subscribing via UDP active.
16#8600	Configuration error: Check whether enough "WriterGroups", related to the block input "numWriterGroups", have been created. In addition, check whether the fields of the structures are assigned correctly.
16#8601	Connection error for "LMQTT_Client": For more information, refer to the "diagnostics" block output or check the "Status" parameter in the "instLMQTT_Client" multi-instance.
16#8602	Receive error for "LMQTT_Client": For more information, refer to the "diagnostics" block output or check the "Status" parameter in the "instLMQTT_Client" multi-instance.
16#8603	Send error for "LMQTT_Client": For more information, refer to the "diagnostics" block output or check the "Status" parameter in the "instLMQTT_Client" multi-instance.
16#8604	Error while encoding the data: Check the metadata on the "writerGroup" parameter.
16#8605	Error decoding the data: Received data type is not supported. Extend the function block or use a different data type with the Publisher.

6.3.7. Integration into the User Project

You will find a detailed application example for the integration of the library into your user project and further information about OPC UA PubSub in the Siemens Industry Online Support:

- MQTT with UADP-Encoding
  https://support.industry.siemens.com/cs/ww/en/view/109797826
- MQTT with JSON-Encoding
  https://support.industry.siemens.com/cs/ww/en/view/109814033

6.4. PLC Data Types

The following descriptions explain the PLC data types that map the PubSub components for communication.

LOpcUa_typePublisherID

The PLC data type "LOpcUa_typePublisherID" is used to identify the Publisher.

© Siemens 2026

Table 6-11: Parameters of "LOpcUa_typePublisherID"

Name	Data type	Description
type	UDInt	Selects the data type to be used for the Publisher ID to be sent: • "3": id8 • "5": id16 • "7": id32 • "9": id64 • "12": idString
id8	USInt	The tag selected with "type" contains the Publisher ID that identifies your Publisher. You can choose the ID freely. Example: *"type" = "7"* corresponds to *"id32 (UDInt)"*
id16	UInt	
id32	UDInt	
id64	ULInt	
idString	WString	

LOpcUa_typeWriterGroup

The PLC data type "LOpcUa_typeWriterGroup" implements the WriterGroups.

Table 6-12: Parameters of "LOpcUa_typeWriterGroup"

Name	Data type	Description
name	String	The name of the "WriterGroup" (optional, will not be transferred).
priority	USInt	The priority for sorting (higher value = higher priority).
publishingInterval	USInt	The publishing interval for the configuration of the send clock. The transmission rate is given by the following formula: *Send clock = OB cycle time * PublishingInterval*
resetInterval	USInt	Must contain the same value as the "PublishingInterval".
numDataSetWriter	USInt	Number of "DataSetWriters" you want to use.
dataSetWriter	Array [0..X] of "LOpcUa_typeDataSetWriter"	Implements the "DataSetWriter" per "WriterGroup".

© Siemens 2026 |

LOpcUa_typeDataSetWriter

The PLC data type "LOpcUa_typeDataSetWriter" implements the DataSetWriter.

Table 6-13: Parameters of "LOpcUa_typeDataSetWriter"

Name	Data type	Description
name	String	The name of the "DataSetWriter" (optional, will not be transferred).
id	UInt	Identification number of the "DataSetWriter" (any).
sequenceNumber	UInt	Is not filled in (required by FB "LOpcUa_PubUdp").
dataSet	UInt	Reference to the index of the "PublishedDataSet". Each "DataSet" refers to a different index of the "PublishedDataSets".

LOpcUa_typePublishedDataSet

The PLC data type "LOpcUa_typePublishedDataSet" implements the PublishedDataSets.

Table 6-14: Parameters of "LOpcUa_typePublishedDataSet"

Name	Data type	Description
name	String	The name of the "PublishedDataSet" (optional, will not be transmitted).
fieldCount	UInt	Number of "DataSetFields" in the "PublishedDataSet" to be published.
dataSetField	Array [0..X] of "LOpcUa_typeVariant"	Implements the "DataSetFields" which contain the individual process tags to be sent.

LOpcUa_typeVariant

The PLC data type "LOpcUa_typeVariant" implements the data type "Variant" for PubSub communication.

Table 6-15: Parameters of "LOpcUa_typeVariant"

Name	Data type	Description
encodingMask	USInt	Specifies the data type of the process value to be sent or received.
value	Dependent on "encodingMask"	Used field according to the "encodingMask". Take the assignment between "encodingMask" and "Value" from the comments in the data type "LOpcUa_typeVariant". The corresponding field contains the process value that you can describe in your user program for sending. Example: "encodingMask" = "1" corresponds to "boolean"

LOpcUa_typeReaderGroup

The PLC data type "LOpcUa_typeReaderGroup" implements the ReaderGroup.

Table 6-16: Parameters of "LOpcUa_typeReaderGroup"

Name	Data type	Description
name	String	The name of the "ReaderGroup" (optional).
dataSetReader	Array [0..X] of "LOpcUa_typeDataSetReader"	Implements the "DataSetReader" per "ReaderGroup".

© Siemens 2026 |

LOpcUa_typeDataSetReader

The PLC data type "LOpcUa_ typeDataSetReader" implements the DataSetReader.

Table 6-17: Parameters of "LOpcUa_typeDataSetReader"

Name	Data type	Description
name	String	The name of the "DataSetReader" (optional).
publisherID	"LOpcUa_type PublisherID"	The received PublisherID.
dataSetWriterID	UInt	The received DataSetWriterID.
sequenceNumber	UInt	Sequence number of the received PubSub packets
subscribedDataSet	"LOpcUa_type SubscribedDataSet"	Contains the "DataSetFields" of type "LOpcUa_typeVariant" in which the received process values are stored.

LOpcUa_typeSubscribedDataSet

The PLC data type "LOpcUa_typeSubscribedDataSet" implements the SubscribedDataSets.

Table 6-18: Parameters of "LOpcUa_typeSubscribedDataSet"

Name	Data type	Description
name	String	The name of the SubscribedDataSet" (optional, will not be transmitted).
fieldCount	UInt	Number of "DataSetFields" in the "PublishedDataSet" to be published.
dataSetField	Array [0..X] of "LOpcUa_ typeVariant"	Implements the "DataSetFields" which contain the individual process tags to be sent.

LOpcUa_typeWriterGroupJson

The PLC data type "LOpcUa_typeWriterGroupJson" implements the WriterGroups.

Table 6-19: Parameters of "LOpcUa_typeWriterGroupJson"

Name	Data type	Description
name	String	The name of the "WriterGroup" (optional, will not be transferred).
priority	USInt	The priority for sorting (higher value = higher priority).
publishingInterval	USInt	The publishing interval for the configuration of the send clock. The transmission rate is given by the following formula: *Send clock = OB cycle time * PublishingInterval*
resetInterval	USInt	Must contain the same value as the "PublishingInterval".
numDataSetWriter	USInt	Number of "DataSetWriters" you want to use.
dataSetWriter	Array [0..X] of "LOpcUa_ typeDataSetWriterJson"	Implements the "DataSetWriter" per "WriterGroup".

© Siemens 2026 |

LOpcUa_typeDataSetWriterJson

The PLC data type "LOpcUa_typeDataSetWriterJson" implements the DataSetWriter.

Table 6-20: Parameters of "LOpcUa_typeDataSetWriterJson"

Name	Data type	Description
id	WString	Identification number of the "DataSetWriter" (any).
sequenceNumber	UInt	Is not filled in (required by FB "LOpcUa_PubUdpJson").
dataSet	UInt	Reference to the index of the "PublishedDataSet". Each "DataSet" refers to a different index of the "PublishedDataSets".

LOpcUa_typePublishedDataSetJson

The PLC data type "LOpcUa_typePublishedDataSetJson" implements the PublishedDataSets.

Table 6-21: Parameters of "LOpcUa_typePublishedDataSetJson"

Name	Data type	Description
name	WString	The name of the "PublishedDataSet".
fieldCount	UInt	Number of "DataSetFields" in the "PublishedDataSet" to be published.
dataSetField	Array [0..X] of "LOpcUa_ typeVariant"	Implements the "DataSetFields" which contain the individual process tags to be sent.

LOpcUa_typeReaderGroupJson

The PLC data type "LOpcUa_typeReaderGroupJson" implements the ReaderGroup.

Table 6-22: Parameters of "LOpcUa_typeReaderGroupJson"

Name	Data type	Description
name	String	The name of the "ReaderGroup" (optional).
dataSetReader	Array [0..X] of "LOpcUa_ typeDataSetReaderJson"	Implements the "DataSetReader" per "ReaderGroup".

LOpcUa_typeDataSetReaderJson

The PLC data type "LOpcUa_ typeDataSetReaderJson" implements the DataSetReader.

Table 6-23: Parameters of "LOpcUa_typeDataSetReaderJson"

Name	Data type	Description
name	String	The name of the "DataSetReader" (optional).
publisherID	WString	The received PublisherID.
dataSetWriterID	WString	The received DataSetWriterID.
sequenceNumber	UInt	Sequence number of the received PubSub packets
subscribedDataSet	"LOpcUa_type SubscribedDataSetJson"	Contains the "DataSetFields" of type "LOpcUa_typeVariant" in which the received process values are stored.

© Siemens 2026 |

LOpcUa_typeSubscribedDataSetJson

The PLC data type "LOpcUa_typeSubscribedDataSetJson" implements the SubscribedDataSets.

Table 6-24: Parameters of "LOpcUa_typeSubscribedDataSetJson"

Name	Data type	Description
name	WString	The name of the SubscribedDataSet" (optional, will not be transmitted).
fieldCount	UInt	Number of "DataSetFields" in the "PublishedDataSet" to be published.
dataSetField	Array [0..X] of "LOpcUa_typeVariant"	Implements the "DataSetFields" which contain the individual process tags to be sent.

LOpcUa_typeConnParamMqtt

The PLC data type "LOpcUa_typeConnParamMqtt" defines the connection parameters for the MQTT connection.

Table 6-25: Parameters of "LOpcUa_typeConnParamMqtt"

Name	Data type	Description
mqttConnParam	"LMQTT_typeConnParam"	Basic MQTT connection parameters.
mqttClientIdentifier	WString	MQTT Client identifier used when establishing the connection.
mqttUserName	WString	Optional: Username for connection establishment.
mqttPassword	WString	Optional: Password for connection establishment.
mqttTopic	WString	MQTT topic used for publish/subscribe.

© Siemens 2026 |

7. LSNMP

7.1. Overview

7.1.1. Range of functions

The status of SNMP-capable network components can be monitored and, if necessary, controlled by network management systems, such as SINEC NMS, using the “Simple Network Management Protocol”, short “SNMP”.

S7-CPU as SNMP Manager

The blocks of the "LSNMP" library allow a SIMATIC Controller with a PROFINET interface, acting as a simple SNMP manager, to query information from the network components and, if necessary, to control them as well.

The blocks of this library support the following functions:

- Request and Response-based data access : Get, GetNext, GetBulk and Set
- Receive Traps from SNMP Agents

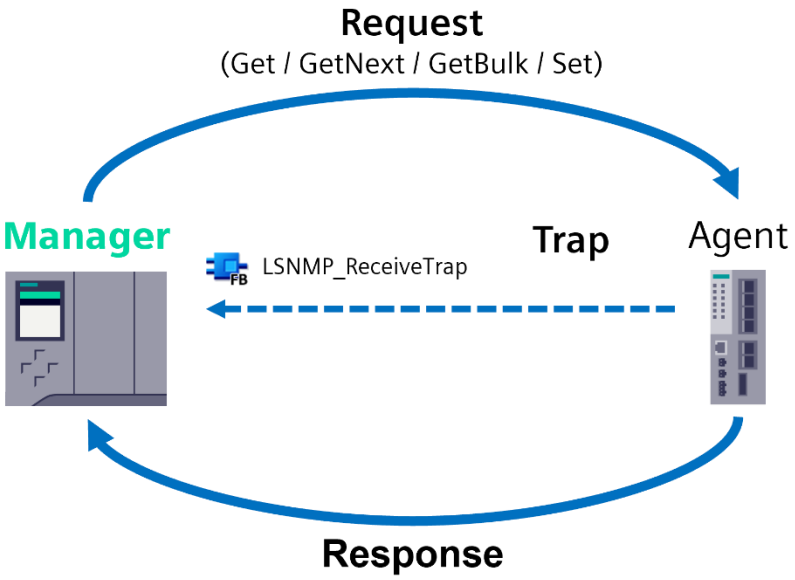


Figure 7-1: S7-CPU as SNMP Manager

S7-CPU as SNMP Agent

The blocks of this library support the following functions:

- Send Traps to SNMP Managers

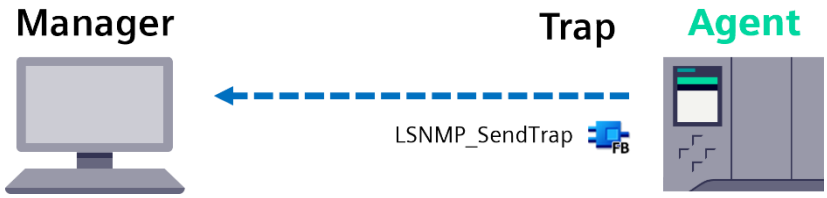


Figure 7-2: S7-CPU as SNMP Agent

© Siemens 2026 |

7.1.2. Function blocks

The following table shows which function blocks must be used to perform the different functions:

Table 7-1: Function blocks of the library

Function	SNMPv1	SNMPv2	SNMPv3 (or SNMPv1)
Get (request a SNMP tag)	**LSNMP_Get** with "getNext" = FALSE, for 1 tag, V6.0.1*		**LSNMP_GetOrSet** with "mode" = 1, for up to 10 tags V2.0.0
GetNext (Request next SNMP tag after a specified start address)	**LSNMP_Get** with "getNext" = TRUE, for 1 tag, V6.0.1*		**LSNMP_GetOrSet** with "mode" = 2, for up to 10 tags, V2.0.0
GetBulk (Request a range of SNMP tags after a specified start address)		**LSNMP_GetBulk** for any number of tags from 1 OID, V6.0.1*	**LSNMP_GetOrSet** with "mode = 5", for up to 10 tags from 1 OID, V2.0.0
Set (Change SNMP tag)	**LSNMP_Set** for 1 tag, V6.0.0*		**LSNMP_GetOrSet** with "mode" = 3, for up to 10 tags, V2.0.0
ReceiveTrap (receive SNMP traps from SNMP agents)	**LSNMP_ReceiveTrap** V1.1.0, Only valid from TIA Portal V18 and S7-1500 Firmware V2.6		
SendTrap (send SNMP traps to a SNMP manager)	**LSNMP_SendTrap** V5.1.0		

* The function blocks "LSNMP_Get", "LSNMP_GetBulk" and "LSNMP_Set" were modified during the expansion of the library with the SNMPv3 blocks to maintain the consistency of the library. There have been no functional changes with the resulting versions 6.0.x.

7.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0 (SNMP_ReceiveTrap from Firmware V2.6)
- SIMATIC S7-1200 Controllers as of firmware V4.2 (SNMP_ReceiveTrap from Firmware V4.5)
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.0 (SNMP_ReceiveTrap from Firmware V2.6)
- SIMATIC S7-1500 Software Controllers as of Firmware V2.0 (SNMP_ReceiveTrap from Firmware V2.6)
- CP 1543-1, CP 1542SP-1, CP 1542SP-1 IRC, CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V2.0

© Siemens 2026

7.2. LSNMP_Get

Description

SNMP provides different data packets to request tags from an SNMP agent. Two of them are implemented in this block:

- GetRequest to request a specific tag
- GetNextRequest to request the next tag in the MIB tree

For this purpose, the block sends either an SNMP GetRequest or SNMP GetNextRequest command for a tag by means of an Object Identifier (OID) to an SNMP agent. The SNMP agent responds with an SNMP GetResponse telegram containing the requested data or an error message.

Parameter

Figure 7-3: LSNMP_Get

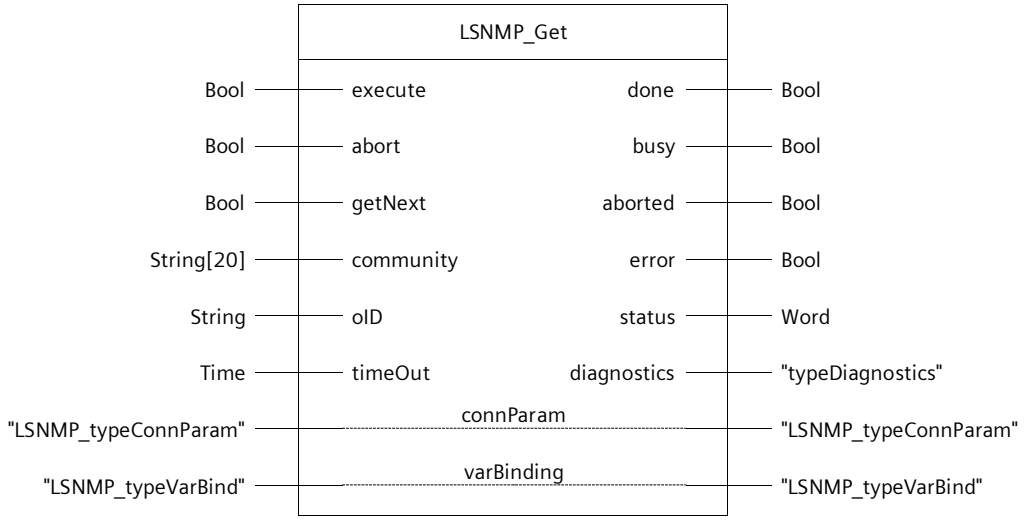


Table 7-2: Parameters of LSNMP_Get

Name	Declaration	Data type	Comment
execute	Input	Bool	The job is started with a positive edge. "abort" must be FALSE.
abort	Input	Bool	With a positive edge, the current job is cancelled. The command is only accepted if the block is currently busy.
getNext	Input	Bool	FALSE: Requests the tag for the specified OID TRUE: Requests the next tag in the MIB tree for the specified OID
community	Input	String[20]	Community
oID	Input	String	Object Identifier of the requested tag according to dot notation, e.g. '1.3.6.1.2.1.1.5.0'
timeOut	Input	Time	Time after which a job should be automatically cancelled
done	Output	Bool	TRUE: Job successfully executed. The received data is available at the "varBinding" parameter.
busy	Output	Bool	TRUE: Job is being executed
aborted	Output	Bool	TRUE: Job was cancelled

© Siemens 2026

Name	Declaration	Data type	Comment
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 0)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
connParam	InOut	"LSNMP_typeConnParam"	Connection parameters
varBinding	InOut	"LSNMP_typeVarBind"	Received variable binding

7.3. LSNMP_GetBulk

Functional description

The block "LSNMP_GetBulk" is used to read out large amounts of data with only one response telegram.

The block "LSnmp_GetBulk" uses the SNMP GetBulk request command and requires SNMPv2 for this – an extension of SNMPv1.

The GetBulk command performs several GetNext queries in the SNMP agent. The maximum number of GetNext commands can be specified via a parameterizable repetition factor in the GetBulk telegram.

The return values of all queried objects are compiled in a single response telegram and transferred. The block "LSNMP_GetBulk" then outputs the received data stream divided into the individual tags.

Parameter

Figure 7-4: LSNMP_GetBulk

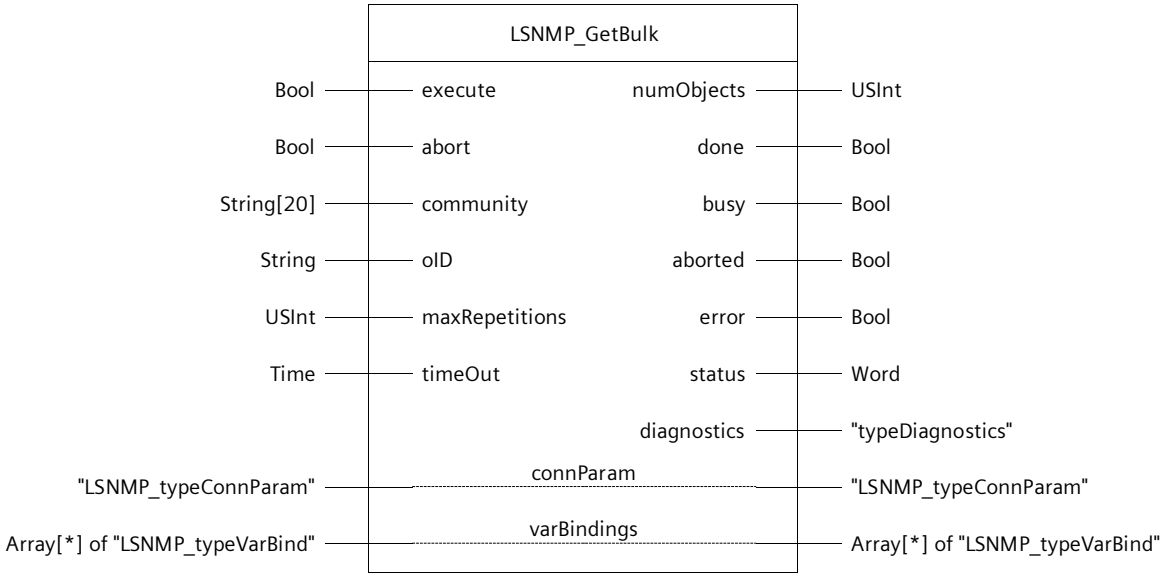


Table 7-3: Parameters of LSNMP_GetBulk

Name	Declaration	Data type	Comment
execute	Input	Bool	The job is started with a positive edge. "abort" must be FALSE.
abort	Input	Bool	With a positive edge, the current job is cancelled. The command is only accepted if the block is currently busy.

© Siemens 2026

Name	Declaration	Data type	Comment
community	Input	String[20]	Community
oID	Input	String	Object Identifier of the first requested tag according to dot notation, e.g. '1.3.6.1.2.1.1.5.0'
maxRepetitions	Input	USInt	Max. number of objects to be returned
timeOut	Input	Time	Time after which a job should be automatically cancelled
numObjects	Output	USInt	Number of objects received
done	Output	Bool	TRUE: Job successfully executed. The received data is available at the "varBindings" parameter.
busy	Output	Bool	TRUE: Job is being executed
aborted	Output	Bool	TRUE: Job was cancelled
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 0)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
connParam	InOut	"LSNMP_typeConnParam"	Connection parameters
varBindings	InOut	Array[*] of "LSNMP_typeVarBind"	Received variable bindings

© Siemens 2026 |

7.4. LSNMP_Set

Description

The "LSNMP_Set" block sends a write job to an SNMP agent via the SNMP SetRequest command for an SNMP tag.

In the SetResponse telegram the block receives the result of the write request from the SNMP agent. If the tag read in does not match the written value, an error will be read out. The write job must be set again.

Parameter

Figure 7-5: LSNMP_Set

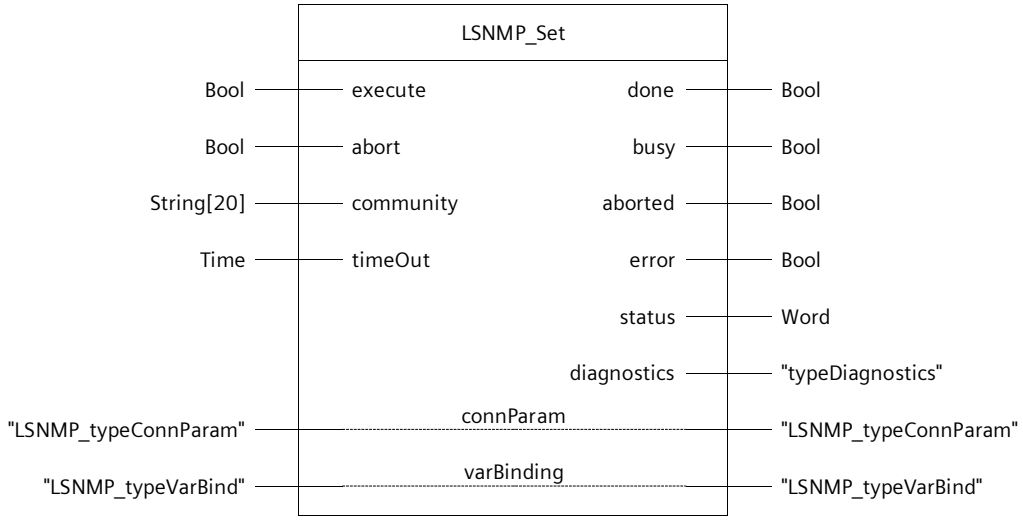


Table 7-4: Parameters of LSNMP_Set

Name	Declaration	Data type	Comment
execute	Input	Bool	The job is started with a positive edge. "abort" must be FALSE.
abort	Input	Bool	With a positive edge, the current job is cancelled. The command is only accepted if the block is currently busy.
community	Input	String[20]	Community
timeOut	Input	Time	Time after which a job should be automatically cancelled
done	Output	Bool	TRUE: Job successfully executed
busy	Output	Bool	TRUE: Job is being executed
aborted	Output	Bool	TRUE: Job was cancelled
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 0)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
connParam	InOut	"LSNMP_typeConnParam"	Connection parameters
varBinding	InOut	"LSNMP_typeVarBind"	Tag binding to be sent

© Siemens 2026 |

7.5. LSNMP_GetOrSet

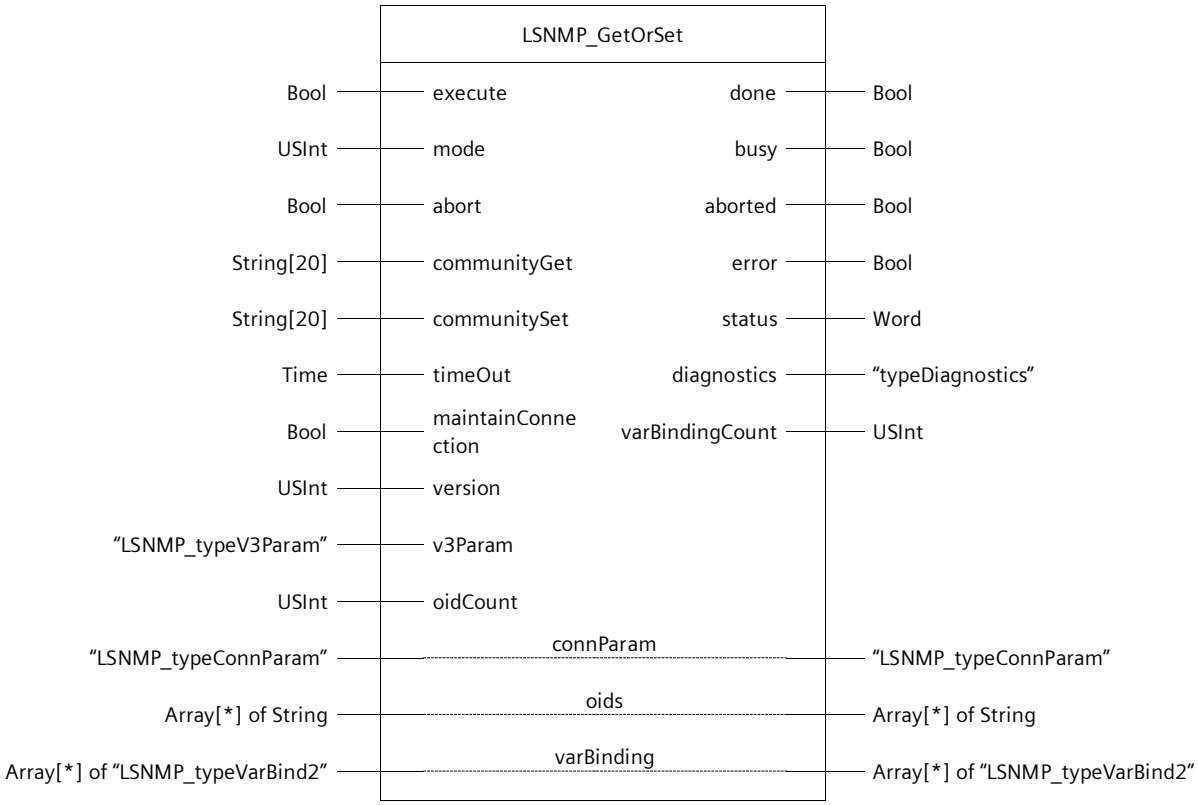
The "LSNMP_GetOrSet" module supports secure communication via SNMPv3, i.e. including authentication and encryption.

In contrast to the individual function blocks for SNMPv1 ("LSNMP_Get", "LSNMP_GetBulk" and "LSNMP_Set"), the function block covers different functions. The desired action is determined by the input parameter "mode":

- **Get** (mode 1) : Reading the values of up to 10 tags of an agent that are explicitly known. The addresses to be read ("OIDs") are specified in the InOut array "oids". The number of OIDs must be specified at the "oidCount" input.
- **GetNext** (mode 2) : Reading the values of up to ten tags of an agent that are not exactly known to the manager in advance. The InOut array « oids » specifies OIDs for which the agent should return the value of the next valid OID of its data model each. The number of requested values must be specified at the "oidCount" input.
- **GetBulk** (mode 5) : Reading the values of a range of up to 10 tags of an agent that are not known to the manager. In the InOut array « oids », only one start address is specified, from which the agent should return a range of consecutive OIDs of its data model. The number of OIDs to be delivered must be specified at the "oidCount" input.
- **Set** (mode 3) : Set one or more tags of an agent. In the InOut array « varBinding », the OIDs and values of the tags are explicitly specified. The number of OIDs to be set must be specified at the "oidCount" input.

On the one hand, the execution of the different functions with a single function block reduces the effort required for the care and maintenance of the library. On the other hand, a common and permanent context can be used for the various accesses within a single FB instance, which includes, for example, a common connection resource or localized security keys. Furthermore, the total demand for function blocks and memory will also remain lower.

Figure 7-6: LSNMP_GetorSet



© Siemens 2026

SNMPv3 security features

SNMPv3 of the "LSNMP_GetOrSet" function block is based on the "User-based Security Model", or "USM" for short. The communication participants must use consistent credentials consisting of username and password to establish secure communication:

Table 7-5: SNMPv3 security features

Feature	noAuthNoPriv	authNoPriv	authPriv	Description
User-based access	✓	✓	✓	The user used by the manager must be stored in the agent's user management and support the selected level. Thus, the user-specific passwords are also fixed.
Security level	1	2	3	This parameter determines which level of secure communication is aimed for and which of the following features both sides must use.
Timeliness	✓	✓	✓	In their messages, the communication partners refer to the current runtime and the current number of startup operations of the leading ("authoritative") partner, i.e. usually the SNMP agent. Only if this information is up-to-date and correctly stated can the freshness of a message be assumed (replay protection).
Authentication		✓	✓	The sender of a message uses a cryptographic hash algorithm (MD5, SHA-1, or SHA-256) to reduce the content to a digest, which is also transmitted as part of the message. Only if the recipient calculates the same digest of the message as the sender can he assume the authenticity of the message (integrity). However, the message content remains openly readable. For the calculation, the partners use a localized key that results from the confidential authentication password.
Privacy			✓	The payload of a message is additionally encrypted by the sender using a cipher algorithm (DES CBC or AES-125/256-CFB/CBC). A recipient can only make the data readable again if they use the same localized key for decryption that results from the confidential privacy password.

A complete communication handshake on the manager's side consists chronologically of the following steps (at security level "authPriv"):

- Encryption of the request (with an empty Authentication field)
- Authentication of the (encrypted) request
- Sending the request
- Receiving the response
- Authentication of the (un-decrypted) response
- Decryption the response

Localized keys are derived from the passwords for authentication and privacy, each of which belongs to a specific communication partner. This is to prevent all participants with the identical user information from being affected in the event of a specific key being compromised.

The localized keys are generated once at the beginning of each new communication relationship. This process can take a few seconds or even minutes, depending on how the function block „LSNMP_GetOrSet" is called and parameterized. The stores the secrets of up to 5 different communication relationships (SNMPV3_USMSTORE_MAX).

It should be noted that SNMP communication is based on UDP and is therefore basically connectionless. The security mechanisms are therefore purely message-based and not used to negotiate a permanent session, as is common with TCP-based protocols.

LSNMP_typeV3Param

This input type contains the relevant parameters for SNMPv3 configuration:

Table 7-6: LSNMP_typeV3Param

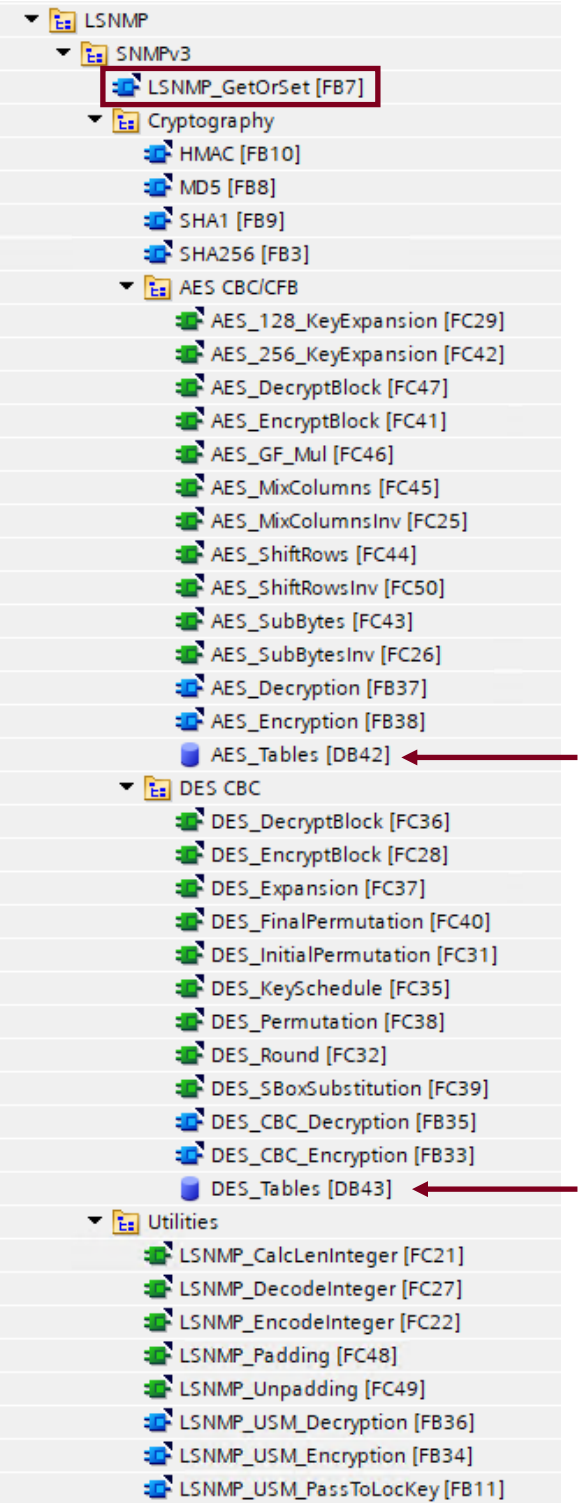
Name	Data type	Comment
userName	String[32]	user on whose behalf the message is being exchanged
securityLevel	USInt	1 = noAuthNoPriv (no Authentication, no Privacy) 2 = authNoPriv (Authentication, no Privacy) 3 = authPriv (Authentication and Privacy)
authProtocol.digestAlgorithm	USInt	Digest algorithm used for Authentication (if securityLevel ≥ 2): 10 = MD5 20 = SHA1 30 = SHA256
authPassword	String[16]	password for Authentication
authChunksPerCycle	UInt	byte chunks to be hashed per cycle: 0 = as many as possible 1 = least >1 = more, default: 5 (The total number of required hash block calculations depends on the message length. If the setting is set too low, then the key localization during the initialization phase might take up to several minutes. A higher setting might cause cycle time peaks.)
privProtocol.cipherAlgorithm	USInt	Encryption/Decryption algorithm used for Privacy (if securityLevel ≥ 3): 10 = CBC-DES 20 = AES-256-CBC 21 = AES-128-CBC 30 = AES-256-CFB 31 = AES-128-CFB
privPassword	String[16]	password for Privacy (Encryption/Decryption)
privBlocksPerCycle	UInt	blocks to be en-/decrypted per cycle: 0 = all 1 = least (default/recommended) >1 = more (The total number of required cipher block calculations depends on the message length. Calculating more (>1) or even all the blocks (0) in one cycle might lower the total duration but will cause cycle time peaks.)
skipRespAuth	Bool	skip response authentication for faster performance (= reduced security level!)

© Siemens 2026

Library types

The function block "LSNMP_GetOrSet" uses several other library types (FCs, FBs and data types):

Figure 7-7: Library types for "LSNMP_GetOrSet"



NOTE Additionally, there are two global DBs that are provided as master copies and must be copied to the project:

AES_Tables [DB]

DES_Tables [DB]

© Siemens 2026

7.6. LSNMP_ReceiveTrap

Description

SNMP agents (e.g. an S7 CPU) send messages to the network management system via SNMP traps to display changes in the network status.

The "LSNMP_ReceiveTrap" block is a parameterizable function block for receiving SNMP traps as an SNMP manager. The reception of a trap is indicated by the status 16#0000.

NOTE The function block is only able to resolve one incoming trap per cycle.
The SNMP agents will not receive acknowledgement for received traps.

Parameter

Figure 7-8: LSNMP_ReceiveTrap

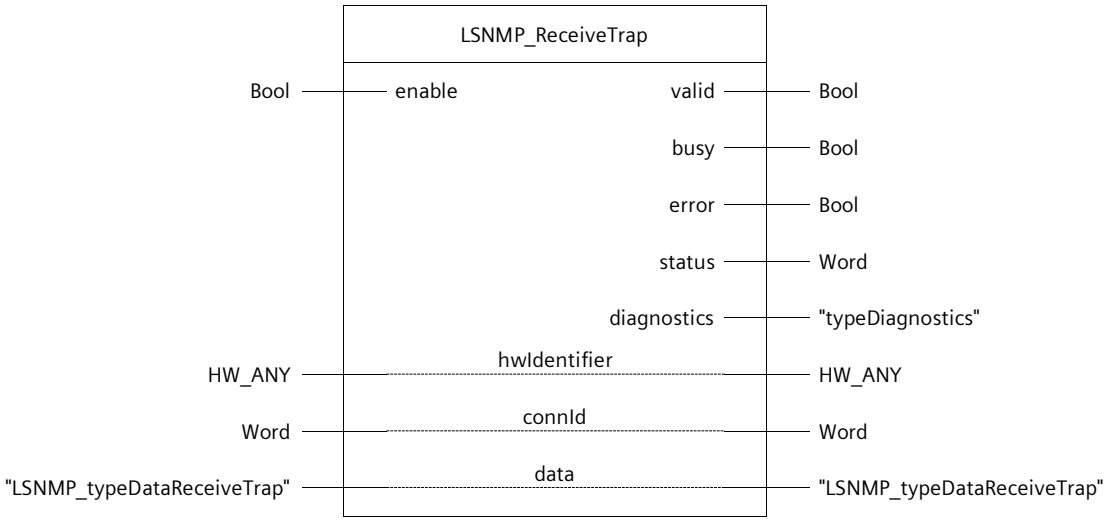


Table 7-7: Parameters of LSNMP_ReceiveTrap

Name	Declaration	Data type	Comment
enable	IN	Bool	TRUE: Enable the function of the FB
valid	OUT	Bool	TRUE: Valid set of output values available at the FB
busy	OUT	Bool	TRUE: FB is not yet complete and further output values may be expected
error	OUT	Bool	TRUE: An error has occurred whilst executing the FB
status	OUT	Word	Status and error codes (see chapter 0)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
hwIdentifier	InOut	HW_ANY	Hardware-ID of the interface
connId	InOut	Word	Connection-ID
data	InOut	"LSNMP_typeDataReceiveTrap", see LSNMP_typeDataSendTrap	Data to be sent with the SNMP trap

© Siemens 2026

7.7. LSNMP_SendTrap

Description

SNMP agents (e.g. an S7 CPU) send messages to the network management system via SNMP traps to display changes in the network status.

The "LSNMP_SendTrap" block is a parameterizable function block for sending SNMP traps.

NOTE The SNMP agents do not receive acknowledgement for sent traps.

Parameter

Figure 7-9: LSNMP_ReceiveTrap

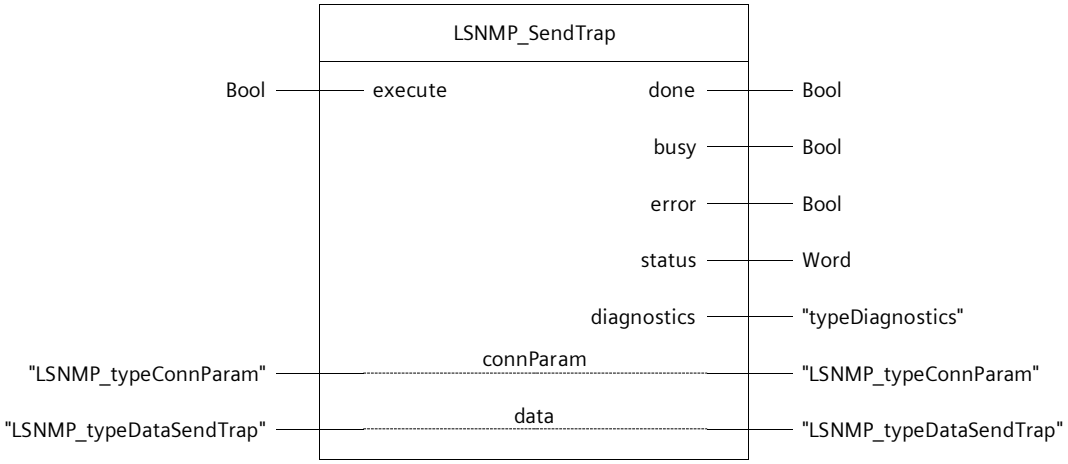


Table 7-8: Parameters of LSNMP_SendTrap

Name	Declaration	Data type	Comment
execute	Input	Bool	The job is started with a positive edge.
done	Output	Bool	TRUE: Job successfully executed
busy	Output	Bool	TRUE: Job is being executed
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 0)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
connParam	InOut	"LSNMP_typeConnParam"	Connection parameters
data	InOut	"LSNMP_typeDataSendTrap"	Data to be sent with the SNMP trap

© Siemens 2026

7.8. PLC Data Types

LSNMP_typeConnParam

The PLC data type "LSNMP_typeConnParam" contains parameters that are required to establish a connection to the partner.

Table 7-9: Parameters of LSNMP_typeConnParam

Name	Data type	Comment
hwIdentifier	HW_ANY	Hardware identification of the Ethernet interface
connID	Word	Connection ID
ipAddress	IP_V4	IP address of the partner
localPort	UInt	Local port

NOTE If you want to run multiple blocks of the library or instances of the same block at the same time, the parameters "connID" and "localPort" must be unique for each instance.

LSNMP_typeDataSendTrap

The PLC data type "LSNMP_typeDataSendTrap" contains all parameterizable data that can be sent to an SNMP manager with "LSNMP_SendTrap".

Table 7-10: Parameters of LSNMP_typeDataSendTrap

Name	Data type	Comment
community	String[20]	Community
oID	String	Object Identifier of the agent generating the trap, in dot notation, e.g. '1.3.6.1.2.1.1.5.0'
ipAgent	IP_V4	IP address of the agent generating the trap
genericTrap	Int	General Trap ID • 0: Cold start (coldStart) • 1: Warm start (warmStart) • 2: Link Down (linkDown) • 3: Link Up (linkUp) • 4: Authentication failure (authenticationFailure) • 5: EGP neighbour lost (egpNeighborLoss) • 6: company-specific (enterpriseSpecific)
specificTrap	Int	enterprise-specific trap ID If the general trap ID "6" is selected, an enterprise-specific trap ID can be sent with this parameter. Otherwise, the enterprise-specific trap ID is "0".
varBinding	"LSNMP_typeVarBind"	Optional tag binding to be sent with the trap. In this case, the trap ID is considered sufficient information.

© Siemens 2026

LSNMP_typeVarBind

With SNMP, tags are transmitted as "tag bindings". A tag binding consists of the Object Identifier of the tag and the actual value. The PLC data type "LSNMP_typeVarBind" defines such a tag binding. So that the block or the user can interpret the value correctly, the data type and the length are also specified.

Table 7-11: Parameters of LSNMP_typeVarBind

Name	Data type	Comment
oID	String	Object Identifier of the tag in dot notation, e.g. '1.3.6.1.2.1.1.5.0'
type	Byte	Data type of the tag Common data types: 16#02: Integer 16#04: String 16#41: Counter 16#43: Timeticks
length	Int	Tag length
value	Array[0..255] of byte	Value of the tag

7.9. Integration into the user program

SiePortal provides application examples for integrating the library into the user program:

https://support.industry.siemens.com/cs/document/57249109

https://support.industry.siemens.com/cs/document/109987064

© Siemens 2026 |

7.10. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The following are the status and error codes specific to the "LSNMP" library. Some of them only apply to the SNMPv1 blocks, others only to the SNMPv3 blocks.

Table 7-12: Status and error codes

Code	Description
16#0000	Job completed successfully
16#0FFF	Job was cancelled
16#7000	No job active
16#7001	First call of the command
16#7002	Follow-up call of the command
16#7003	Cancels the job
16#8107	The received value is too large for the memory area at the "varBinding" or "varBindings" parameter.
16#8109	The received packet is too large for the internal reception area.
16#8201	The value at the parameter "maxRepetitions" is larger than the array at the parameter "varBindings".
16#8202	Incorrect input parameter “mode” specified
16#8203	Incorrect input parameter “version” specified
16#8401	Malformed packet received. Possible reasons: • Invalid header • Packet contains invalid lengths
16#8402	Incorrect “msgVersion” used by the communication partner.
16#8403	Incorrect “msgID” used by the communication partner.
16#8404	Incorrect “msgMaxSize” used by the communication partner.
16#8405	Incorrect “msgFlags” used by the communication partner.
16#8406	Incorrect “msgSecurityModel” used by the communication partner.
16#8407	Incorrect “msgAuthoritativeEngineID” used by the communication partner.
16#8408	Time-out: The job could not be completed within the specified time. Possible reasons: • Partner is not available • Community is wrong
16#8409	Incorrect “snmpAuthoritativeEngineBoots” used by the communication partner.
16#8410	Incorrect “snmpAuthoritativeEngineTime” used by the communication partner.
16#8411	Incorrect “snmpAuthoritativeEngineTime” used by the communication partner, possible replay.
16#8412	Incorrect “msgUserName” used by the communication partner.
16#8413	Incorrect “msgAuthenticationParameters” used by the communication partner.

Code	Description
16#8414	Incorrect Authentication by the communication partner.
16#8415	Incorrect “msgPrivacyParameters” used by the communication partner.
16#85xx	SNMP error received from partner. The SNMP error code is output in the lower byte and "diagnostics.subfunctionStatus". Descriptions of the SNMP error codes can be found online.
16#8600	The FB is in an invalid state.
16#8601	Error in subordinate command "TCON" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8602	Error in subordinate command "TUSEND" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8603	Error in subordinate command "TURCV" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8604	Error during password to key localization (in subordinate instruction “LSNMP_USM_PassToLocKey”)
16#8605	Error during request Authentication (in subordinate instruction “HMAC”)
16#8606	Error during response Authentication (in subordinate instruction “HMAC”)
16#8607	Error during Encryption (in subordinate instruction “LSNMP_USM_Encryption”)
16#8608	Error during Decryption (in subordinate instruction “LSNMP_USM_Decryption”)

© Siemens 2026

8. LSNTP

8.1. Overview

8.1.1. Range of Functions

For automation cells or subsystems, it is often secondary to use the exact "atomic time". It is usually sufficient to have a common time base for all automation components.

The use of a SIMATIC S7 CPU as an SNTP server enables flexible and simple synchronization of systems and subsystems, for example, to obtain meaningful timestamps for error messages and logging data system wide.

The library provides a function block that performs the following functions:

- Receiving and evaluating an NTP telegram from an (S)NTP Client
- Creating and sending an NTP telegram to the client for time synchronization

The inaccuracy of the SNTP server is less than 10 ms.

Example scenario

The following figure shows a possible example configuration with a SIMATIC S7-1500 CPU as the SNTP server. Here, the CPU synchronizes its time with an external NTP server. However, configurations with other clocks are also possible.

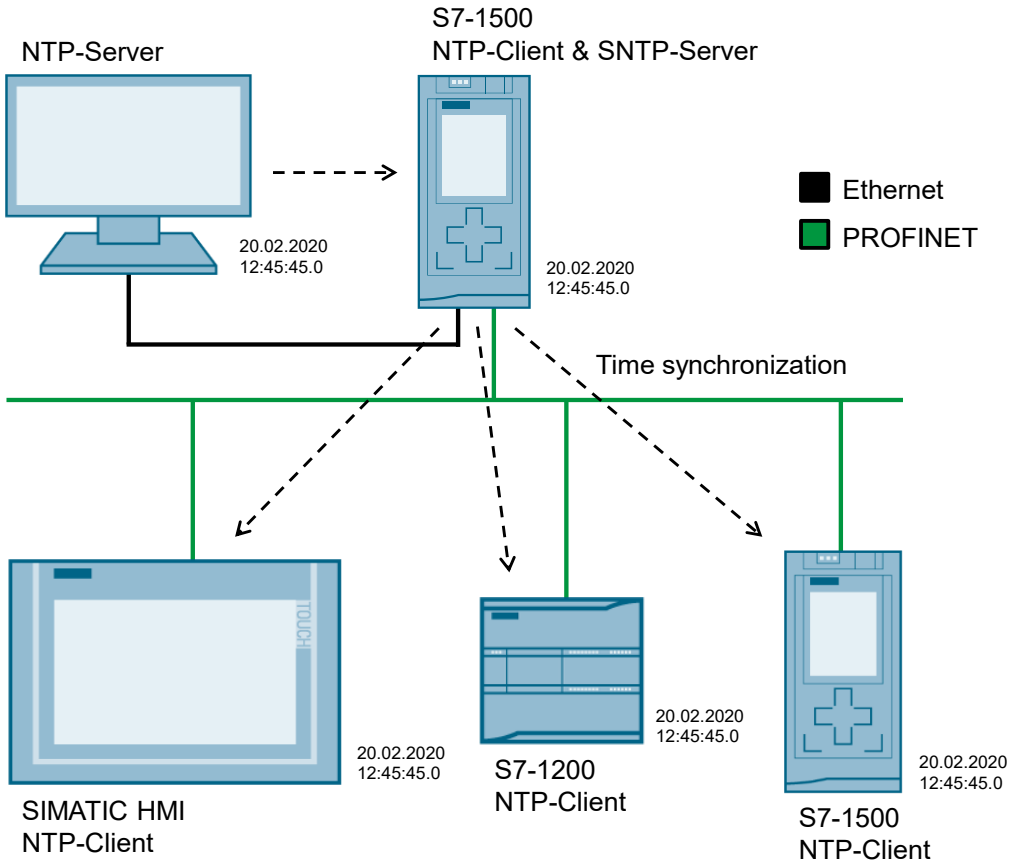


Figure 8-1: Example scenario

© Siemens 2026 |

8.1.2. Components of the Library

The "LSNTP" library consists of the following blocks and data types.

Function blocks

Table 8-1: Function blocks

Name	Type	Version	Description
LSNTP_Server	FB	V4.0.0	Realizes the function of the SNTP server.

PLC Data Types

Table 8-2: PLC data types

Name	Type	Version	Description
LSNTP_typeTelegram	Data type	V1.0.0	Describes the structure of the NTP telegram.
LSNTP_typeTimestamp	Data type	V1.0.0	Describes the structure of the timestamp.

8.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.2
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.0
- SIMATIC S7-1500 Software Controllers as of Firmware V2.0
- CP 1543-1, CP 1542SP-1, CP 1542SP-1 IRC, CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V2.0

© Siemens 2026

8.2. LSNTP_Server

8.2.1. Interface Description

Description

The function block implements the function of the SNTP server.

Parameter

Figure 8-2: LSNTP_Server

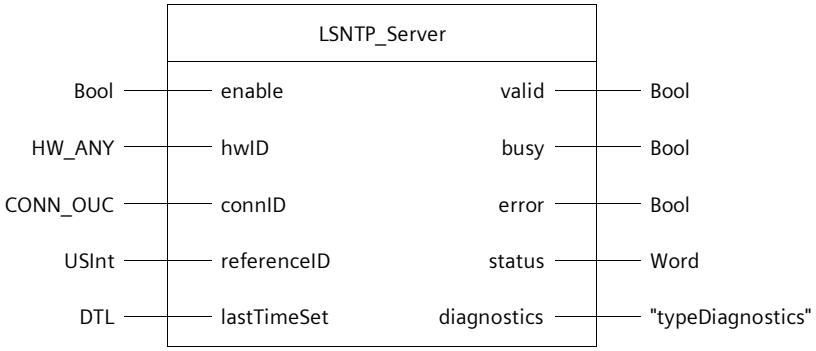


Table 8-3: Parameter of LSNTP_Server

Name	Declaration	Data type	Comment
enable	Input	Bool	TRUE: Activates the SNTP server
hwID	Input	HW_ANY	Hardware identifier of the PN/IE interface
connID	Input	CONN_OUC	Unique connection ID
referenceID	Input	USInt	The input specifies from which time source the server CPU obtains the time: • 0: uncalibrated (set "by hand") • 1: primary reference (e.g., DCF 77) • 2: secondary reference (e.g., from GPS receiver) The information is passed on to the NTP client in the SNTP protocol.
lastTimeSet	Input	DTL	Time specification when the time of the controller was last set. This information is passed on to the NTP client in the SNTP protocol. If the parameter is not assigned, the current time of the CPU is transferred instead.
valid	Output	Bool	TRUE: The block is working without error
busy	Output	Bool	TRUE: The block is processed
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 8.2.2)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)

© Siemens 2026

8.2.2. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The status and error codes specific to the "LSNTP" library are listed below.

Table 8-4: Status and error codes

Code	Description
16#0000	NTP telegram sent to an (S)NTP client
16#0001	NTP telegram of an (S)NTP client received
16#7000	The block is not executed
16#7001	First call of the command
16#7002	Follow-up call of the command
16#7003	Block is deactivated
16#7F01	Invalid package received
16#8600	The FB is in an invalid state.
16#8601	Error in subordinate command "TCON" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8602	Error in subordinate command "TUSEND" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8603	Error in subordinate command "TURCV" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

© Siemens 2026

9. LSQL

9.1. Overview

9.1.1. Range of functions

The LSQL library enables the interaction with an SQL database directly from the user program of a controller by replicating the TDS protocol based on "Open User Communication". It enables the following actions:

- Establishing a connection and login to a Microsoft SQL Server database
- Transferring SQL statement
- Analyzing received data
- Dissecting and passing received data to the use-case specific data structures → The solution to this task is use-case specific and not part of this library. A master copy is provided as a template to be adapted. For more information, see the related application example of chapter 9.5.

9.1.2. Components of the library

The LSQL library consists of the following objects. Both function blocks use several FCs and FBs internally which are not described here.

Function Blocks

Table 9-1 Function blocks

Name	Type	Version	Description
LSQL_Microsoft	FB	5.0.3	Organizes the sending and composing of telegrams.
LSQL_AnalyzeTokens	FB	1.5.3	Analyzes the token stream of a SQL Server response This function block became part of the library with FB LSQL_Microsoft V5.x.

PLC Data Types

Table 9-2: PLC data types

Name	Version	Description
LSQL_typeColumn	3.2.0	Properties of a SQL table column, e.g. column name.
LSQL_typeColumnFlags	3.0.3	Additional property flags of a SQL table column, e.g. "caseSensitive".
LSQL_typeConnParam	2.0.4	Connection parameters and credentials required for connection establishment between client and SQL Server.
LSQL_typeErrortoken	1.1.0	Structure for Error token from TDS command response.
LSQL_typeLoginInformation	1.2.0	Credentials and parameter for login to SQL Server.
LSQL_typeTokenCount	1.2.0	Structure for indicating the number of tokens of different token types in a token stream from a SQL Server response.

9.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 Controller as of Firmware V2.9
- SIMATIC S7-1200 Controller as of Firmware V4.5
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0

9.2. LSQL_Microsoft

The function block establishes the connection to the SQL server and performs the data exchange by sending commands to the server.

9.2.1. Interface description

Figure 9-1: LSQL_Microsoft

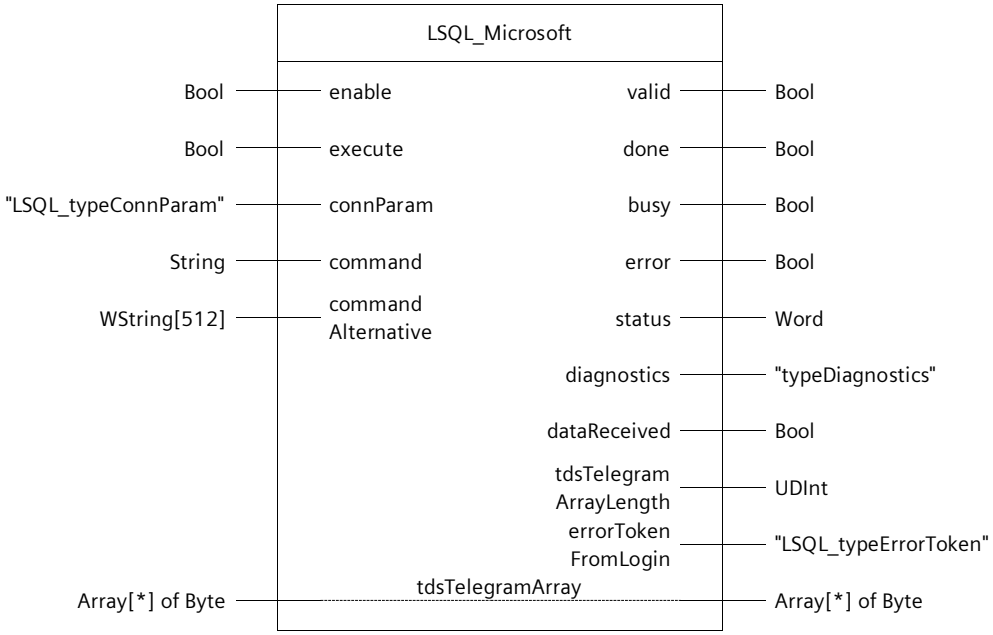


Table 9-3: Parameters of LSQL_Microsoft

Name	Declaration	Data Type	Comment
enable	Input	Bool	TRUE: Enable functionality of FB
execute	Input	Bool	TRUE: execute SQL command once
connParam	Input	LSQ_typeConnParam	Connection parameters, see Table 9-9
command	Input	String	SQL command to execute when executeSqlBatch = TRUE
commandAlternative	Input	WString[512]	SQL command as longer WSTRING --> has priority over 'command' input. Leave empty if 'command' is still to be used. WStrings with shorter length can be used at this input as well.
valid	Output	Bool	TRUE: Valid set of output values available at the FB
done	Output	Bool	TRUE: Command was executed successfully
busy	Output	Bool	TRUE: FB is not finished and new output values can be expected
error	Output	Bool	TRUE: An error occurred during the execution of the FB
status	Output	Word	Status and error codes (see chapter 9.2.2)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
dataReceived	Output	Bool	TRUE: new data have been received

© Siemens 2026

Name	Declaration	Data Type	Comment
tdsTelegramArrayLength	Output	UDInt	number of Bytes received in buffer area for reassembled TDS protocol
errorTokenFromLogin	Output	LSQL_typeErrorToken	error token that was received due to a faulty login, see Table 9-10
tdsTelegramArray	InOut	Array[*] of Byte	array for reconstructed TDS protocol data

9.2.2. Error handling

For general information on the status outputs and diagnostics of the function blocks of these libraries, see Chapter 14.2.

The following are the status and error codes specific to the LSQL library.

Table 9-4: Status and error codes

Code	Description
16#0000	Job completed successfully
16#7000	No job is currently being processed.
16#7001	First call after incoming new job (rising edge at parameter "enable").
16#7002	Subsequent call during active processing without additional information.
16#7003	Establishing the TCP/IP Connection
16#7004	TCP/IP connection successfully established
16#7005	Sending a SQL Command
16#7006	Receiving SQL response
16#7011	"TDISCON" successfully executed
16#8001	Incorrect operation of the FB
16#8200	Errors during parameterization
16#8400	External error
16#8600	Internal Error
16#8601	Error in subordinate command "TCON" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8602	Error in subordinate command "TSEND" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8603	Error in subordinate command "TRCV" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8604	Error in subordinate command "TDISCON" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

Code	Description
16#8613	Error in converting the data for pre-login
16#8614	Error converting the login data
16#8615	Error converting the data for the SQL command
16#8616	Received data array overflow
16#8617	Login error, see "errorTokenFromLogin"
16#8800	Error due to reserved area
16#8900	Error due to an undefined state in the state machine.

© Siemens 2026 |

9.3. LSQL_AnalyzeTokens

Splits SQL response data array into TDS header, tokens (especially ColumnMetaData and Error token) and row payload data.

9.3.1. Interface description

Figure 9-2: LSQL_AnalyzeTokens

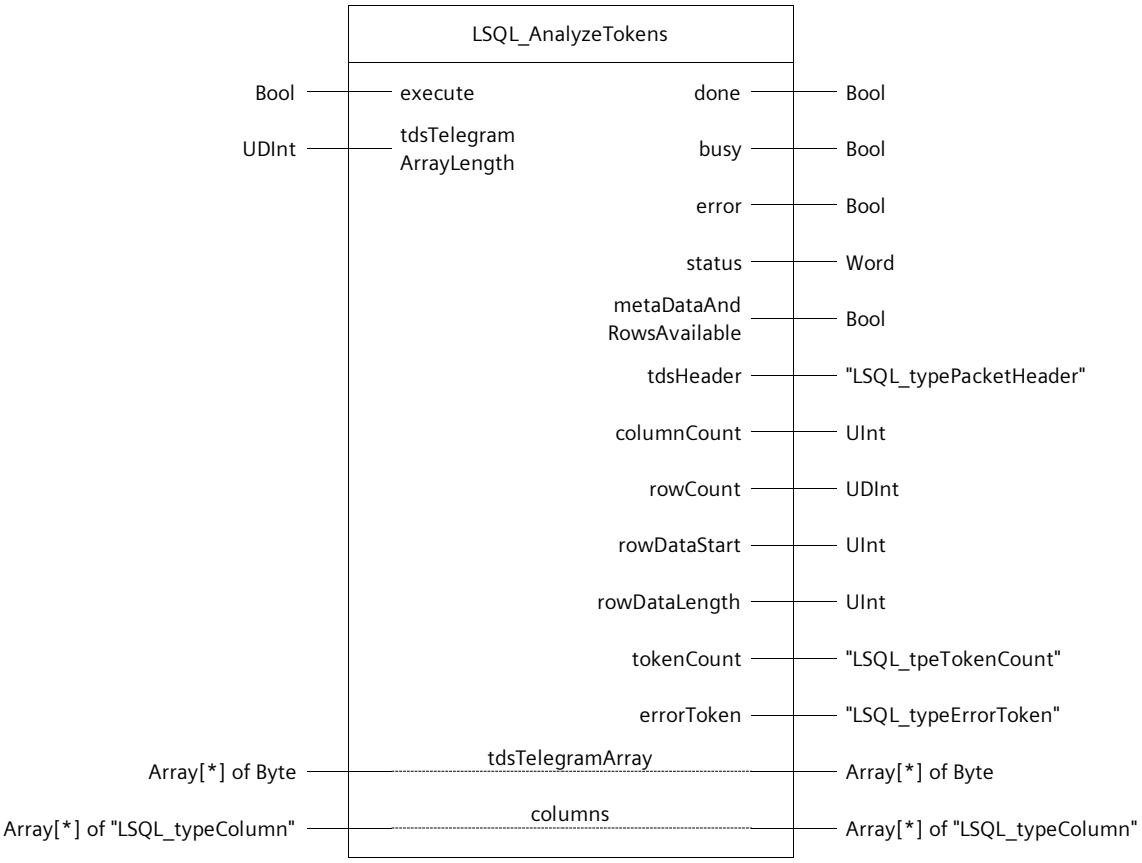


Table 9-5 Parameters of LSQL_AnalyzeTokens

Name	Declaration	Data Type	Comment
execute	Input	Bool	TRUE: execute SQL command once
tdsTelegramArrayLength	Input	UDInt	Number of bytes received by TDS
done	Output	Bool	TRUE: Command was executed successfully
busy	Output	Bool	TRUE: FB is not finished and new output values can be expected
error	Output	Bool	TRUE: An error occurred during the execution of the FB
status	Output	Word	Status and error codes (see chapter 9.2.2)
metaDataAndRowsAvailable	OUT	Bool	Rows and metadata are available.
tdsHeader	OUT	"LSQL_typePacketHeader"	TDS header
columnCount	OUT	UInt	Number of columns received.

© Siemens 2026

Name	Declaration	Data Type	Comment
rowCount	OUT	UDInt	Number of (at least) affected rows.
rowDataStart	OUT	UInt	Start byte address of row data area.
rowDataLength	OUT	UInt	Length of the token stream "Row data".
tokenCount	OUT	"LSQL_typeTokenCount"	Structure for indicating the number of tokens received
errorToken	OUT	"LSQL_typeErrorToken"	Structure for the ERROR token of the TDS response telegram
tdsTelegramArray	IN_OUT	Array[*] of Byte	Receive buffer in which the received bytes of the TDS response telegram are stored.
columns	IN_OUT	Array[*] of "LSQL_typeColumn"	Properties of the columns.

9.3.2. Error handling

Table 9-6: Error codes

Error message/name of the constant in the FB	Value	Description
ERR_UNKNOWNTOKEN	16#8601	Wrong type for the token stream "ColumnMetaData".
ERR_HEADER_INCORRECTLENGTH	16#8602	Wrong length for the token stream "ColumnMetaData".
ERR_COLUMNS_SIZENOTDEFINED	16#8603	At least one column size is not defined.
ERR_COLUMNS_TYPENOTDEFINED	16#8604	At least one column type is undefined or unknown.
ERR_COLUMNS_OVERFLOW	16#8605	Column overflow.
ERR_TOKENDONE_UNKNOWNSTATUS	16#8606	The DONE token has an unknown status.
ERR_TOKENDONE_WRONGTOKENTYPE	16#8607	The DONE token has the wrong type.
ERR_HEADER_WRONGSTATUS	16#8608	The TDS header has an incorrect status.
ERR_TOKENERROR	16#8609	An ERROR token was found.
ERR_TOKENROW_NOMETADATA	16#8610	No "ColumnMetaData" were decoded.
ERR_LENGTHINCORRECT	16#8611	The length of the analyzed data does not match the total TDS length.
ERR_TYPESIZEINCORRECT	16#8612	A type size is incorrect or exceeding the internal array boundaries.
ERR_ARRAYLENGTHERROR	16#8613	Specified length is exceeding data array.

© Siemens 2026

9.4. PLC Data Types

LSQL_typeColumn

Information structure about SQL table column properties.

Table 9-7: Parameters of LSQL_typeColumn

Name	Data Type	Comment
userType	UDInt	Column user type
columnType	UInt	Column data type index
columnNameLength	Int	Length of the column header name string
columnName	String	Column header name
columnSizeFieldSize	USInt	Size (number of bytes) of the "columnSize" parameter
columnSize	USInt	Column data size
flags	"LSQL_typeColumnFlags"	Collection of flags for SQL column properties. See Table 9-8

LSQL_typeColumnFlags

Collection of flags for SQL table column properties.

Table 9-8: Parameters of LSQL_typeColumnFlags

Name	Data Type	Comment
nullable	Bool	TRUE: Saving of a null value is not allowed
caseSensitive	Bool	TRUE: Column search is case-sensitive
updateable	USInt	0: Column is read-only. 1: Column is readable and writable. 2: Updatability of the column is unknown.
identity	Bool	TRUE: Column is an identity column
computed	Bool	TRUE: Column is computed.

LSQL_typeConnParam

Connection parameters and credentials required for connection establishment between client and SQL Server.

Table 9-9: Parameters of LSQL_typeConnParam

Name	Data Type	Comment
hwId	HW_ANY	Hardware identifier of the interface (0 = interface is selected automatically)
connId	CONN_OUC	Unique connection identifier
server	String	Hostname or IP address of the server
port	UInt	Remote port on the server (Default: 1433)
tls	Struct	Parameters for TLS encryption

© Siemens 2026

Name	Data Type	Comment
loginInformation	"LSql_typeLoginInformation"	Login data for authentication between client and SQL server.

LSQL_typeErrorToken

Structure for Error Token from TDS command response.

Table 9-10: Parameters of LSQL_typeErrorToken

Name	Data Type	Comment
tokenLength	UInt	The total length of the error data in bytes.
sqlErrorNumber	UDInt	SQL error code
state	Byte	Status number of the error
class	USInt	The class (severity) of the error. A class smaller than 10 corresponds to an information message.
errorMessageLength	UInt	The length of the message text in characters.
errorMessage	String	Message text.
serverNameLength	USInt	The length of the server name in characters.
serverName	String	Server name.
procNameLength	USInt	The length of the stored procedure name in characters.
procName	String	The stored procedure name.
lineNumber	UDInt	The line number in the SQL batch or stored procedure where the error occurred. The line number starts with 1. If the line number is not relevant for the error message, the value is 0.

LSQL_typeLoginInformation

Credentials and parameter for login to SQL server

Table 9-11: Parameters of LSQL_typeLoginInformation

Name	Data Type	Comment
hostName	String	Optional: Name of the local host
userName	String	Required: Username for logging in to the database
password	String	Required: Password for logging in to the database
appName	String	Optional: Name of the application connecting with the database.
serverName	String	Required: Server name of the database
libraryName	String	Optional: Name of the user interface
language	String	Optional: Language of the user interface.
databaseName	String	Required: Database being read or written.
sspi	String	Optional/not supported: Encryption via Security Support Provider Interface (SSPI)

© Siemens 2026

Name	Data Type	Comment
attachDbfile	String	Optional: File name to be added during transmission.
changePassword	String	Optional: New password, should the old one be modified.

LSQL_typeTokenCount

Structure for indicating the number of tokens of different token types in a token stream from a SQL Server response

Table 9-12: Parameters of LSQL_typeTokenCount

Name	Data Type	Comment
columnMetaData	USInt	number of COLMETADATA tokens
row	UDInt	number of ROW tokens
order	USInt	number of ORDER tokens
error	USInt	number of ERROR tokens
done	USInt	number of DONE tokens
doneInProc	USInt	number of DONEINPROC tokens
doneProc	USInt	number of DONEPROC tokens
returnStatus	USInt	number of RETURNSTATUS tokens
info	USInt	number of INFO tokens
envChange	USInt	number of ENVCHANGE tokens
loginAck	USInt	number of LOGINACK tokens
unknown	USInt	number of other or unknown tokens
unknownTypes	String[40]	identifiers of unknown tokens (separated by semicolon)

9.5. Integration into the User Project

You will find a detailed application example for the integration of the library into your user project in the Siemens Industry Online Support: https://support.industry.siemens.com/cs/ww/en/view/109779336

The data types and function blocks required for the application example can be found in "master copies > LSQL_Example". See Chapter 13.4.

© Siemens 2026

10. LSyslog

10.1. Overview

10.1.1. Range of Functions

With the simply designed syslog protocol, applications can send messages, warnings, or error conditions to a syslog server. syslog is typically used for computer system management and security monitoring and has now established itself as a standard (RFC 5424) in logging.

This library emulates the syslog protocol and offers the possibility to send messages via UDP or TCP with optional TLS encryption to a syslog server, such as SINEC INS.

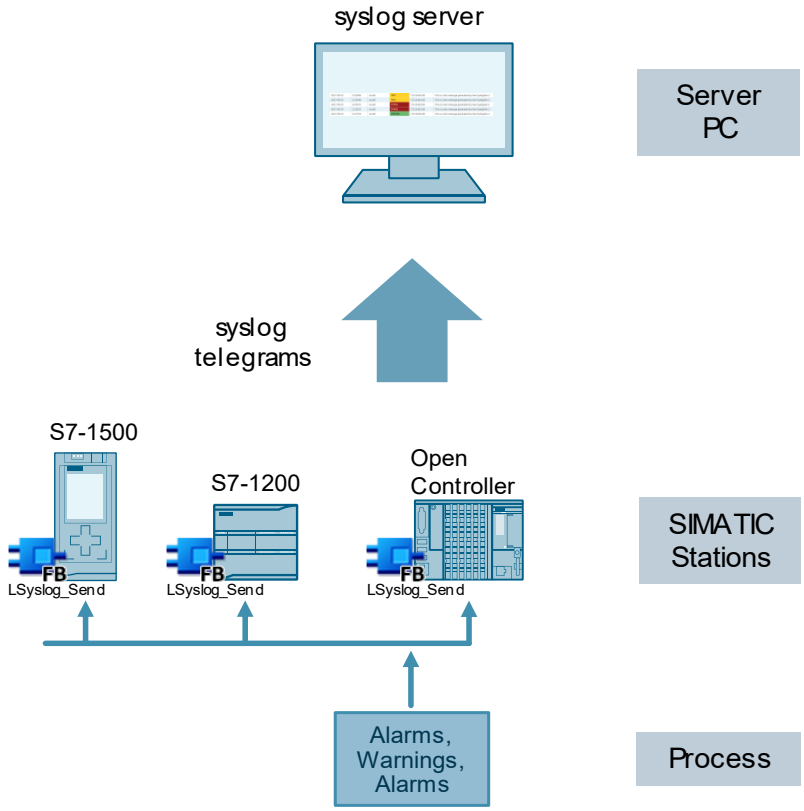


Figure 10-1: Overview

© Siemens 2026 |

10.1.2. Components of the Library

The "LSyslog" library contains the following objects.

Function blocks

Table 10-1: Function blocks

Name	Version	Description
LSyslog_Send	V2.0.0	Organizes the transmission and creation of the telegrams.

PLC Data Types

Table 10-2: PLC data types

Name	Version	Description
LSyslog_typeMessage	V1.1.0	This data type describes all tag data of a syslog message.

10.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.4
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- CP 1543-1, CP 1542SP-1, CP 1542SP-1 IRC, CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V2.0

© Siemens 2026 |

10.2. LSyslog_Send

10.2.1. Interface Description

Description

The FB "LSyslog_Send" establishes a connection to the syslog server via UDP or TCP with optional TLS encryption and sends syslog messages to the syslog server upon request. Afterwards, the connection is automatically broken again.

Parameter

Figure 10-2: LSyslog_Send

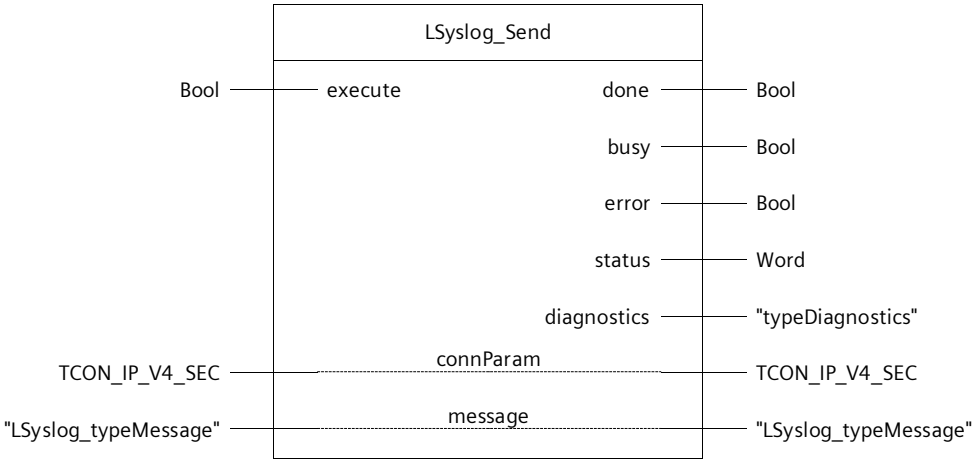


Table 10-3: Parameters of LSyslog_Send

Name	Declaration	Data type	Comment
execute	Input	Bool	With a positive edge, the connection to the syslog server is established and the syslog message is sent.
done	Output	Bool	TRUE: The send job was successfully executed
busy	Output	Bool	TRUE: The block is executed
error	Output	Bool	TRUE: An error has occurred
status	Output	Word	Status and error codes (see chapter 10.2.2)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
connParam	InOut	TCON_IP_V4_SEC	Connection parameters
message	InOut	"LSyslog_typeMessage"	syslog message to be sent

NOTE

In the FB the octet stuffing method for syslog is implemented over TCP by appending a line feed to the end of the message.

The octet counting method is also implemented, but commented out.

If you want to use the octet counting method for your application instead, you can adjust the comments in the build message region accordingly.

© Siemens 2026

NOTE You will find an application example for the usage of certificates in TIA Portal in the Siemens Industry Online Support:
https://support.industry.siemens.com/cs/ww/en/view/109769068

10.2.2. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The following are the status and error codes specific to the "LSyslog" library.

Table 10-4: Status and error codes

Code	Description
16#0000	Job completed successfully
16#7000	No job active
16#7001	First call of the command
16#7002	Follow-up call of the command
16#8408	Time-out: The job could not be completed within the specified time. Possible reasons for a time-out: • Partner is not available
16#8600	The FB is in an invalid state.
16#8601	Error in subordinate command "TCON" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8602	Error in subordinate command "TUSEND" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8603	Error in subordinate command "TSEND" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

© Siemens 2026

10.3. PLC Data Types

LSyslog_typeMessage

The PLC data type describes all parameters of the syslog message to be sent.

Table 10-5: Parameters of LSyslog_typeMessage

Name	Type	Comment
facility	Int	Facility (origin of the message) • 0: kernel messages • 1: user-level messages • 2: mail system • 3: system daemons • 4: security/authorization messages • 5: messages generated internally by syslog • 6: line printer subsystem • 7: network news subsystem • 8: UUCP subsystem • 9: clock daemon • 10: security/authorization messages • 11: FTP daemon • 12: NTP subsystem • 13: log audit • 14: log alert • 15: clock daemon (note 2) • 16-23: local0-local7
severity	Int	Severity (severity of the message) • 0: Emergency • 1: Alert • 2: Critical • 3: Error • 4: Warning • 5: Notice • 6: Informational • 7: Debug
hostname	String	Host name that identifies the machine that originally sent the syslog message, e.g. FQDN, IP address, or PLC name. If no host name is used, the tag must be set to "-".
appName	String	Application name that identifies the device or application that generated the message. If no application name is used, the tag must be set to "-".
msgID	String	Message ID that identifies the type of message. If no message ID is used, the tag must be set to "-".
message	String	Message text in free form

10.4. Integration into the User Project

You will find a detailed application example for the integration of the library into your user project in the Siemens Industry Online Support:

https://support.industry.siemens.com/cs/ww/en/view/51929235

© Siemens 2026

11. LCoAP

11.1. Overview

11.1.1. Range of functions

The Constrained Application Protocol (CoAP) is a lightweight data transfer protocol designed for resource-constrained devices in IoT (Internet of Things) environments.

Compared with other communication protocols, CoAP has many advantages, such as:

- Allows simple IoT devices (microcontrollers, low-power sensors, or devices that do not support HTTP, among others) to communicate with the Internet.
- Consumes fewer resources than other protocols, such as HTTP, due to smaller packets. This makes it ideal for systems where energy efficiency is critical.
- Supports multiple message formats: Plaintext, XML, JSON, CBOR, and Octet-stream.

The library "LCoAP" provides a set of function blocks for implementing the CoAP protocol in SIMATIC S7-1200/1500 controllers.

11.1.2. CoAP Protocol Overview

CoAP messages are encoded in a simple binary format. Each message consists of a fixed-size 4-byte header, followed by fields token, options, and payload (optional).

Header

The header contains critical control information, including the version, message type, Token length, message code, and message ID.

The first two bits of the header specify the **CoAP version**, which must be set to 1. Any other value is considered invalid and will be ignored. Following the version field, the header defines the **message type**. CoAP supports four types of messages:

- **Confirmable (CON).** Used when reliable delivery is required. Upon transmission, the sender expects a response, which may be either an Acknowledgement or a Reset message. If no response is received within a defined timeout period, the message is retransmitted.
- **Non-confirmable (NON).** Non-confirmable messages do not require any form of acknowledgment. They are particularly suitable for scenarios involving periodic or repetitive transmissions.
- **Acknowledgment (ACK).** Acknowledgement messages are sent in response to a Confirmable message to indicate that it was successfully received. By itself, an Acknowledgement message does not indicate success or failure of any request contained in the Confirmable message. However, it may carry a piggybacked response, allowing the recipient to include the actual reply within the Acknowledgement.
- **Reset (RST).** A Reset message indicates that a specific message (Confirmable or Non-confirmable) was received, but it could not be processed due to missing context.

CoAP uses a **Token** to correlate requests and responses at the application level. The Token is an opaque byte sequence generated by the client and repeated by the server in response. This mechanism allows matching responses to their original requests, even when multiple transactions are active simultaneously. **Token length** is defined at the header, after the message type and must not exceed eight bytes, otherwise it will be considered invalid.

The **Message Code** defines the semantics of the message, indicating whether it is a request, response, or empty message. For requests, it encodes the method being invoked. The following methods are supported:

- **GET (0.01).** Retrieves the current state or value of a resource.
- **POST (0.02).** Submits data to the server for processing.
- **PUT (0.03).** Updates or creates a resource with the provided representation.
- **DELETE (0.04).** Removes the specified resource from the server.

Once the server receives and processes a request, it sends back a response. The message code in the response indicates the result of the server's attempt to interpret and fulfil the request. The first three bits of the 8-bit response code number define the class of response:

- **Success (2.xx).** The request was successfully received, understood, and accepted.
- **Client Error (4.xx).** The request contains bad syntax or cannot be fulfilled
- **Server Error (5.xx).** The server failed to fulfil an apparently valid request.

Finally, the header contains the **message ID**, a 16-bit unsigned integer used to detect duplicates and to match ACK or RST messages with their corresponding CON requests.

Token

The header is followed by the **Token** field, whose length is specified in the Token Length field of the header. The Token is an opaque byte sequence and is generated by the client to uniquely identify a request. The server must use the same Token in its response, allowing the client to correlate replies with their corresponding requests, even when multiple transactions are active simultaneously. The Token is used only for application-level matching and does not influence transport-layer behaviour.

Options

The **Options** field provides additional parameters that define how the message should be processed. Options are identified by a number, and they must appear in ascending order inside the message. Each option is encoded using a delta-based format, which reduces the message size by representing option numbers as the difference from the previous option.

In Table 11-1, all the options supported by the protocol are listed. For information regarding each option, refer to the CoAP standard (RFC 7252).

Table 11-1: Options supported by LCoAP

Number	Name	Format	Length	Supported by default
3	Uri-Host	String	1-255	No
7	Uri-Port	UInt	0-2	No
8	Location-Path	String	0-255	No
11	Uri-Path	String	0-255	Yes
12	Content- Format	UInt	0-2	Yes
14	Max-Age	UInt	0-4	No
15	Ury-Query	String	0-255	Yes
17	Accept	UInt	0-2	No
20	Location-Query	String	0-255	No
35	Proxy-Uri	String	1-1034	No
39	Proxy-Scheme	String	1-255	No
60	Size	UInt	0-5	No

NOTE Although LCoAP is prepared to support all the options listed in Table 11-1, the library blocks only implement the following options by default: **Content-Format, Uri-Path and Uri-Query**.

If any of the other options are required, the user must modify the library blocks accordingly.

Payload

The **payload** carries the actual data being transmitted and may be omitted in messages when no data is required, such as empty acknowledgments. When present, the payload must be preceded by a payload marker (0xFF), which explicitly indicates the start of the payload section. The format and interpretation of the payload depend on the Content-Format option, if provided.

NOTE LCoAP does not provide blocks to encode or decode the payload, only the header. It must be the user who implements this, depending on the use case and format of the data being sent or received.

For example, the Library for Data Streams (LStream) can be used for serializing and deserializing XML and JSON payloads.

11.1.3. Components of the library

The "LCoAP" library contains the following objects:

Main Blocks

Table 11-2: Main blocks of the library

Name	Type	Version	Description
LCoAP_ClientServer	FB	V1.1.0	Implement both CoAP server and client functions.
LCoAP_EncodeHeader	FC	V1.1.0	Encodes the header of a CoAP message.
LCoAP_DecodeHeader	FC	V1.1.0	Decodes the header of a CoAP message.
LCoAP_GenerateMessageId	FC	V1.1.0	Generates "random" values for message IDs.
LCoAP_GenerateToken	FC	V1.1.0	Generates "random" values for token values.
LCoAP_TokenCompare	FC	V1.1.0	Determines whether two tokens match.

Subordinate blocks

Subordinate blocks are auxiliary functions called by the main blocks to perform specific tasks, such as encoding/decoding options. While they are not intended for direct use in user projects, they can be reused if needed. Their purpose is to simplify code reuse, enhance readability, and support modular design, making the library easier to maintain and extend.

Table 11-3: Subordinate blocks of the library

Name	Type	Version	Description
LCoAP_EncodeOptionLengthDelta	FC	V1.1.0	Encodes option length and delta.
LCoAP_EncodeOptionString	FC	V1.1.0	Encodes String formatted options.
LCoAP_EncodeOptionUInt	FC	V1.1.0	Encodes UInt formatted options.
LCoAP_EncodePayloadMarker	FC	V1.1.0	Encodes a payload marker (0XFF).
LCoAP_DecodeCode	FC	V1.1.0	Decodes the message code.
LCoAP_DecodeLengthDelta	FC	V1.1.0	Decodes option length and delta.
LCoAP_DecodeOptionString	FC	V1.1.0	Decodes String formatted options.
LCoAP_DecodeOptionUInt	FC	V1.1.0	Decodes UInt formatted options.
LCoAP_DecodePayloadMarker	FC	V1.1.0	Detects start of payload marker (0XFF).

Name	Type	Version	Description
LCoAP_DecodeToken	FC	V1.1.0	Decodes token from the message.
LCoAP_DecodeType	FC	V1.1.0	Decodes the message type.

PLC Data Types

Table 11-4: PLC library data types

Name	Version	Description
LCoAP_typeCode	V1.1.0	Data type for encoding or decoding the message code of a request or a response.
LCoAP_typeConnectionInfo	V1.1.0	Data type that contains the parameters needed to connect to the remote CoAP partner.
LCoAP_typeContentFormat	V1.1.0	Data type for encoding or decoding the format in which the payload contained in a request or response is encoded. Plaintext, XML, JSON, CBOR, and Octet-stream are supported by default.
LCoAP_typeMessageType	V1.1.0	Data type for encoding and decoding the four possible type of CoAP messages: Confirmable, Non-confirmable, Acknowledgement and Reset.
LCoAP_typeProtocolParameters	V1.1.0	Data type that contains parameters of the mode in which messages are transferred through the CoAP protocol.
LCoAP_typeRcvMessageInfo	V1.1.0	Data type that contains information about a received CoAP message.
LCoAP_typeRequestCode	V1.1.0	Data type for encoding or decoding the four possible types of request message codes: GET, POST, PUT and DELETE.
LCoAP_typeResponseCode	V1.1.0	Data type for encoding or decoding all supported response message codes

11.1.4. Validity

The library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.4
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- CP 1543-1 as of firmware V2.0
- CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V3.2

© Siemens 2026

11.2. LCoAP_ClientServer

11.2.1. Interface description

Description

The function block "LCoAP_ClientServer" integrates the CoAP client and server functionalities, enabling the exchange of CoAP messages with remote endpoints. Communication is handled over UDP and can be configured with custom connection parameters.

Parameter

Figure 11-1: LCoAP_ClientServer

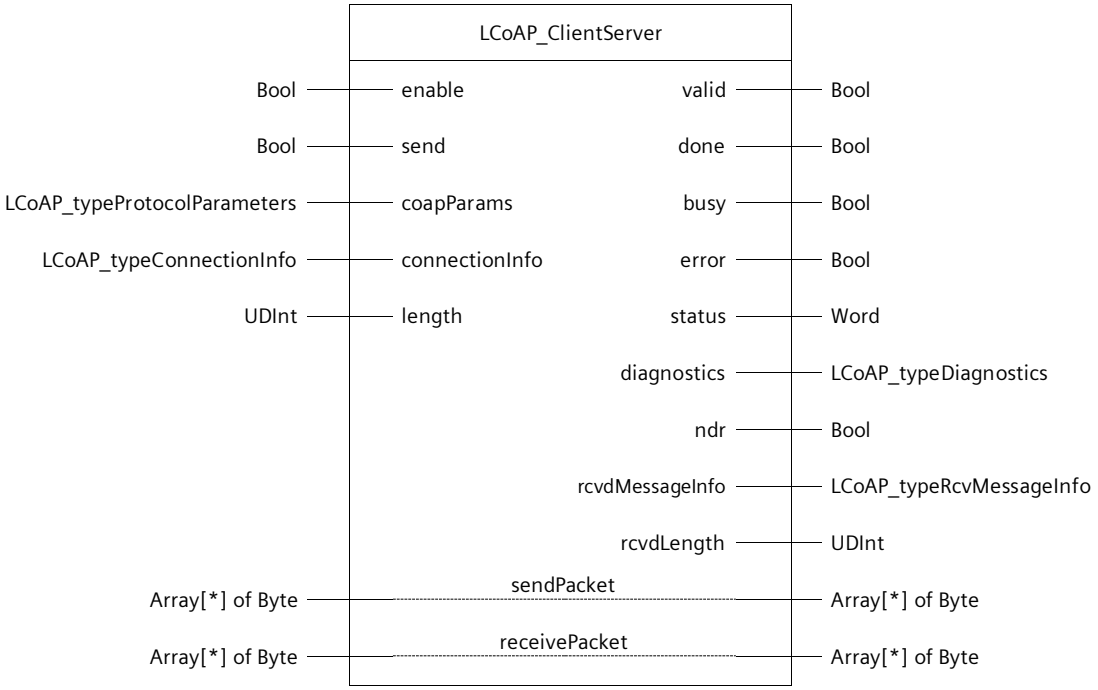


Table 11-5: Parameters of LCoAP_ClientServer

Name	Declaration	Data Type	Comment
enable	Input	Bool	With a positive edge, the block is enabled and communication is possible.
send	Input	Bool	With a positive edge, the block attempts to send the provided message to the configured remote partner.
coapParams	Input	LCoAP_typeProtocolParameters	Parameters that determine the behaviour of the CoAP communication carried out by the block.
connectionInfo	Input	LCoAP_typeConnectionInfo	Parameters needed to connect to the remote CoAP partner.
length	Input	UDInt	Length of the message to be sent in bytes.
valid	Output	Bool	TRUE: The FB executes its function without error.
Done	Output	Bool	TRUE: The current job has finished successfully.

© Siemens 2026

Name	Declaration	Data Type	Comment
busy	Output	Bool	TRUE: Block execution has not finished, and new values can be expected.
error	Output	Bool	TRUE: An error occurred during the execution of the FB.
status	Output	Word	Status / Error code of the block (see chapter 11.2.3)
diagnostics	Output	"typeDiagnostics"	Diagnosis information of the block (see chapter 14.2).
ndr	Output	Bool	TRUE: A new message has been received.
rcvdMessageInfo	Output	LCoAP_typeRcvMessageInfo	Information about the new received message.
rcvdLength	Output	UDInt	Length of the received message in bytes.
sendPacket	InOut	Array[*] of Byte	Packet to send. The byte array must start at 0 and have a maximum size of 2048 bytes.
receivePacket	InOut	Array[*] of Byte	Received packet. The byte array must start at 0 and have a maximum size of 2048 bytes.

11.2.2. Operation

The LCoAP_ClientServer function block facilitates the exchange of CoAP messages over UDP, supporting both client and server roles. Its operational role (client or server) is dynamically determined by the communication context, specifically, which party initiates the communication and the nature of the request. This is, whoever makes the request assumes the role of client, while whoever sends the response acts as the server.

For instance, if the block receives a GET request from a remote device, it operates as a server, and will answer with the requested data from the PLC. On the other hand, if the PLC sends a GET request to retrieve information from a CoAP-enabled device, the block operates as a client.

Regardless of the role assumed during communication, all messages sent by the block adhere to a consistent set of rules and are governed by the same configuration parameters, as detailed below.

Unsolicited messages

Whenever the block receives a message that it is not expecting, the message is flagged as **unsolicited** (rcvdMessageInfo.isUnsolicited = TRUE). In contrast, messages received as direct responses to requests sent by the block are considered part of the expected CoAP communication flow and are therefore not flagged.

While unsolicited messages typically indicate that the communication was initiated by the partner device, implying the block is operating as server, this is not always the case. Depending on specific configuration parameters and certain operational conditions, unsolicited messages may also be encountered when the block is operating as client. These scenarios are outlined in the sections below.

CON Messages – Retransmissions

When the block sends a Confirmable (CON) message, an Acknowledgement (ACK) message is expected from the receiver. If the PLC does not receive an ACK during the period defined by the parameter "**coapParams.ackTimeout**", the block automatically initiates a retransmission process.

The number of retries is limited by the parameter "**coapParams.retries**". If the retries parameter is set to 0, then the retries are unlimited, provided that the response has not timed out (see the section on response timeout below).

Once all retry attempts have been exhausted, the block will consider the request to have expired and will raise an error. If a response to the expired request is subsequently received, the message will be considered unsolicited, as it no longer aligns with the expected communication flow.

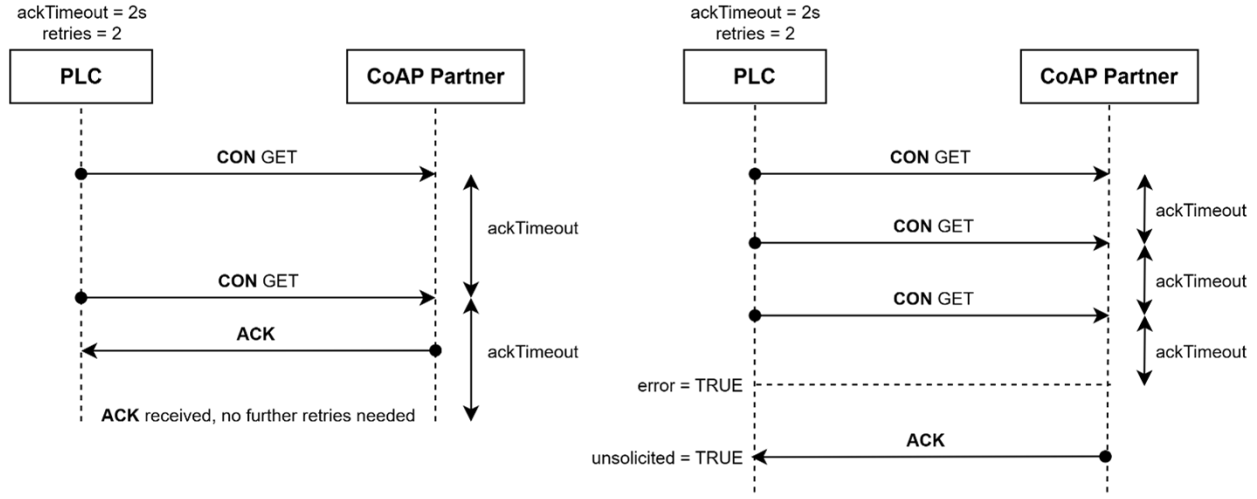


Figure 11-2: Retransmission of a CoAP message

CON Messages – Piggybacked responses

If the ACK to a CON message is received in a timely manner, the block will then check whether the ACK has a piggybacked response. If the received ACK message is not empty or the "**coapParams.expectPiggybackPayload**" parameter is activated, the block will consider that the response to the request has been received and will consider it finished.

On the other hand, if the received ACK is empty and the "**coapParams.expectPiggybackPayload**" parameter is deactivated, the block will wait for the response to be received in a separate CON message. This asynchronous communication mechanism is used in CoAP since many devices do not have the computing power needed to generate the response as they receive the request. They simply send an empty ACK, confirming that the request has been received, and will later send the requested content when it is ready.

Note that, in the first case where the "**coapParams.expectPiggybackPayload**" parameter is activated, if the communication partner were to subsequently send a message with content, this will be considered unsolicited.

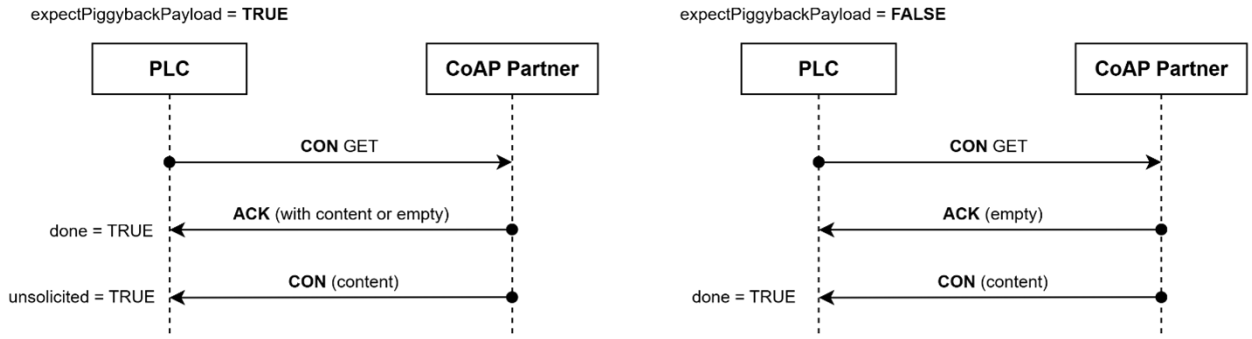


Figure 11-3: Piggybacked responses

CON Messages – Response timeout

When the block sends a CON message, it expects the response to be received during a certain period. This is independent of whether the response is received piggybacked in the ACK, or in a separate message. This time is defined in the "**coapParams.responseTimeout**" parameter, and it starts counting right after the first attempt to send the CON message is made.

For the case mentioned in the retransmission section, where the retries are unlimited: if the ACK is not received, the block will try to retransmit the message every time the ACK timeout expires, but only until the response times out.

For the case mentioned in the piggybacked response section, where the received ACK is empty and the piggybacked payload is not expected: the block will wait for the response to arrive in a separate CON message, but only until the response times out.

In both situations, if the ACK or the separate CON message is not received before the response timeout expires, the block will raise an error, and, like before, if the response arrives after this moment, these message will be considered unsolicited.

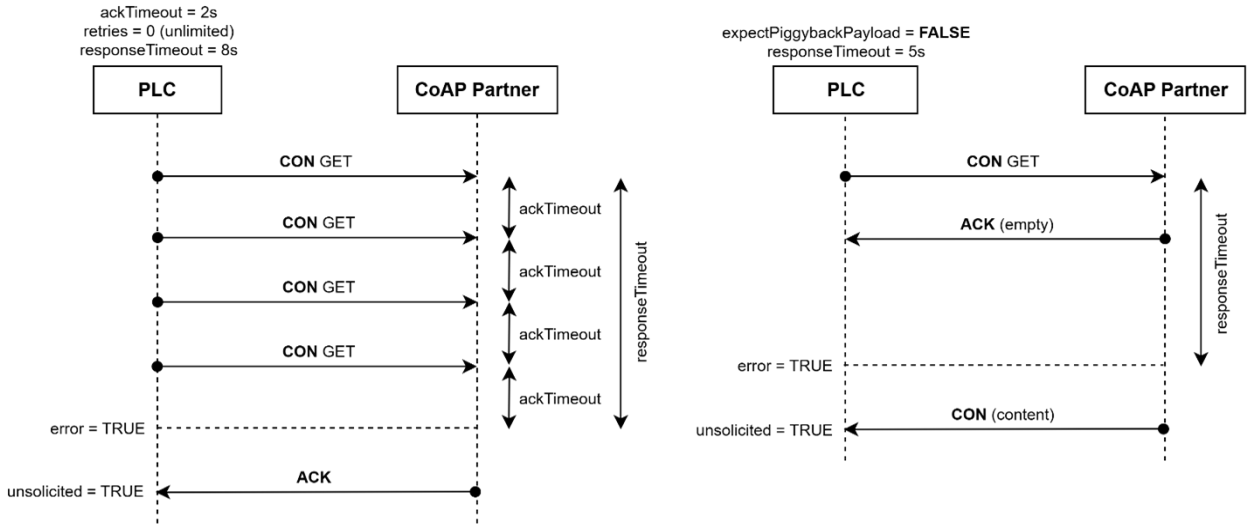


Figure 11-4: Response timeout

CON Messages – Automatic acknowledgement

In cases where the block sends a CON message and the response is not received piggybacked in the ACK, but rather in a separate CON message, this latter CON message must also be acknowledged. The behaviour of the block in this situation is determined by the "**coapParams.autoACK**" parameter.

If the parameter is activated, the block will automatically build an ACK message and send it to the remote partner, reducing programming effort and ensuring timely response to CON messages. However, if the parameter is deactivated, the ACK must be built manually and then sent by the block.

Normally, this parameter will be kept activated, but manual acknowledgement might be required for certain applications.

Note that the automatic acknowledgement feature of the block only works for CON messages that are part of the expected CoAP communication flow. This means that automatic acknowledgements are not sent for received unsolicited CON messages.

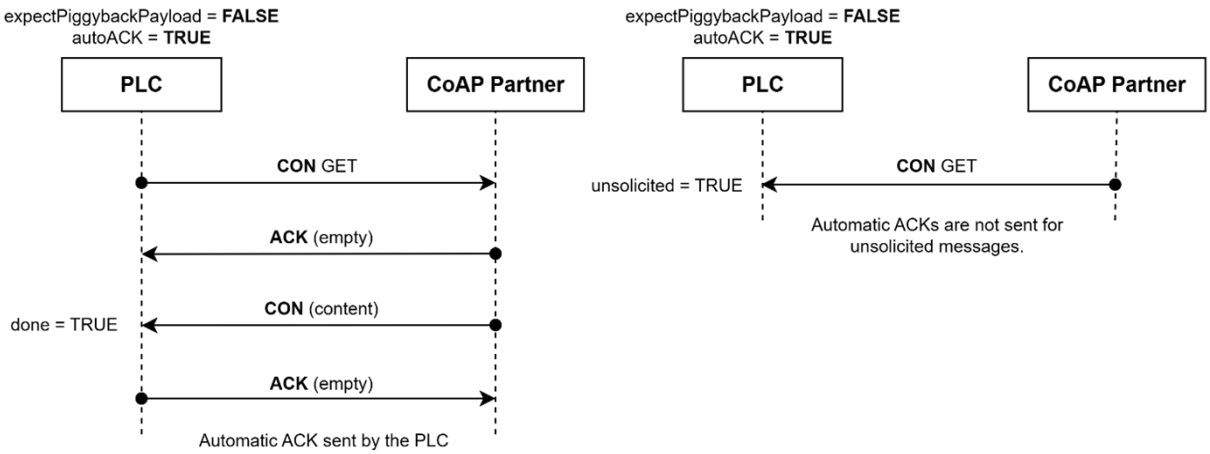


Figure 11-5: Automatic acknowledgement of CON messages

© Siemens 2026

CON Messages – Tokens

As mentioned in the protocol overview, CoAP uses tokens to correlate replies with their corresponding requests. If the block sends a CON message with a specific token, the response must also contain the same token. If a CON message is received as a response to the request, but the tokens do not match, this message will be considered unsolicited.

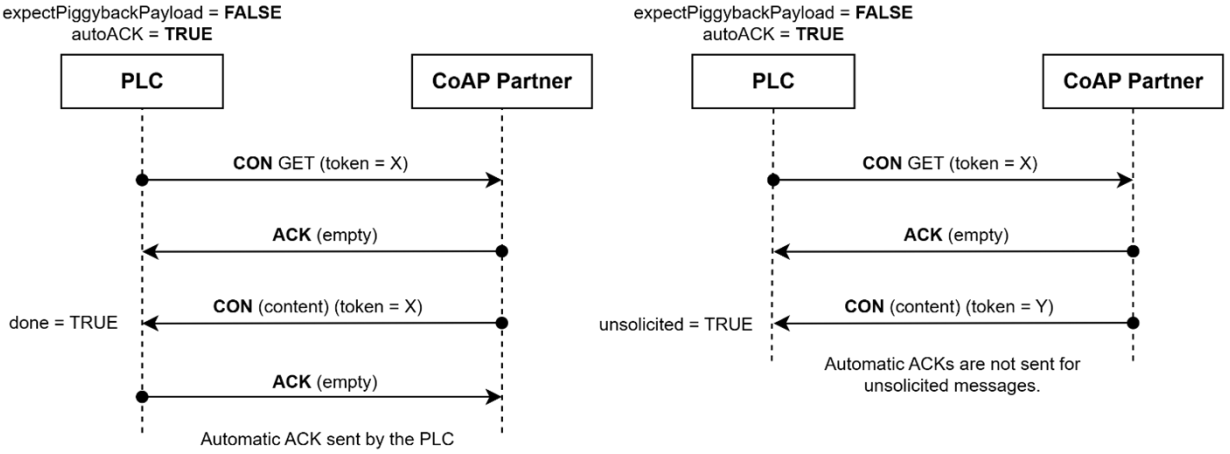


Figure 11-6: CoAP Tokens

NON, ACK and RST messages

When Non-confirmable (NON), Acknowledgment (ACK) or Reset (RST) messages are sent, the block does not expect any response from the receiver, so the message is sent immediately, no retries are performed, and the block does not consider any of the previously explained timeouts.

The job is completed once the packet is transmitted. As such, all responses to these types of messages will be considered unsolicited by the block.

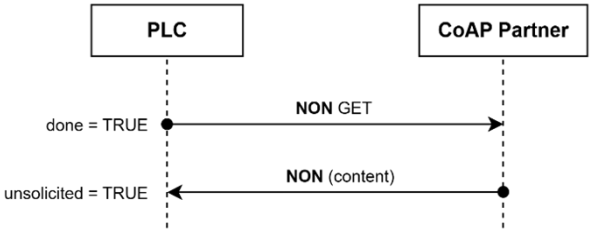


Figure 11-7: Send non-confirmable messages

© Siemens 2026 |

11.2.3. Error handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The status and error codes specific to the block “LCoAP_ClientServer” are listed below. Note that codes between 16#0000 and 16#7FFF are status codes, whereas codes between 16#8000 and 16#FFFF are error codes.

Table 11-6: Status and error codes of the block LCoAP_ClientServer

Code	Description
16#7000	No job being currently processed.
16#7001	First call after incoming new job.
16#7002	Subsequent call during active processing without further details.
16#7003	Connected and ready to receive/send.
16#7004	Awaiting response for a sent CON message.
16#7005	Acknowledgement message received.
16#7006	Response for previously sent CON message received.
16#7007	NON, ACK or RST message sent (no response expected).
16#8210	The packet to be sent is malformed or invalid.
16#8220	CoAP message version different from 1.
16#8230	Retries limit exceeded.
16#8240	Number of retries has changed and surpassed the configured limits.
16#8250	Received RST message.
16#8260	No response from service within response time limit.
16#8610	Error on TCON. The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8620	Error on TSEND. The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8630	Error on TRCV. The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

© Siemens 2026

11.3. LCoAP_EncodeHeader

11.3.1. Interface description

Description

The "LCoAP_EncodeHeader" block is responsible for constructing a valid CoAP message by encoding the header and options using the information provided in the inputs.

Parameter

Figure 11-8: Block LCoAP_EncodeHeader

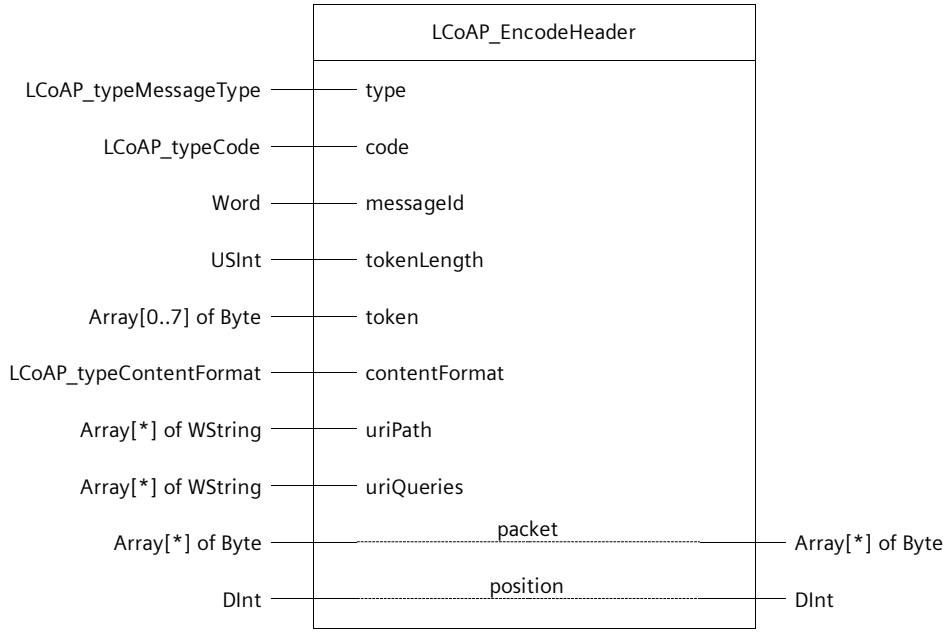


Table 11-7: Parameters of LCoAP_EncodeHeader

Name	Declaration	Data type	Comment
type	Input	LCoAP_typeMessageType	Type of the message. Only one type must be selected.
code	Input	LCoAP_typeCode	Code of the message. It defines the type of response/request.
messageId	Input	Word	Message identifier. Set to 0 to let the network stack auto-generate it.
tokenLength	Input	USInt	Length (in bytes) of the token to encode. It must be between 0 and 8 (defaults to 0).
token	Input	Array [0...7] of Byte	Token to be encoded as byte array.
contentFormat	Input	LCoAP_typeContentFormat	Content-Format to be encoded. Specifies the format of the payload.
uriPath	Input	Array [*] of WString	Uri-Path to be encoded into the CoAP message. Each element of the array represents an individual segment of the path.
uriQueries	Input	Array [*] of WString	Uri-Queries to be encoded into the CoAP message. Each element of the array represents an individual query.

© Siemens 2026

Name	Declaration	Data type	Comment
packet	InOut	Array of [*] Byte	Byte array where the encoded CoAP message is being built.
position	InOut	DInt	Current write position in the packet array. Updated as encoding progresses.
Ret_Val	Return	Word	Status / Error code of the block (see 11.3.4)

11.3.2. Operation

The "LCoAP_EncodeHeader" builds the header of a CoAP message and adds the options.

Header

For specific information regarding the encoding of the CoAP header, please refer to the TIA Portal block description, or the CoAP standard (RFC 7252).

Only one message type can be encoded onto the CoAP header. If multiple message types are selected on the "type" input, or no type is selected at all, then the encoding will fail and the block will raise an error.

Likewise, only one message code can be encoded onto the CoAP header. However, in this case, if multiple message codes are selected on the "code" input, only the first one will be encoded. If no code is selected, the code will be empty.

If the token length is lower than 0 or higher than 8, the encoding will fail and the block will raise an error.

Options

As mentioned in the CoAP Protocol Overview chapter, LCoAP is prepared to support all options listed on Table 11-1. However, this block only implements encoding of the following options: **Content-Format, Uri-Path and Uri-Query**.

Should any more options be needed, the user can make use of the auxiliar functions LCoAP_EncodeOptionString and LCoAP_EncodeOptionUInt to increase the functionalities of this block.

NOTE If support for more options is added, note that options must be encoded in ascending order of option number. This is because each option is encoded using a delta-based format, which reduces the message size by representing option numbers as the difference from the previous option.

The Uri-Path option specifies the location of a resource in a CoAP server. Each element of the "uriPath" array represents an individual segment of the path. For instance, if the resource we are trying to obtain is located at the "sensor1/temperature" path, then "sensor1" must be set on "uriPath[0]" and "temperature" on "uriPath[1]".

The Uri-Query option specifies the modifiers to the request. Each element of the "uriQueries" array represents an individual query. For instance, if we are trying to read the temperature from the sensor above, we might want to obtain it in degrees Celsius and with a precision of 2 decimals. The query "unit=celsius" must be set on "uriQueries[0]" and the query "decimals=2" must be set on "uriQueries[1]".

The Content-Format option specifies the format in which the payload of the CoAP message is encoded, even though the payload is not encoded by this block.

Options are not mandatory. If left empty, they won't be encoded and the block will not raise any errors.

Payload

LCoAP does not provide blocks to encode the payload, only the header and options. It must be the user who implements this, depending on the use case and format of the data being sent or received.

However, via the "position" InOut, this block informs the user about the index of the last byte used for encoding the header and options onto the packet array. With this information, the user can then use the provided function LCoAP_EncodePayloadMarker to encode the payload marker at the given position, and right after it, encode the payload in the way that is needed for their particular use case.

© Siemens 2026

11.3.3. Auxiliar functions

The LCoAP_EncodeHeader block relies on several auxiliar functions to successfully build CoAP messages (header and options).

11.3.3.1. LCoAP_EncodeOptionLengthDelta

Description

LCoAP_EncodeOptionLengthDelta calculates and encodes the option length and the delta value, based on the difference between the current option number and the previous one.

For specific information regarding the encoding of the CoAP Options length and delta, please refer to the TIA Portal block description, or the CoAP standard (RFC 7252).

Parameter

Figure 11-9: LCoAP_EncodeOptionLengthDelta

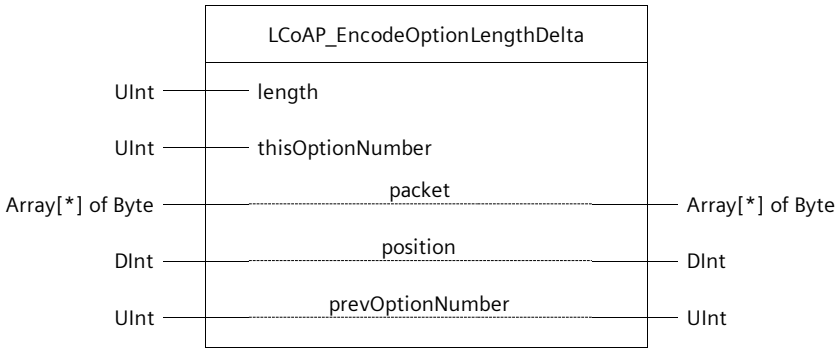


Table 11-8: Parameters of LCoAP_EncodeOptionLengthDelta

Name	Declaration	Data type	Comment
length	Input	UDInt	Length of the option to be encoded.
thisOptionNumber	Input	UDInt	Number of the option to be encoded.
packet	InOut	Array of [*] Byte	Byte array where the encoded CoAP message is being built.
position	InOut	DInt	Current write position in the packet array. Updated as encoding progresses.
prevOptionNumber	InOut	UInt	Option number from the previous call. Used to calculate the delta.
Ret_Val	Return	Word	Status / Error code of the block. 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds. 16#8230: Length of encoded value out of the specified limits. 16#8240: Option delta error. 16#8250: Trying to encode an option with a lower option number than the previously encoded option.

© Siemens 2026

11.3.3.2. LCoAP_EncodeOptionString

Description

LCoAP_EncodeOptionString encodes and adds string-formatted options into the CoAP message, such as Uri-Host, Location-Path, Uri-Path, or Proxy-Uri, among others. For the full list, please refer to the TIA Portal block description.

This function calls LCoAP_EncodeOptionLengthDelta for the task of encoding the length and delta of the option, while this one encodes the value itself.

Parameter

Figure 11-10: LCoAP_EncodeOptionString

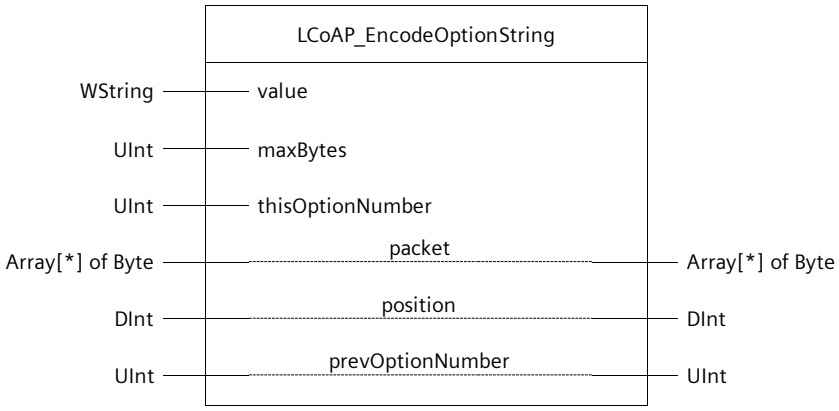


Table 11-9: Parameters of LCoAP_EncodeOptionString

Name	Declaration	Data type	Comment
value	Input	WString	Value as WString of be option to be encoded.
thisOptionNumber	Input	UDInt	Number of the option to be encoded.
maxBytes	Input	UInt	Maximum number of bytes to be encoded, as specified in the CoAP standard.
packet	InOut	Array of [*] Byte	Byte array where the encoded CoAP message is being built.
position	InOut	DInt	Current write position in the packet array. Updated as encoding progresses.
prevOptionNumber	InOut	UInt	Option number from the previous call. Used to calculate the delta.
Ret_Val	Return	Word	Status / Error code of the block. 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds. 16#8220: Length of encoded value out of the specified limits.. 16#8610: Error encoding UTF-8 byte stream

© Siemens 2026

11.3.3.3. LCoAP_EncodeOptionUInt

Description

LCoAP_ EncodeOptionUInt encodes and adds UInt-formatted options into the CoAP message, such as Uri-Port, Content-Format or Max-Age, among others. For the full list, please refer to the TIA Portal block description.

This function calls LCoAP_EncodeOptionLengthDelta for the task of encoding the length and delta of the option, while this one encodes the value itself.

Parameter

Figure 11-11: LCoAP_EncodeOptionUInt

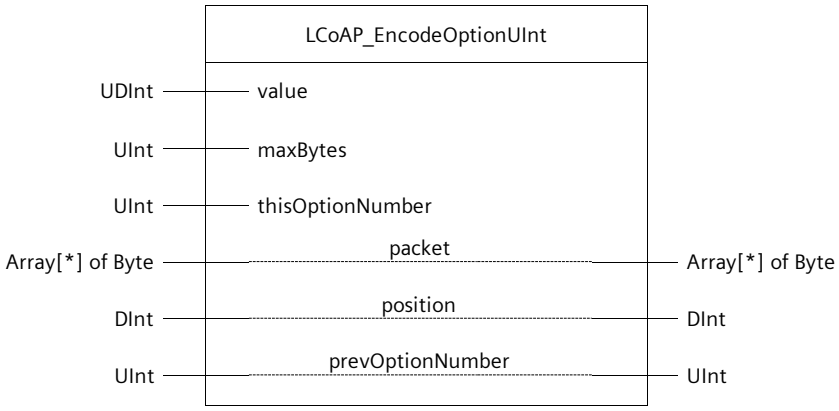


Table 11-10: Parameters of LCoAP_EncodeOptionUInt

Name	Declaration	Data type	Comment
value	Input	UDInt	Value as UDInt of the option to be encoded.
maxBytes	Input	UInt	Maximum number of bytes to be encoded, as specified in the CoAP standard.
thisOptionNumber	Input	UDInt	Number of the option to be encoded.
packet	InOut	Array of [*] Byte	Byte array where the encoded CoAP message is being built.
position	InOut	DInt	Current write position in the packet array. Updated as encoding progresses.
prevOptionNumber	InOut	UInt	Option number from the previous call. Used to calculate the delta.
Ret_Val	Return	Word	Status / Error code of the block. 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds. 16#8220: Length out of the specified limits.

© Siemens 2026

11.3.3.4. LCoAP_EncodePayloadMarker

Description

LCoAP_EncodePayloadMarker inserts a payload marker (16#FF) at the specified position.

Although this function is not used by the LCoAP_EncodeHeader block, it is added here since it can be used by an user who want to add a payload to the CoAP message.

Parameter

Figure 11-12: LCoAP_EncodePayloadMarker

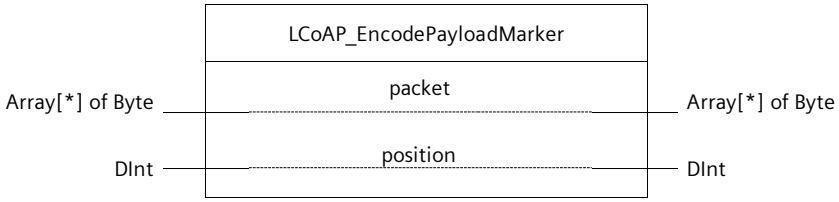


Table 11-11: Parameters of LCoAP_EncodePayloadMarker

Name	Declaration	Data type	Comment
packet	InOut	Array of [*] Byte	Byte array where the encoded CoAP message is being built.
position	InOut	DInt	Current write position in the packet array. Updated as encoding progresses.
Ret_Val	Return	Word	Status / Error code of the block. 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds.

11.3.4. Error handling

The status and error codes specific to the block “LCoAP_EncodeHeader” are listed below. Note that codes between 16#0000 and 16#7FFF are status codes, whereas codes between 16#8000 and 16#FFFF are error codes.

Table 11-12: Status and error codes of the block LCoAP_EncodeHeader

Code	Description
16#0000	Encoding finished without errors.
16#8210	Position parameter is beyond array bounds.
16#8220	Provided message type is invalid.
16#8230	Provided message code is invalid.
16#8240	Provided token length is invalid
16#8610	Unsuccessful encoding of Uri-Path option
16#8620	Unsuccessful encoding of Uri-Query option
16#8630	Unsuccessful encoding of Content-Format option

11.4. LCoAP_DecodeHeader

11.4.1. Interface description

Description

The "LCoAP_DecodeHeader" block is responsible for decoding the content of a received CoAP message, identifying its header (version, type, code, message ID and token) and options. It enables the PLC to interpret incoming CoAP communication and respond accordingly.

Parameter

Figure 11-13: LCoAP_DecodeHeader

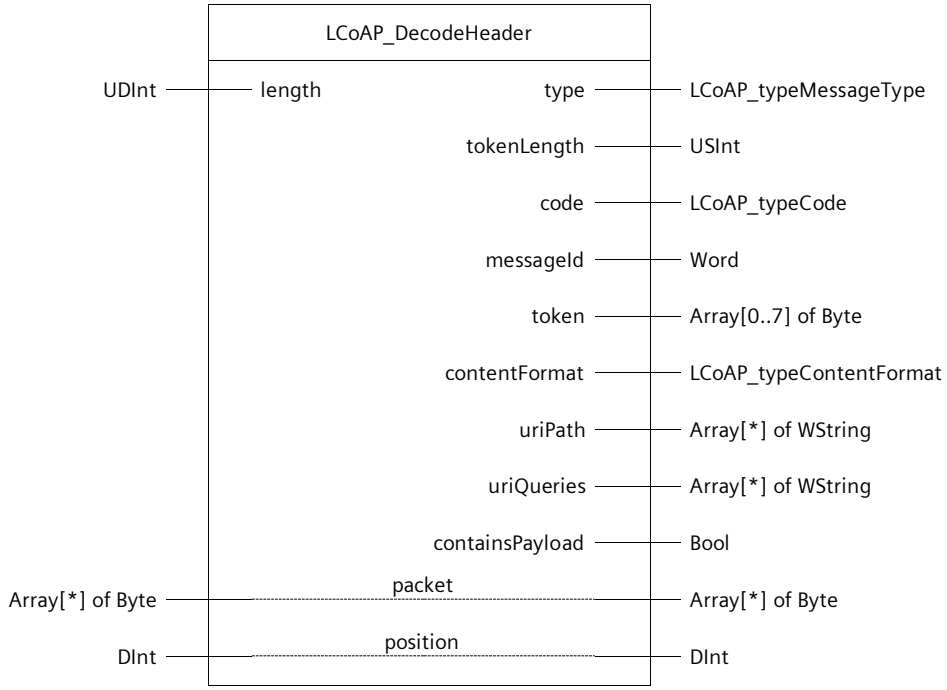


Table 11-13: Parameters of LCoAP_DecodeHeader

Name	Declaration	Data type	Comment
length	Input	UDInt	Length in bytes of the received message.
type	Output	LCoAP_typeMessageType	Decoded CoAP message type.
tokenLength	Output	USInt	Decoded token length.
code	Output	LCoAP_typeCode	Decoded message code. It defines the type of response/request.
messageId	Output	Word	Decoded message identifier.
token	Output	Array [0...7] of Byte	Decoded token as byte array.
contentFormat	Output	LCoAP_typeContentFormat	Decoded Content-Format. Specifies the format of the payload.
uriPath	Output	Array [*] of WString	Decoded Uri-Path. Each element of the array represents an individual segment of the path.

© Siemens 2026

Name	Declaration	Data type	Comment
uriQueries	Output	Array [*] of WString	Decided Uri-Queries. Each element of the array represents an individual query.
containsPayload	Output	Bool	TRUE: The decoded message contains payload.
packet	InOut	Array of [*] Byte	Byte array where the received CoAP message is stored.
position	InOut	DInt	Current read position in the packet array. Updated as decoding progresses.
Ret_Val	Return	Word	Status / Error code of the block (see 11.4.4)

11.4.2. Operation

The “LCoAP_DecodeHeader” extracts the information from a received CoAP message and decodes it in a way that is understandable for the user and helps react accordingly.

Header

For specific information regarding the decoding of the CoAP header, please refer to the TIA Portal block description, or the CoAP standard (RFC 7252).

The block will try to decode the different header fields and store the information in the “type”, “code”, “messageId”, “tokenLenght” and “token” outputs. Should the block fail to decode any of the fields, it will raise an error.

Options

This block works much like the LCoAP_EncodeHeader block, but the other way around. Refer to the explanation of the latter for an overview of the different options.

There are, however, some peculiarities. The LCoAP_DecodeHeader block has been limited to only decode a maximum of 30 options, in order to avoid high processing times. If, for a particular application, this limit needs to be increased, the “MAX_NO_OF_OPTIONS” constant can be modified by the user.

Additionally, if the block encounters while decoding an unsupported option, it will be skipped. The user is notified via a warning, but the decoding does not fail. The block will simply continue decoding the next option.

Like before, this block’s functionalities for decoding options can also be increased by the user if needed. The auxiliar functions LCoAP_DecodeOptionString and LCoAP_DecodeOptionUInt can be used for this purpose.

Payload

As mentioned before, LCoAP does not provide any means to decode the payload contained in the message, only the header and the options. It must be the user who implement this.

However, via de “containsPayload” and “position” Outputs, this block informs the user whether the received CoAP message contains a payload and the index in the packet array where the payload starts (if present). This makes it easy to later implement a custom solution for extracting and decoding the payload appropriately (also using the information provided by the “contentFormat” Output).

© Siemens 2026

11.4.3. Auxiliar functions

LCoAP_DecodeHeader block relies on several auxiliar functions to successfully decode CoAP messages (header and options).

11.4.3.1. LCoAP_DecodeType

Description

LCoAP_DecodeType extracts the CoAP type (Confirmable, Non-confirmable, Acknowledgement, Reset) from the first byte of the header of the received message.

Parameter

Figure 11-14: LCoAP_DecodeType

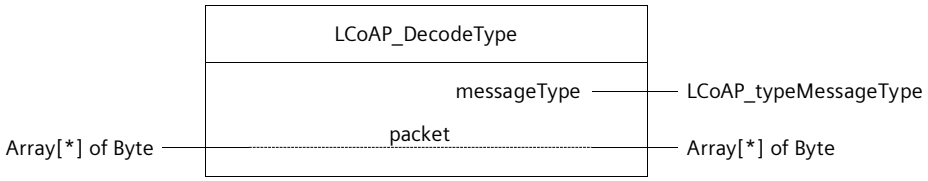


Table 11-14: Parameters of LCoAP_DecodeType

Name	Declaration	Data type	Comment
messageType	Output	LCoAP_typeMessageType	Decoded message type.
packet	InOut	Array of [*] Byte	Byte array where the received CoAP message is stored.
Ret_Val	Return	Word	Status / Error code of the block. 16#0000: Execution finished without errors. 16#8220: Unexpected message type.

11.4.3.2. LCoAP_DecodeCode

Description

LCoAP_DecodeCode extracts the code (request, response, empty) from the first byte of the header of the received message.

Parameter

Figure 11-15: LCoAP_DecodeCode

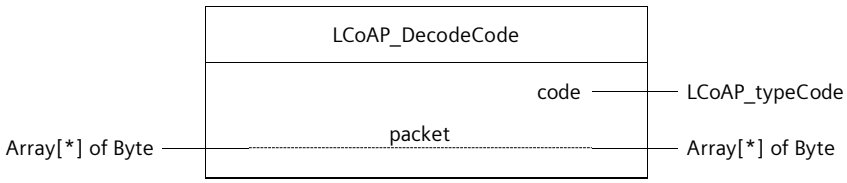


Table 11-15: Parameters of LCoAP_DecodeCode

Name	Declaration	Data type	Comment
code	Output	LCoAP_typeCode	Decode message code.
packet	InOut	Array of [*] Byte	Byte array where the received CoAP message is stored..

© Siemens 2026

Name	Declaration	Data type	Comment
Ret_Val	Return	Word	Status / Error code of the block. 16#0000: Execution finished without errors. 16#8220: Unexpected message code.

11.4.3.3. LCoAP_DecodeToken

Description

LCoAP_DecodeToken extracts the token and its length from the header of the received CoAP message.

Parameter

Figure 11-16: LCoAP_DecodeToken

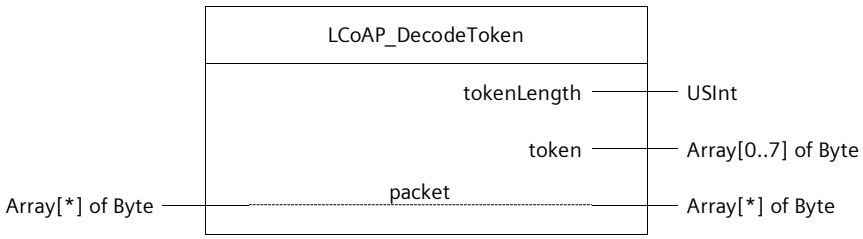


Table 11-16: Parameters of LCoAP_DecodeToken

Name	Declaration	Data type	Comment
tokenLength	Output	USInt	Decode token length.
token	Output	Array [0...7] of Byte	Decoded token as byte array.
packet	InOut	Array of [*] Byte	Byte array where the received CoAP message is stored.
Ret_Val	Return	Word	Status / Error code of the block. 16#0000: Execution finished without errors. 16#8220: Token outside array bounds. 16#8230: Token length is not valid.

11.4.3.4. LCoAP_DecodeOptionLengthDelta

Description

LCoAP_DecodeOptionLengthDelta extracts the delta and length values of a CoAP option during decoding.

For specific information regarding the decoding of the CoAP Options length and delta, please refer to the TIA Portal block description, or the CoAP standard (RFC 7252).

© Siemens 2026

Parameter

Figure 11-17: LCoAP_DecodeOptionLengthDelta

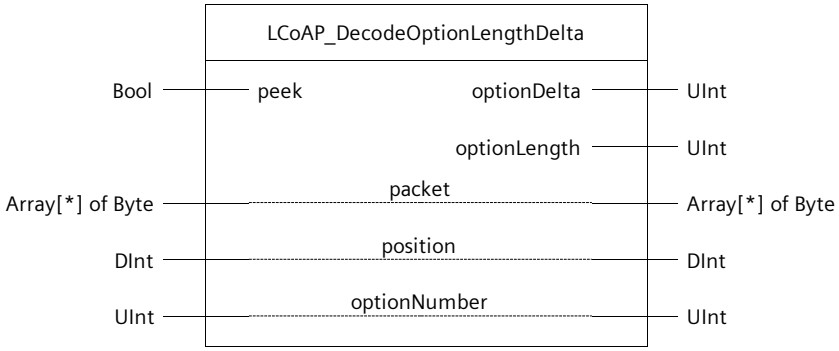


Table 11-17: Parameters of LCoAP_DecodeOptionLengthDelta

Name	Declaration	Data type	Comment
peek	Input	Bool	TRUE: Indicates that the function should only inspect the next option without advancing the position.
optionDelta	Output	UInt	Decoded delta value of the current option.
optionLength	Output	UInt	Decoded length of the current option.
packet	InOut	Array of [*] Byte	Byte array where the encoded CoAP message is stored.
position	InOut	Dint	Current read position in the packet array. Updated as decoding progresses.
optionNumber	InOut	UInt	Option number after applying the delta.
Ret_Val	Return	Word	Status / error code of the block. 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds. 16#8230: Option length out of specification. 16#8240: Option delta out of specification. 16#8250: No option provided.

11.4.3.5. LCoAP_DecodeOptionString

Description

LCoAP_DecodeOptionString is used to extract and decode string-formatted CoAP options from a received message, such as Uri-Host, Location-Path, Uri-Path, or Proxy-Uri, among others. For the full list, please refer to the TIA Portal block description.

This function calls LCoAP_DecodeOptionLengthDelta for the task of decoding the length and delta of the option, while this one decode the value itself.

© Siemens 2026

Parameter

Figure 11-18: LCoAP_DecodeOptionString

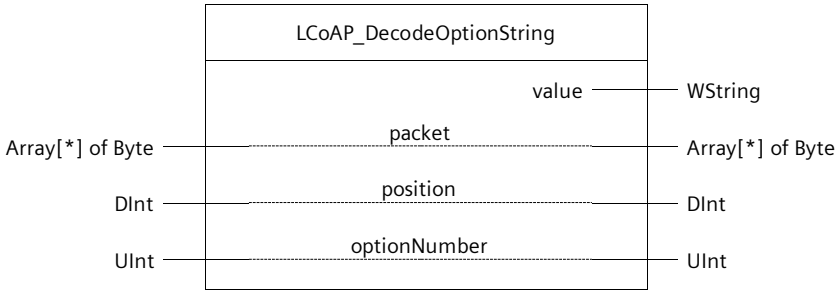


Table 11-18: LCoAP_DecodeOptionString

Name	Declaration	Data type	Comment
value	Output	WString	Decoded option value as WString.
packet	InOut	Array of [*] Byte	Byte array where the received CoAP message is stored.
position	InOut	DInt	Current read position in the packet array. Updated as decoding progresses.
optionNumber	InOut	UInt	Option number after applying the delta.
Ret_Val	Return	Word	Status / Error code of the block 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds. 16#8610: Error decoding UTF8 byte stream.

11.4.3.6. LCoAP_DecodeOptionUInt

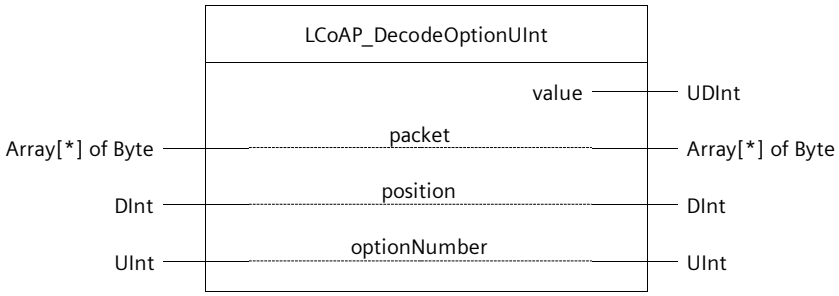
Description

LCoAP_DecodeOptionUInt is used to extract and decode UInt-formatted CoAP options from a received message, such as Uri-Port, Content-Format or Max-Age, among others. For the full list, please refer to the TIA Portal block description.

This function calls LCoAP_DecodeOptionLengthDelta for the task of decoding the length and delta of the option, while this one decode the value itself.

Parameter

Figure 11-19: LCoAP_DecodeOptionUInt



© Siemens 2026

Table 11-19: Parameters of LCoAP_DecodeOptionUInt

Name	Declaration	Data type	Comment
value	Output	UDInt	Decoded option value as UDInt.
packet	InOut	Array of [*] Byte	Byte array where the received CoAP message is stored.
position	InOut	DInt	Current read position in the packet array. Updated as decoding progresses.
optionNumber	InOut	UInt	Option number after applying the delta.
Ret_Val	Return	Word	Status / Error code of the block 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds. 16#8220: Length of the encoded value out of the specified limits.

11.4.3.7. LCoAP_DecodePayloadMarker

Description

LCoAP_DecodePayloadMarker is tasked with detecting the payload marker (0xFF) within a CoAP message being decode.

Parameter

Figure 11-20: LCoAP_DecodePayloadMarker

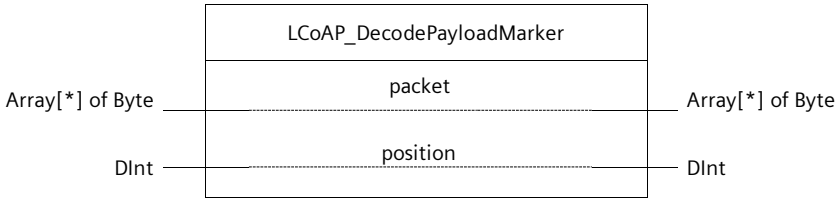


Table 11-20: Parameters of LCoAP_DecodePayloadMarker

Name	Declaration	Data type	Comment
packet	InOut	Array of [*] Byte	Byte array where the received CoAP message is stored.
position	InOut	DInt	Current read position in the packet array. Updated as decoding progresses.
Ret_Val	Return	Word	Status / Error code of the block 16#0000: Execution finished without errors. 16#8210: Position parameter is beyond array bounds. 16#8220: Payload marker not found.

© Siemens 2026

11.4.4. Error handling

The status and error codes specific to the block "LCoAP_DecodeHeader" are listed below. Note that codes between 16#0000 and 16#7FFF are status codes, whereas codes between 16#8000 and 16#FFFF are error codes.

Table 11-21: Status and error codes of the block LCoAP_DecodeHeader

Code	Description
16#0000	Execution finished without errors
16#7003	Warning: Unsupported content format.
16#7004	Warning: Found additional unsupported options.
16#8210	Position parameter is beyond array bounds.
16#8220	Unexpected or unrecognized message type
16#8230	Unexpected or unrecognized message code
16#8240	CoAP version is not one
16#8250	Invalid packet length
16#8610	Error decoding option delta or length
16#8620	Error decoding token
16#8630	Error decoding Uri-Path
16#8640	Error decoding Content-Format
16#8650	Error decoding Uri-Query

© Siemens 2026

11.5. Utilities

11.5.1. LCoAP_GenerateMessageID

Description

The LCoAP_GenerateMessageID function block is designed to generate pseudo-random message IDs, ensuring that each CoAP message has a unique identifier. These values are generated by using the nanosecond component of the CPU system time.

This block can be found under the “Utilities” folder of the LCoAP library, and it is used internally by encoding functions, though it can be reused directly if needed.

Parameter

Table 11-22: Parameters of LCoAP_GenerateMessageID

Name	Declaration	Data type	Comment
Ret_Val	Return	UDInt	Random number in the range 0 to 65535.

11.5.2. LCoAP_GenerateToken

Description

The LCoAP_GenerateToken function block is designed to generate a pseudo-random CoAP token of maximum length (8 bytes) as an array of bytes. Tokens are used by the protocol to correlate requests and responses.

This function uses the nanosecond component of the CPU system time to ensure that each message has a different token value.

This block can be found under the “Utilities” folder of the LCoAP library.

Parameters

Figure 11-21: LCoAP_GenerateToken



Table 11-23: Parameters of LCoAP_GenerateToken

Name	Declaration	Data type	Comment
token	Output	Array [0...7] of Byte	Generated token as byte array.

11.5.3. LCoAP_TokenCompare

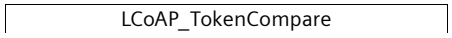
Description

LCoAP_TokenCompare function block is used to detect mismatches between two token values. Tokens are byte arrays which are used by the protocol to correlate requests and responses.

This block can be found under the “Utilities” folder of the LCoAP library, and it is used internally by encoding functions, though it can be reused directly if needed.

Parameters

Figure 11-22: LCoAP_TokenCompare



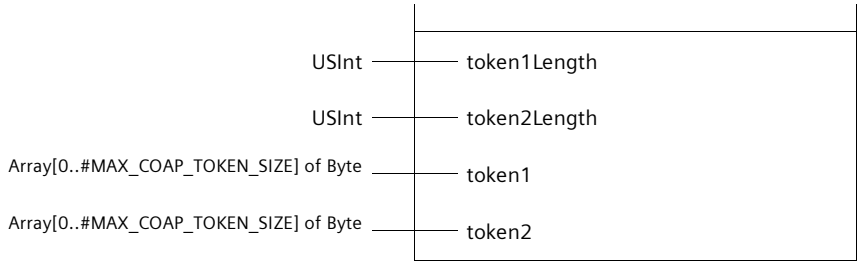


Table 11-24: Parameters of LCoAP_TokenCompare

Name	Declaration	Type	Comment
token1Length	Input	USInt	Length of the first token
token2Length	Input	USInt	Length of the second token
token1	Input	Array [*] of Byte	First token to compare
token2	Input	Array [*] of Byte	Second token to compare
Ret_Val	Return	Bool	• TRUE: Both tokens match • FALSE: Error or token mismatch

© Siemens 2026 |

11.6. PLC Data types

LCoAP_typeConnectionInfo

The following table shows the structure of the PLC data type "LCoAP_typeConnectionInfo".

Table 11-25: PLC data type LCoAP_typeConnectionInfo

Name	Type	Comment
connectionID	UInt	Unique identifier for the CoAP connection.
interfaceID	HW_ANY	Hardware ID of the network interface used for communication.
remoteAddress	String	IP address or FQDN of the remote CoAP partner.
remotePort	ULInt	Destination port number.
localPort	UInt	Local port number used for the CoAP communication . If set to 0, the block will use the port number resulting from adding the connectionID and an offset of 40000.

LCoAP_typeProtocolParameters

The following table shows the structure of the PLC data type "LCoAP_typeProtocolParameters".

Table 11-26: PLC data type LCoAP_typeProtocolParameters

Name	Type	Comment
expectPiggybackPayload	Bool	• TRUE: The block expects the ACK of a Confirmable (CON) request to carry the response payload (piggybacked Response). • FALSE: The block expects the ACK to be sent immediately as an empty message, followed by the response as a separate Confirmable (CON) message.
autoACK	Bool	• TRUE: Enable automatic acknowledgment (ACK) of expected CON messages (does not acknowledge unsolicited messages). • FALSE: Manual acknowledgment control is required.
retries	UInt	Number of times the block will try to send a Confirmable (CON) message if the Acknowledgement (ACK) is not received in a timely manner. If set to 0, the retries are unlimited.
ackTimeout	Time	Adjustable delay for awaiting ACK reception. If after this time the ACK is not received, a new retry will be initiated by the block (unless the number of retries has been exceeded or the response has timed out). Must have a value higher than 0s.
responseTimeout	Time	Adjustable delay for awaiting reception of a response to a CON message sent by the block. If after this time the response is not received, the request will expire. Must have a value higher than 0s.

© Siemens 2026

LCoAP_typeRcvMessageInfo

The following table shows the structure of the PLC data type "LCoAP_typeRcvMessageInfo".

Table 11-27: PLC data type LCoAP_typeRcvdMessageInfo

Name	Type	Comment
rcvAddress	TADDR_Param	IP Address and UDP port of the remote partner that sent the CoAP message.
isUnsolicited	Bool	TRUE: The received message is unsolicited. It could be a new request sent from a remote CoAP client, or an incorrect response (for a request that has already timed out, for example).
isSuccess	Bool	TRUE: The request was successfully received, understood, and accepted.
isClientError	Bool	TRUE: The request contains bad syntax or cannot be fulfilled by the server.
isServerError	Bool	TRUE: The server failed to fulfil an apparently valid request.
type	LCoAP_typeMessageType	Type of the received message.
code	LCoAP_typeCode	Code of the received message.

LCoAP_typeMessageType

The following table shows the structure of the PLC data type "LCoAP_typeMessageType".

Table 11-28: PLC data type LCoAP_typeMessageType

Name	Type	Comment
confirmable	Bool	Message is of type CON. Requires a response/ acknowledgment from receiver.
nonConfirmable	Bool	Message is of type NON. No acknowledgment is expected.
acknowledgment	Bool	Message is of type ACK. Response to a confirmable message.
reset	Bool	Message is of type RST. Indicates error or message rejection.

LCoAP_typeCode

The following table shows the structure of the PLC data type "LCoAP_typeCode".

Table 11-29: PLC data type LCoAP_typeCode

Name	Type	Comment
empty	Bool	The message is empty. Used for ACKs.
request	LCoAP_typeRequestCode	Supported request message codes.
response	LCoAP_typeResponseCode	Supported response message codes.

© Siemens 2026

LCoAP_typeRequestCode

The following table shows the structure of the PLC data type "LCoAP_typeRequestCode".

Table 11-30: PLC data type LCoAP_typeRequestCode

Name	Type	Comment
get	Bool	Code 0.01: Request to read data.
post	Bool	Code 0.02: Request to create/send.
put	Bool	Code 0.03: Request to update data.
delete	Bool	Code 0.04: Request to remove data.

LCoAP_typeResponseCode

The following table shows the structure of the PLC data type "LCoAP_typeResponseCode".

Table 11-31: PLC data type LCoAP_typeResponseCode

Name	Type	Comment
isSuccess	Bool	Indicates a successful response.
isClientError	Bool	Indicates a client-side error.
IsServerError	Bool	Indicates a server-side error.
created	Bool	Code 2.01: Resource created (POST/PUT).
deleted	Bool	Code 2.02: Resource deleted.
valid	Bool	Code 2.03: Entity-tag matched.
changed	Bool	Code 2.04: Resource updated (POST/PUT).
content	Bool	Code 2.05: Successful GET response.
badRequest	Bool	Code 4.00: Malformed request.
unauthorized	Bool	Code 4.01: Missing or invalid credentials.
badOption	Bool	Code 4.02: One or more unrecognized or malformed options.
forbidden	Bool	Code 4.03: Access denied.
notFound	Bool	Code 4.04: Resource not found.
methodNotAllowed	Bool	Code 4.05: Method not allowed.
preconditionFailed	Bool	Code 4.12: Precondition not met.
requestEntityTooLarge	Bool	Code 4.13: Payload too large.
unsopportedContentFormat	Bool	Code 4.15: Unsupported content format.
internalServerError	Bool	Code 5.00: Server error.
notImplemented	Bool	Code 5.01: Method not implemented.
badGateway	Bool	Code 5.02: Invalid upstream response.

© Siemens 2026

Name	Type	Comment
serviceUnavailable	Bool	Code 5.03: Server unavailable.
gatewayTimeout	Bool	Code 5.04: Upstream timeout.
proxyingNotSupported	Bool	Code 5.05: The server is unable or unwilling to function as a forward-proxy for the specified URI.

LCoAP_typeContentFormat

The following table shows the structure of the PLC data type “LCoAP_typeContentFormat”.

Table 11-32: LCoAP_typeContentFormat

Name	Type	Comment
cbor	Bool	Message payload encoded in CBOR format.
json	Bool	Message payload encoded in JSON format.
octet	Bool	Message payload encoded in binary/octet format.
plainText	Bool	Message payload encoded in plain text format.
xml	Bool	Message payload encoded in XML format.

© Siemens 2026

12. LDNS

12.1. Overview

12.1.1. Range of functions

DNS (Domain Name System) is the system to resolve FQDN (Fully Qualified Domain Name) like www.siemens.com to an IP Address. In IT and OT context it has an important function: It makes a host constantly reachable among constantly changing IP addresses.

The LDNS block extends the built-in DNS resolution of the Ethernet interfaces with a DNS resolver. This includes the following features:

- Continuous address resolution, especially for UDP communication

  UDP communication is connectionless, i.e. a change in the target IP address is not detected by the sender. The device solves this problem by resolving periodically. In doing so, it uses the TTL (Time-to-Live) information of the DNS server. If this expires, it will be resolved again.

- Server redundancy and prioritization

  In addition, the same address can be resolved by a second DNS server. The 180° Phase Shift option ensures that the requests to the two servers alternate halfway through the query cycle.

- Caching the IP address in the DB

  Of the two resolved addresses, the more current one is stored in the cache and can be used by the user program, e.g. by the LCoAP module.

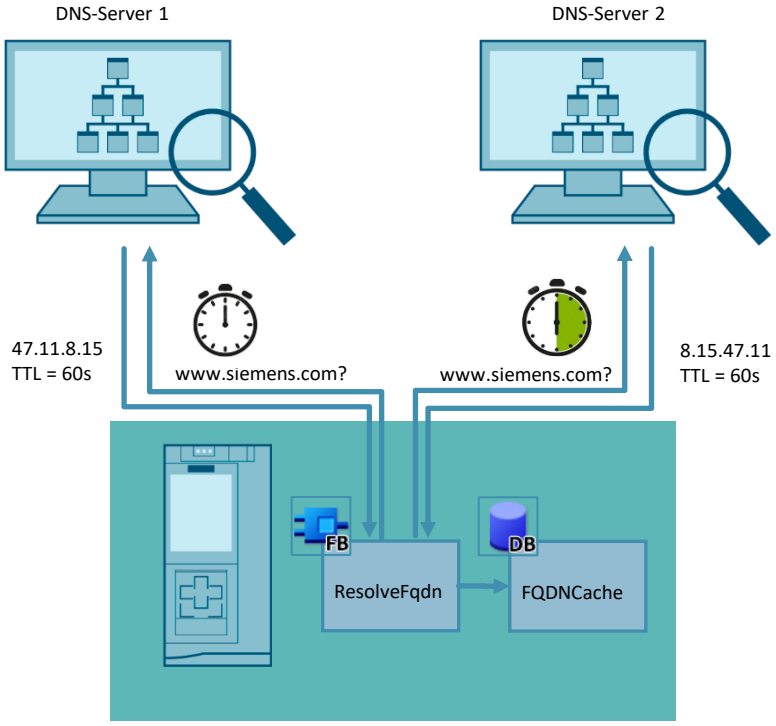


Figure 12-1: Overview

© Siemens 2026

12.1.2. Components of the library

Function blocks

Table 12-1: Function blocks of the library

Name	Type	Version	Description
LDNS_ResolveFqdn	FB	V1.0.1	Controls the DNS requests from the PLC to the servers.

PLC Data Types

Table 12-2: PLC library data types

Name	Version	Description
LDNS_typeResolvedAddress	V1.0.0	Contains FQDN and resolved IP address. Can be applied as array if more than one address needs to be resolved
LDNS_DnsCache	V1.0.0	Part of LDNS_typeResolvedAddress

12.1.3. Validity

The library is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.5
- SIMATIC S7-1200 G2 Controllers as of firmware V1.0
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- CP 1543-1 as of firmware V2.0
- CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V3.2

© Siemens 2026

12.2. LDNS_ResolveFqdn

12.2.1. Interface Description

Figure 12-2: LDNS_ResolveFqdn

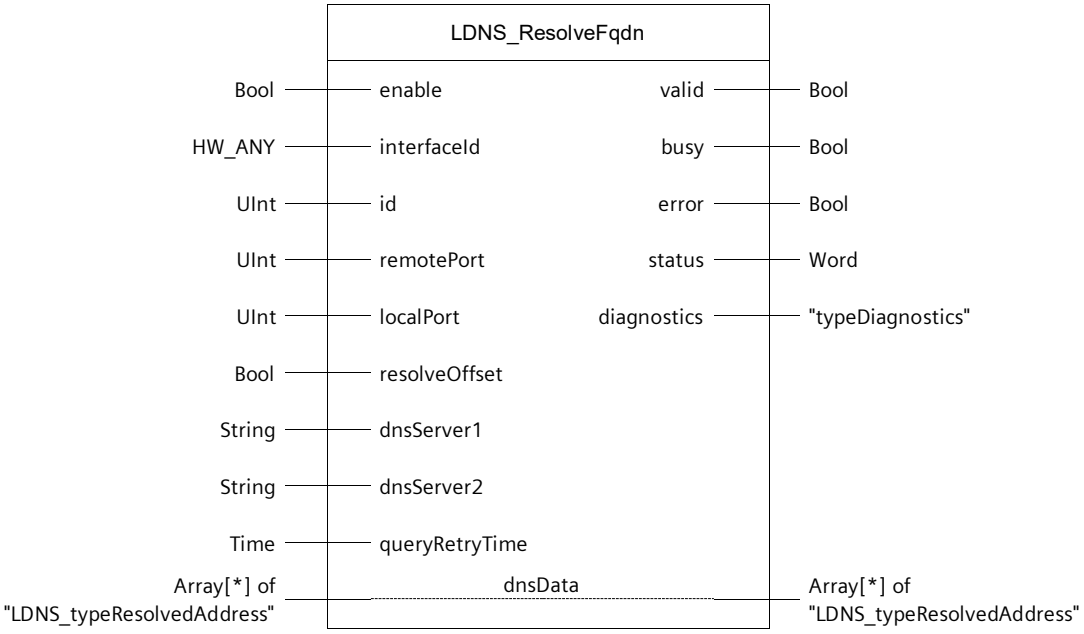


Table 12-3: Parameters of LDNS_ResolveFqdn

Name	Declaration	Data type	Comment
enable	IN	Bool	TRUE: Enable functionality of FB
interfaceId	IN	HW_ANY	HW-identifier of IE-interface submodule
id	IN	UInt	Connection reference / identifier
remotePort	IN	UInt	Remote UDP port (for DNS servers, the default listening port is 53)
localPort	IN	UInt	Local UDP port number (leave as 0 to let the function add 50000 to the connection ID)
resolveOffset	IN	Bool	Attempts to create an 180deg TTL phase offset between caches. Only recommended for servers that provided a consistent TTL for a given FQDN.
dnsServer1	IN	String	Remote IP address of DNS Server 1 (IPv4)
dnsServer2	IN	String	Remote IP address of DNS Server 2 (IPv4)
queryRetryTime	IN	Time	Time between two consecutive request retries if DNS server is not responding
valid	OUT	Bool	TRUE: Valid set of output values available at the FB
busy	OUT	Bool	TRUE: FB is not finished and new output values can be expected
error	OUT	Bool	TRUE: An error occurred during the execution of the FB

© Siemens 2026

Name	Declaration	Data type	Comment
status	OUT	Word	16#0000 - 16#7FFF: Status of the FB, 16#8000 - 16#FFFF: Error identification
diagnostics	OUT	"LCom_typeDiagnosticsFqdn"	Diagnostics information of FB
dnsData	IN_OUT	Array[*] of "LCom_typeResolvedAddress"	Parameter and Results of DNS Requests. Start index must be 0. Maximum number of elements 20

12.2.2. Error handling

The status and error codes specific to the block “LDNS_ResolveFqdn” are listed below. Note that codes between 16#0000 and 16#7FFF are status codes, whereas codes between 16#8000 and 16#FFFF are error codes.

Table 12-4: Error codes of the block "LDNS_ResolveFqdn"

Code	Description
16#8201	Error: Wrong array bounds of dnsData input
16#8202	Error: Unable to decode IPv4 address of DNS Server 1
16#8203	Error: Unable to decode IPv4 address of DNS Server 2
16#8204	Error: DNS Server address changed while processing
16#8401	Error: Bad response from DNS Server or array size changed, response index out of range
16#8600	Error: Due to an undefined state in state machine
16#8601	Error on TCON. The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8602	Error on TUSEND. The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.
16#8603	Error on TURCV. The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system.

© Siemens 2026

12.3. PLC Data Types

LDNS_typeDnsCache

Table 12-5: PLC data type LDNS_typeDnsCache

Name	Type	Comment
valid	Bool	TRUE = This cached IP address is valid and the resolved FQDN matches the requested one.
provided	DTL	Timestamp of the DNS response
expiration	DTL	Expiration timestamp of the cached IP address.
originalDnsTtl	Time	Original TTL (time to live) of cached IP address.
relativeTtl	Time	Remaining TTL (time to live) of the cached IP address.
ipv4Byte1	USInt	(xxx.yyy.yyy.yyy) First byte of the IPV4 address.
ipv4Byte2	USInt	(yyy.xxx.yyy.yyy) Second byte of the IPV4 address.
ipv4Byte3	USInt	(yyy.yyy.xxx.yyy) Third byte of the IPV4 address.
ipv4Byte4	USInt	(yyy.yyy.yyy.xxx) Fourth byte of the IPV4 address.

LDNS_typeResolvedAddress

Table 12-6: PLC data type LDNS_typeResolvedAddress

Name	Type	Comment
fqdnToResolve	String[75]	The domain name to resolve and cache.
resolvedFqdn	String[75]	The domain name that the cache is valid for, used for change detection.
activeCache	"LDNS_typeDnsCache"	Cache with the greatest TTL
dnsCache1	"LDNS_typeDnsCache"	Cached response from DNS server 1
dnsCache2	"LDNS_typeDnsCache"	Cached response from DNS server 2

12.4. Integration into the User Project

At "Master copies" you will find the folder "LDNS_Example", which contains a ready-made configuration. It addresses the internet DNS servers 1.1.1.1 and 9.9.9.9. In this case, the PLC needs a defined standard gateway for internet connection.

Furthermore, the parameter “InterfaceID” at the LDNS_ResolveFqdn FB must be set to match the interface. To start DNS resolving, enable the EN input parameter.

13. Master Copies

13.1. OUC Master Copies

13.1.1. Overview

13.1.1.1. Range of Functions

TCP/IP-based Open User Communication (OUC) has become the standard for communication with SIMATIC Controllers.

In SIMATIC Controller OUC is implemented based on commands (e.g. TCON, TSEND, TRCV, and TDISCON). The user parameterizes the commands in their user program and calls them in an error-tolerant manner.

To facilitate this, we offer function blocks in SCL as master copies.

The FBs call the OUC commands in the order and manner recommended in the manuals. In addition, FBs already contain the following mechanisms:

- Connection management with the commands "TCON" and "TDISCON"
- Send data to a partner
- Receive data from a partner

You can use the FBs as a template for your own communication projects or the implementation of other IP-based protocols.

13.1.1.2. Components

The following FBs are available as master copies:

Table 13-1: Master copies

Name	Description
IsoOnTcpTemplate	Master copy to implement communication via ISO-on-TCP protocol
TcpTemplate	Master copy to implement communication via TCP protocol
TcpSecTemplate	Master copy to implement secured communication via TCP protocol
UdpTemplate	Master copy to implement communication via UDP protocol

© Siemens 2026

13.1.1.3. Validity

The master copies "IsoOnTcpTemplate", "TcpTemplate" and "UdpTemplate" are valid for:

- SIMATIC S7-1500 Controller as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.2
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- SIMATIC S7-300 Controllers
- SIMATIC S7-400 Controllers
- CP 1543-1, CP 1542SP-1, CP 1542SP-1 IRC, CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V2.0

The master copy "TcpSecTemplate" is valid for:

- SIMATIC S7-1500 Controllers as of firmware V2.0
- SIMATIC S7-1200 Controllers as of firmware V4.4
- SIMATIC ET 200SP Open Controllers as of firmware V2.5
- SIMATIC S7-1500 Software Controllers as of firmware V2.5
- CP 1543-1 as of firmware V2.0
- CP 1543SP-1 as of firmware V1.0
- CP 1243-1, CP 1243-8 as of firmware V3.2

© Siemens 2026 |

13.1.2. IsoOnTcpTemplate

Functional description

The FB "IsoOnTcpTemplate" implements a complete ISO-on-TCP communication relation to a partner. It encapsulates all OUC commands in a user-friendly shell to perform the following functions:

- Connection establishment and termination via the input "enable"
- Send user data of length "sendLen" to the partner via the "sendData" input as soon as the "send" input detects a positive edge.
- Receive data from a partner and output it in the receive area at the "rcvdData" parameter.
- Output the status of transmission and connection at the output parameter "status".

NOTE If the received data is stored in an optimized data block, the input parameter "rcvLen" must contain the value 0. For a standard data block, this input parameter must contain the number of bytes to be received.

Parameter

Figure 13-1

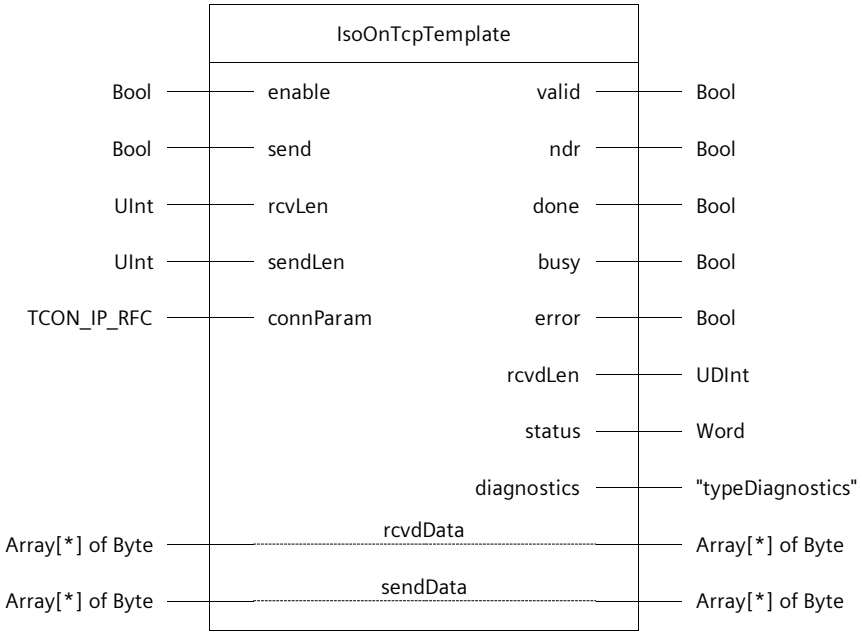


Table 13-2

Name	Declaration	Data type	Comment
enable	Input	Bool	Release signal for connection setup and data exchange
send	Input	Bool	Triggering the send job
rcvLen	Input	UInt	Receive data length If the receive data is stored in an optimized DB, then rcvLen = 0.
sendLen	Input	UInt	Maximum number of bytes sent with the job
connParam	Input	TCON_IP_RFC	Connection parameters
valid	Output	Bool	TRUE: The block executes its function without error
ndr	Output	Bool	TRUE: New data has been received
done	Output	Bool	TRUE: The send job was successfully executed
busy	Output	Bool	TRUE: The block is executed
error	Output	Bool	TRUE: An error has occurred
rcvdLen	Output	UDInt	Length of received data in bytes
status	Output	Word	Status and error codes (see chapter 13.1.6)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
rcvdData	InOut	Array[*] of Byte	Receive data range
sendData	InOut	Array[*] of Byte	Send data range

© Siemens 2026

13.1.3. TcpTemplate

Functional description

The FB "TcpTemplate" implements a complete TCP communication relationship to a partner. It encapsulates all OUC commands in a user-friendly shell to perform the following functions:

- Connection establishment and termination via the input "enable"
- Send user data of length "sendLen" to the partner via the "sendData" input as soon as the "send" input detects a positive edge.
- Receive data from a partner and output it in the receive area at the "rcvdData" parameter.
- Output the status of transmission and connection at the output parameter "status".

NOTE

Enable the Adhoc mode to receive telegrams with dynamic data length. In this case the input parameter "rcvLen" is irrelevant.

Deactivate the Adhoc mode to receive telegrams with fixed data length. In this case you have to specify the number of bytes to be received at the input parameter "rcvLen".

Parameter

Figure 13-2

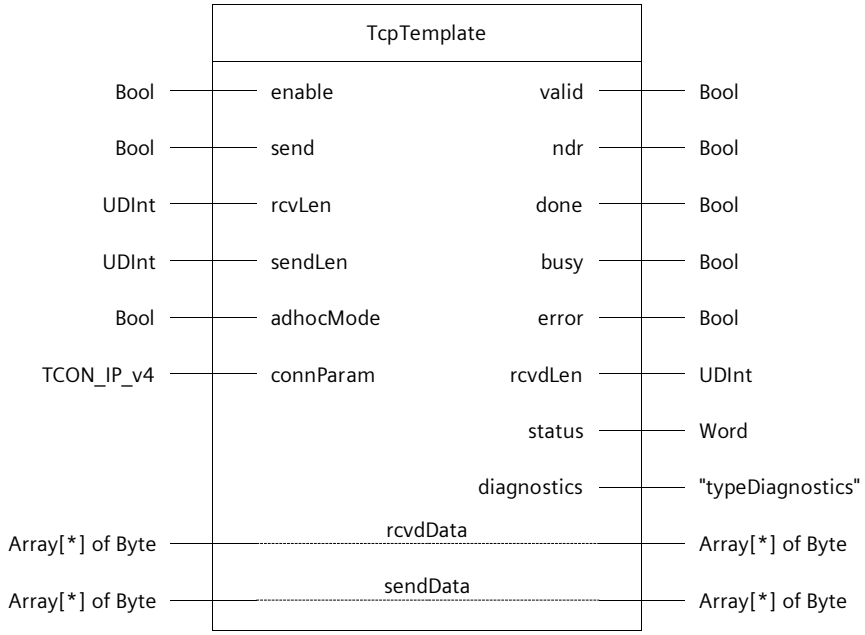


Table 13-3

Name	Declaration	Data type	Comment
enable	Input	Bool	Release signal for connection setup and data exchange
send	Input	Bool	Triggering the send job

© Siemens 2026

Name	Declaration	Data type	Comment
rcvLen	Input	UDInt	Receive data length • S7-1500 CPUs: maximum 65536 bytes • S7-1200 CPUs maximum 8192 bytes **Note** Parameter is irrelevant when Adhoc mode is enabled. As much data is read as is currently available. The max. data length is defined by the length of the receive area referenced by rcvData.
sendLen	Input	UDInt	Maximum number of bytes sent with the job. • S7-1500 CPUs: maximum 65536 bytes • S7-1200 CPUs maximum 8192 bytes
adhocMode	Input	Bool	TRUE (Adhoc enabled): • The data is available immediately. • Data reception with dynamic data length FALSE (Adhoc disabled): • The data is available as soon as the data length specified at parameter LEN has been received completely. • Data reception with specified data length.
connParam	Input	TCON_IP_v4	Connection parameters
valid	Output	Bool	TRUE: The block executes its function without error
ndr	Output	Bool	TRUE: New data has been received
done	Output	Bool	TRUE: The send job was successfully executed
busy	Output	Bool	TRUE: The block is executed
error	Output	Bool	TRUE: An error has occurred
rcvdLen	Output	UDInt	Length of received data in bytes
status	Output	Word	Status and error codes (see chapter 13.1.6)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information
rcvdData	InOut	Array[*] of bytes	Receive data area
sendData	InOut	Array[*] of bytes	Send data area

© Siemens 2026

13.1.4. TcpSecTemplate

Functional description

The FB "TcpSecTemplate" implements a complete TCP communication relationship to a partner.

With the FB "TcpSecTemplate" and with S7-1200 CPUs as of firmware V4.4 and S7-1500 CPUs from firmware V2.0, it is possible to parameterize the communication connections at TCP via IPv4 by means of Secure Communication.

It encapsulates all OUC commands in a user-friendly shell to perform the following functions:

- Connection establishment and termination via the input "enable"
- Send user data of length "sendLen" to the partner via the "sendData" input as soon as the "send" input detects a positive edge
- Receive data from a partner and output it in the receive area at the "rcvdData" parameter.
- Output the status of transmission and connection at the output parameter "status"

NOTE Enable Adhoc mode to receive telegrams with dynamic data length. In this case the input parameter "rcvLen" is irrelevant.

Deactivate the Adhoc mode to receive telegrams with fixed data length. In this case you have to specify the number of bytes to be received at the input parameter "rcvLen".

Parameter

Figure 13-3

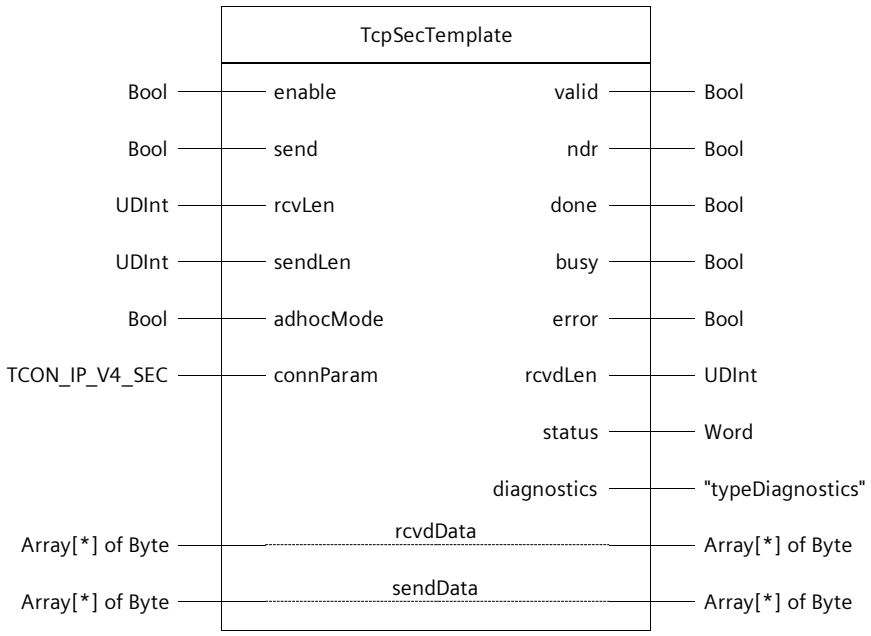


Table 13-4

Name	Declaration	Data type	Comment
enable	Input	Bool	Release signal for connection setup and data exchange
send	Input	Bool	Triggering the send job
rcvLen	Input	UDInt	Receive data length • S7-1500 CPUs: maximum 65536 bytes • S7-1200 CPUs maximum 8192 bytes **Note** Parameter is irrelevant when Adhoc mode is enabled. As much data is read as is currently available. The max. data length is defined by the length of the receive area referenced by rcvData.
sendLen	Input	UDInt	Number of bytes sent with the job. • S7-1500 CPUs: maximum 65536 bytes • S7-1200 CPUs maximum 8192 bytes **Note** If the Adhoc mode is activated, the total length of the telegram is transmitted in the first 4 bytes. These additional 4 bytes must be taken into account at the "sendLen" parameter, i.e. sendLen = 4 byte + user data
adhocMode	Input	Bool	TRUE (Adhoc enabled): • The data is available immediately. • Data reception with dynamic data length FALSE (Adhoc disabled): • The data is available as soon as the data length specified at parameter LEN has been received completely. • Data reception with specified data length.
connParam	Input	TCON_IP_V4_SEC	Connection parameters (see TIA Portal information system)
valid	Output	Bool	TRUE: The block executes its function without error
ndr	Output	Bool	TRUE: New data has been received
done	Output	Bool	TRUE: The send job was successfully executed
busy	Output	Bool	TRUE: The block is executed
error	Output	Bool	TRUE: An error has occurred
rcvdLen	Output	UDInt	Length of received data in bytes
status	Output	Word	Status and error codes (see chapter 13.1.6)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
rcvdData	InOut	Array[*] of bytes	Receive data range
sendData	InOut	Array[*] of bytes	Send data range

© Siemens 2026

13.1.5. UdpTemplate

Functional description

The FB "UdpTemplate" implements a complete UDP communication relation to a partner. It encapsulates all OUC commands in a user-friendly shell module to perform the following functions:

- Connection establishment and termination via the input "enable"
- Send user data at the "sendData" input with the length "sendLen" to the partner as soon as the "send" input detects a positive edge
- Receive data from a partner and output it in the receive area at the "rcvdData" parameter
- Output the status of transmission and connection at the output parameter "status"

Parameter

Figure 13-4

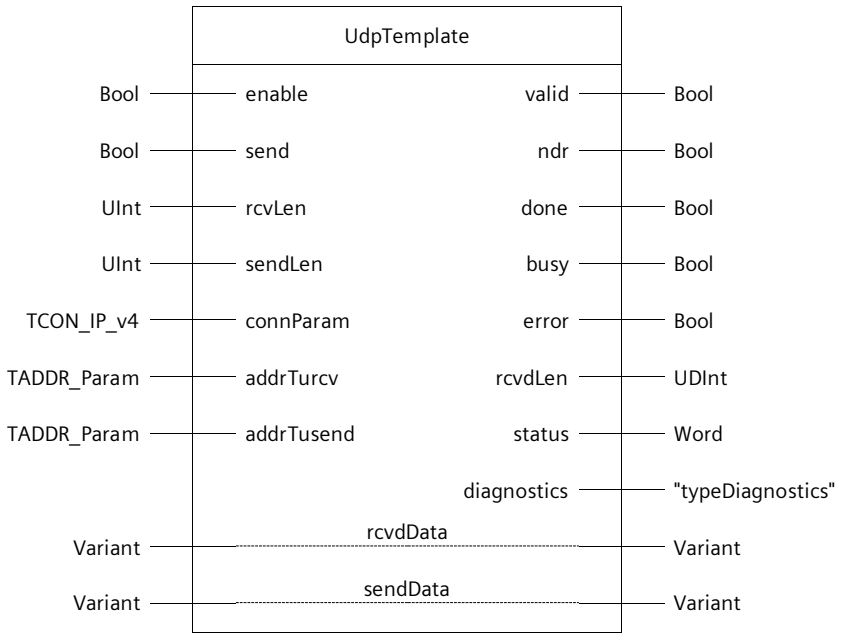


Table 13-5

Name	Declaration	Data type	Comment
enable	Input	Bool	Release signal for connection setup and data exchange
send	Input	Bool	Triggering the send job
rcvLen	Input	UInt	Length of the receive area in bytes: Value range: 0 (recommended) or 1 to 1472 (as of firmware version V2.5 of the S7-1500 CPUs with Unicast or Multicast: 1 to 2048)
sendLen	Input	UInt	Number of bytes that are to be sent with the job Value range: 1 to 1472 (as of firmware version V2.5 of the S7-1500 CPUs with Unicast or Multicast: 1 to 2048)
connParam	Input	TCON_IP_v4	Connection parameters Detailed information can be found in the TIA Portal information system.

© Siemens 2026

Name	Declaration	Data type	Comment
addrTurcv	Input	TADDR_Param	Address information of the communication partner for the "TURCV" command Detailed information can be found in the TIA Portal information system.
addrTusend	Input	TADDR_Param	Address information of the communication partner for the "TSEND" command Detailed information can be found in the TIA Portal information system.
valid	Output	Bool	TRUE: The block executes its function without error
ndr	Output	Bool	TRUE: New data has been received
done	Output	Bool	TRUE: The send job was successfully executed
busy	Output	Bool	TRUE: The block is executed
error	Output	Bool	TRUE: An error has occurred
rcvdLen	Output	UDInt	Length of received data in bytes
status	Output	Word	Status and error codes (see chapter 13.1.6)
diagnostics	Output	"typeDiagnostics"	Advanced diagnostic information (see chapter 14.2)
rcvdData	InOut	Variant	Receive data area
sendData	InOut	Variant	Send data area

NOTE When sending more than 1472 bytes, you must ensure that your receiver supports more than 1472 bytes. If this is not the case, you will not notice the failed reception on the transmitter side.

© Siemens 2026

13.1.6. Error Handling

General information about the status outputs and the diagnostics of the blocks of these libraries can be found in chapter 14.2.

The following are the status and error codes specific to the OUC master copies.

Table 13-6: Status and error codes

Code	Meaning
16#0000	Job completed successfully
16#7000	No job active
16#7001	First call of the command
16#7002	Follow-up call of the command
16#8408	Time-out: The job could not be completed within the specified time. Possible reasons for a time-out: • Partner is not available
16#8600	The FB is in an invalid state.
16#8601	Error in subordinate command "TCON" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system or the STEP 7 online help.
16#8602	Error in subordinate command "TSEND" or "TUSEND" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system or the STEP 7 online help.
16#8603	Error in subordinate command "TRCV" or "TURCV" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system or the STEP 7 online help.
16#8604	Error in subordinate command "TDISCON" The error code of the command is output to "diagnostics.subfunctionStatus". For the meaning of the respective error code, refer to the TIA Portal information system or the STEP 7 online help.

13.2. OPC UA structured data types

Under "Master Copies" you will find the folder "OPC UA data types", which provides you with pre-defined OPC UA structure data types as UDTs. This selection of structured data types is usually used as OPC UA properties and can be mapped using "SiOME" (\9\), for example.

13.3. OPC UA methods-templates

Under "Master copies" you will find the folder "OPC UA templates", which provides ready-made OPC UA methods for you. In the folder you will find two function modules. Both contain the necessary calls to the system function modules "OPC_UA_ServerMethodPre" and "OPC_UA_ServerMethodPost".

Functional sequence

When an OPC UA method is called, the input parameters are passed to the system function "OPC_UA_ServerMethodPre". The system function then provides you with this in the user program. In addition, an interface parameter on the function module signals to you that the method was called by an OPC UA client. Execute your individual function code after the parameter is set. After processing the function code, you must pass the output parameters and the status code to the system function "OPC_UA_ServerMethodPost" and set an interface parameter to signal to the function that the code has been executed. This system function then transmits the output parameters and the status code to the OPC UA client.

"OpcUaMethodTemplateSimple"

In addition to the two system functions, the function module "OpcUaMethodTemplateSimple" contains a predefined region for your individual function code. In addition, after executing the "OPC_UA_ServerMethodPost" system function, the static parameters "finished" and "result" are reset.

"OpcUaMethodTemplateAdvanced "

The number of calls to the system function "OPC_UA_ServerMethodPre" is limited. How many calls are possible depends on the PLC. To reset this limit, the function module "OpcUaMethodTemplateAdvanced" contains an "Enable" handling that allows the function module to be activated and deactivated. In addition, you will find a state machine in the template.

After executing the "OPC_UA_ServerMethodPost" system function, the static parameters "finished" and "result" are reset.

Method limits

	Small	Medium		Large	Large
		S7-1514 SP/S7-1515 (all variants)	S7-1516 (all variants)	S7-1517	S7-1518
Available in address space	20	50	50	4000	8000
Parallel execution	20	20	20	100	200

NOTE The specified limits apply to TIA Portal version V20.

13.4. LSQL_Examples

Under "Master copies" you will find the folder "LSQL_Example", which contains a ready-made configuration of the application example: https://support.industry.siemens.com/cs/ww/en/view/109779336 for LSQL.

The description of the respective Blocks and datatypes can be found in the documentation of the application example.

13.5. LDNS Example

An example of how to configure and use the "LDNS_ResolveDqdn" FB is provided in the master copies templates.

Name	Type	Description
LDNS_ExampleFB	FB	Implementation of an example communication
LDNS_ExampleDB	DB	Data of example communication resolving www.siemens.com

© Siemens 2026

14. Useful Information

14.1. Libraries in TIA Portal

Most of the blocks are stored as types in the library. Thus, the blocks are versioned and can use the following advantages:

- Central update function for library elements
- Versioning of library elements

NOTE For information on library use in general, see the Guide to Library Use:
https://support.industry.siemens.com/cs/ww/en/view/109747503

NOTE All blocks in the library were created in accordance with the programming style guide:
https://support.industry.siemens.com/cs/ww/en/view/81318674

For more information on libraries in the TIA Portal:

- How do you open, edit and upgrade global libraries in the TIA Portal?
  https://support.industry.siemens.com/cs/ww/en/view/37364723
- TIA Portal in under 10 minutes: Time Savers – Global libraries
  https://support.industry.siemens.com/cs/ww/en/view/78529894
- Which elements from STEP 7 (TIA Portal) and WinCC (TIA Portal) can be stored in a library as a type or as a master copy?
  https://support.industry.siemens.com/cs/ww/en/view/109476862
- When starting the TIA Portal V13 and higher, how do you get a global library to open automatically and use it as corporate library, for example?
  https://support.industry.siemens.com/cs/ww/en/view/100451450

© Siemens 2026

14.2. Diagnostics

The blocks use a uniform diagnostics concept according to the programming style guide for SIMATIC S7-1200/1500.

All blocks have the outputs "busy", "error", "status", and "diagnostics". Depending on the type of block (continuous or one-time asynchronous processing), it can also have an output of "valid" or "done".
If the block implements both continuous and one-time asynchronous functions (e.g. connection setup and monitoring and sending a message), it can also have both outputs.

The "valid" or "done", "busy", "error", and "status" outputs show the current status and errors, while the "diagnostics" output provides a diagnostic structure with detailed information in the event of an error.

NOTE **Reset behaviour of the outputs for blocks with "execute"**

As long as the input "execute" is set, the outputs keep their value. If the "execute" input is reset before the processing of the FB is completed, the values of the output parameters are output for one cycle after the job is processed.

For more information, refer to the programming style guide for SIMATIC S7-1200/1500:

https://support.industry.siemens.com/cs/ww/en/view/81318674

status

The "status" output provides current information as well as errors according to the following value range.

Detailed information on the status/error codes can be found in the "Error Handling" chapter of the respective library.

Table 14-1: Value ranges for information

Information	Value range
Job completed, no warning and no further details	16#0000
Job completed, further detailing	16#0001..16#0FFF
No job in process (also initial value)	16#7000
First call after input of a new job (rising edge on "execute")	16#7001
Subsequent call during active processing without further details	16#7002
Follow-up call during active machining with further detail. Occurred warnings that do not further affect the operation.	16#7003..16#7FFF

Table 14-2: Value ranges for errors

Errors	Value range
Incorrect operation of the function block	16#8001..16#81FF
Error in the parameterization	16#8200..16#83FF
Faults when executing from outside (e.g. wrong I/O signals, axis not referenced)	16#8400..16#85FF
Faults when executing internally (e.g. when a system function is called)	16#8600..16#87FF
Reserved	16#8800..16#8FFF

© Siemens 2026

Errors	Value range
User-defined error classes	16#9000..16#FFFF

diagnostics

In the event of an error, the "diagnostics" output gives detailed information about the pending error.

The values of the "diagnostics" output are only written when an error occurs and are retained until a new job is started or another error has occurred and overwrites the values.

Unless otherwise described, the "diagnostics" output uses the PLC data type "typeDiagnostics", which is structured as follows:

Table 14-3: PLC data type typeDiagnostics

Name	Data type	Description
status	Word	Status of the FB at the time of the error.
subfunctionStatus	DWord	Status or returned value from internal commands or FBs where an error occurred. For detailed information, refer to the online help for the respective command or the documentation of the respective FB.
stateNumber	DInt	State of the internal state machine in which the error occurred.

© Siemens 2026

15. Appendix

15.1. Service and Support

SiePortal

The integrated platform for product selection, purchasing and support - and connection of Industry Mall and Online support. The SiePortal home page replaces the previous home pages of the Industry Mall and the Online Support Portal (SIOS) and combines them.

- Products & Services
  In Products & Services, you can find all our offerings as previously available in Mall Catalog.
- Support
  In Support, you can find all information helpful for resolving technical issues with our products.
- mySieportal
  mySiePortal collects all your personal data and processes, from your account to current orders, service requests and more. You can only see the full range of functions here after you have logged in.

You can access SiePortal via this address: sieportal.siemens.com

Technical Support

The Technical Support of Siemens Industry provides you fast and competent support regarding all technical queries with numerous tailor-made offers – ranging from basic support to individual support contracts.
Please send queries to Technical Support via Web form: support.industry.siemens.com/cs/my/src

SITRAIN – Digital Industry Academy

We support you with our globally available training courses for industry with practical experience, innovative learning methods and a concept that's tailored to the customer's specific needs.
For more information on our offered trainings and courses, as well as their locations and dates, refer to our web page: siemens.com/sitrain

Industry Online Support app

You will receive optimum support wherever you are with the "Industry Online Support" app. The app is available for iOS and Android:





© Siemens 2026 |

15.2. Links and Literature

Table 15-1

No.	Topic
\1\	Siemens Industry Online Support https://support.industry.siemens.com
\2\	Link to the entry page of the library https://support.industry.siemens.com/cs/ww/en/view/109780503
\3\	Application example for library "LCom" as well as blocks for S7-300/400 and SIMOTION https://support.industry.siemens.com/cs/ww/en/view/48955385
\4\	Application example for the "LMQTT" library https://support.industry.siemens.com/cs/ww/en/view/109748872
\5\	Application example for OPC UA PubSub UDP https://support.industry.siemens.com/cs/ww/en/view/109782455
\6\	Application example for OPC UA PubSub MQTT https://support.industry.siemens.com/cs/ww/en/view/109797826
\7\	Application example for the "LSNMP" library https://support.industry.siemens.com/cs/ww/en/view/57249109
\8\	Application example for the "LSyslog" library https://support.industry.siemens.com/cs/ww/en/view/51929235
\9\	Siemens OPC UA Modelling Editor (SiOME) https://support.industry.siemens.com/cs/ww/en/view/109755133
\10\	Application example for the "LSQL_Microsoft" library https://support.industry.siemens.com/cs/ww/en/view/109779336

© Siemens 2026

15.3. Change Documentation

NOTE In addition to this change documentation, you'll find a detailed changelog in the respective block.

Version	Date	Change
V2.6.0	09/2026	• LSNMP: New function block "LSNMP_GetOrSet" for secure communication using SNMPv3 • LSQL: - Enabled secure TLS communication with TDS 8.0 - Server responses to Pre-Login and Login are now resolved explicitly - Added "LSQL_AnalyzeTokens" function block used to detect login errors - Other bugfixes and improvements (see changelog in "LSQL_Microsoft" block) • LMQTT: - New function block "LMQTT_Client_MultipleMsg" for processing multiple messages received during the same PLC cycle - Reordered block interface - Added optional "cleanSession" input for resuming previous sessions - Other bugfixes and improvements (see changelog in " LMQTT_Client" block) • LFTP: - Added NOOP command - Added compatibility with IIS FTP Server • LHTTP: - Added feature to extract the headers of an HTTP response in both GET and POST/PUT instructions - Added optional "queryParameters" input to the POST/PUT instruction - Added optional "accept" input to the GET instruction to set the supported content formats • LDNS: Increased the types of DNS responses that are considered valid • LOpcUa: Updated LMQTT_Client to V4.3.0 in LOpcUa_PubMqttUadp, LOpcUa_SubMqttUadp, LOpcUa_PubMqttJson and LOpcUa_SubMqttJson • Master copies: - Added LSNMP master copies for using SNMPv3 - Updated LSQL master copies
V2.5.0	02/2026	• Added "LDNS" library • LMQTT: Bug fix when receiving messages from subscriptions with QoS 2 • LOpcUa: - Updated LMQTT_Client to V4.1.1 in LOpcUa_PubMqttUadp, LOpcUa_SubMqttUadp, LOpcUa_PubMqttJson and LOpcUa_SubMqttJson • LCoAP : Enable block simulation

© Siemens 2026

Version	Date	Change
V2.4.0	12/2025	• Added "LCoAP" library • LMQTT: - Fixed publishing of more than 65535 messages (QoS > 0) - Subscription with QoS2 implemented • LOpcUa: - Updated LMQTT_Client to V4.1.0 in LOpcUa_PubMqttUadp, LOpcUa_SubMqttUadp, LOpcUa_PubMqttJson and LOpcUa_SubMqttJson
V2.3.0	05/2025	• LSNMP_ReceiveTrap function block with corresponding data type added • LOpcUa: Bug fix for LOpcUa_PubMqttJson • Master copies: - Added OpcUaMethodTemplateAdvanced - Added OpcUaMethodTemplateSimple - Added LSQL_Examples
V2.2.0	06/2024	• Added "LSQL" library • LMQTT: Minor bug fix
V2.1.0	04/2024	• Added FC "IsIPAddress" to evaluate whether a partner address in the form of a String contains an IP address or a hostname and integrated it into LFTP, LHTTP & LMQTT • LHTTP: - Changed behaviour that HTTPS is used per default - Improved handling of unusual responses • LMQTT: Fixed area length error when "willMsgLen" = "0" • LOpcUa: - Renamed LOpcUa_PubMqtt to LOpcUaPubMqttUadp and LOpcUa_SubMqtt to LOpcUa_SubMqttUadp - Updated LMQTT_Client to V4.0.3 in LOpcUa_PubMqttUadp, LOpcUa_SubMqttUadp, LOpcUa_PubMqttJson and LOpcUa_SubMqttJson - Enabled simulation support for LOpcUa_SubUdp - Fixed inconsistencies in library • LSNMP: Bug fixes in LSNMP_GetBulk • Removed LMindConn
V2.0.1	09/2023	Minor bug fix in LMQTT
V2.0.0	08/2023	• LHTTP: Improved handling of unusual responses • LMQTT: - Combined addressing the broker with hostname or IP address in one parameter - Bug fixes • LSNMP: Improvements in LSNMP_Get, LSNMP_GetBulk & LSNMP_Set

Version	Date	Change
V1.6.1	04/2023	• LFTP: Bug fixes • LOpcUa: Bug fix in "LOpcUa_PubUdp" and "LOpcUa_Pub_X" FCs
V1.6.0	02/2023	• LCom: - Added QDN as connection type - Added regions in FB "LCom_Communication" • LHTTP: Bug fixes • LMQTT: Bug fixes • Master copies: - Added PLC tags "LCom_Constants" - Added OB "LCom_Example" - Added DB "LCom_ExampleComData" and "LCom_ExampleComBuffers" - Added data type "LCom_typeComUserData" - Added OPC UA structured data types
V1.5.0	12/2022	LOpcUa: • Added "LOpcUa_PubMqttJson" • Added "LOpcUa_SubMqttJson"
V1.4.0	11/2022	• LFTP: - Fixed incompatibility with S7-1200 - Fixed timeout bug • LMQTT: Minor bug fix • LOpcUa: - Minor bug fixes - Optimizations • Deleted "LOpcUa_typeString" and replaced with "WString"
V1.3.0	05/2022	Added library "LFTP"
V1.2.0	01/2022	• LHTTP_Get: Renamed parameter "data" to "queryParams" • LHTTP_PostPut: Added parameter "contentType" to make this header field adjustable • Minor improvements to OPC UA method templates
V1.1.1	09/2021	• Minor bug fixes in "LSNMP" • Fixed references in library for TIA Portal V17

© Siemens 2026

Version	Date	Change
V1.1.0	07/2021	• Added library "LMindConn" • Added library "LSNTP" • Added FBs for PubSub via MQTT to library "LOpcUa" • Master copies: - Revised OUC templates - Added FB "OpcUaMethodTemplateSimple" - Bug fixes in "OpcUaMethodTemplateAdvanced" - Added OPC UA status codes as global constants
V1.0.1	05/2021	LHTTP: • Bug fixes for chunked encoding • Supported chunk size increased
V1.0.0	12/2020	First version

Table 15-2: Change documentation

© Siemens 2026 |